君正®

T23 SDK 开发参考 V2.1

Date: 2024-01

北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

Copyright © 2005-2022 Ingenic Semiconductor Co. Ltd. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Ingenic Semiconductor Co. Ltd.

Trademarks and Permissions

君正 Ingenic 、Ingenic and Ingenic icons are trademarks of Ingenic Semiconductor Co.Ltd. All other trademarks and trade names mentioned in this document are the property of their respective holders.

Disclaimer

All the deliverables and data in this folder serve only as a reference for customer development. Please read through this disclaimer carefully before you use the deliverables and data in this folder. You may use the deliverables in this folder or not. However, by using the deliverables and data in this folder, you agree to accept all the content in this disclaimer unconditional and irrevocable. If you do not find the content in this disclaimer reasonable, you shall not use the deliverables and data in this folder.

The deliverables and data in this folder are provided "AS IS" without representations, guarantees or warranties of any kind (either express or implied). To the maximum extent permitted by law, Ingenic Semiconductor Co., Ltd (Ingenic) provides the deliverables and data in this folder without implied representations, guarantees or warranties, including but not limited to implied representations, guarantees and warranties of merchantability, non-infringement, or fitness for a particular purpose. Deviation of the data provided in this folder may exist under different test environments.

Ingenic takes no liability or legal responsibility for any design and development error, incident, negligence, infringement, and loss (including but not limited to any direct, indirect, consequential, or incidental loss) caused by the use of data in this folder. Users shall be responsible for all risks and consequences caused by the use of data in this folder.

北京君正集成电路股份有限公司
地址：北京市海淀区西北旺东路 10 号院东区 14 号楼君正大厦
电话:**(86-10)56345000**
传真:**(86-10)56345001**
Http: //www.ingenic.cn

前言

概述

本文档主要介绍 Ingenic-T23 软件开发参考以及 T23 模块 API 和数据结构。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
T23 SDK 开发参考	V2.1

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

修订记录累积了每次文档更新的说明。最新版本的文档包含以前所有文档版本的更新内容。

日期	版本	修订章节
2023-06	1.0	第一次正式版本发布
2023-07	1.1	**第 2 章 音频模块** 删除 IMP_AI_SetAlcGain IMP_AI_GetAlcGain IMP_AI_SetDigitalGain IMP_AI_GetDigitalGain IMP_AO_SetDigitalGain IMP_AO_GetDigitalGain

Copyright® 2021-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

		第 3 章 编码模块 新增 IMP_FrameSource_SetDirectModeAttr IMP_FrameSource_GetDirectModeAttr
2024-01	2.1	**第 3 章 编码模块**

Copyright® 2021-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

目录

1 系统控制模块 1
1.1 相关概念 1
1.2 模块绑定（Bind） 2
1.3 System 模块 API 7
1.4 System 数据类型 17

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

1 系统控制模块

系统控制主要实现连接各模块，定义数据流的功能。以下是一些重要概念：

1.1 相关概念

1.1.1 Device

Device 是完成某一（类）功能的集合。如 FrameSource 完成视频源数据的输出，Encoder 完成视频编码或者图像编码的功能。这里的 FrameSource 和 Encoder 就是 Device 的概念。Device 只是一个集合的概念，并不是具体的数据流节点。

1.1.2 Group

Group 是一路数据输入的最小单位。一个 Device 可以有多个 Group，每个 Group 只能接收一路数据输入。

Group 可以有多路输出（1.1.3 Output）。

Group 也是具体功能的容器，可以详见 1.1.5 Channel 部分的解释。

1.1.3 Output

Output 是一个 Group 的一路数据输出的最小单位。一个 Group 可以有多个 Output，每个 Output 只能输出一路数据。

1.1.4 Cell

Cell 指包含了 Device、Group、Output 信息的集合。以 IMPCell 的数据结构呈现。

Cell 主要用来 Bind（bind）。根据 Device、Group、Output 的定义，Output 作为数据输出的节点，而 Group 作为数据输入的节点。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

在 Bind 时数据输出节点的 Cell 索引到输出的 Output，数据输入节点的 Cell 索引到输入的 Group（因此作为数据输入的 Cell，Output 是一个无意义值）。

1.1.5 Channel

Channel 通常指一个单一功能的单元，Channel 在 Create 时(实例化)时被指定具体功能。

例如：

- 对于 Encoder，一个 Channel 完成一路 H264 编码或者 JPEG 编码的功能，具体的编码功能(类型，参数)在通道创建时指定。
- 对于 IVS，一个 Channel 完成一个具体的算法的功能，具体的算法类型参数在通道创建时指定。
- 对于 OSD，有一个同 Channel 类似的概念 Region，一个 Region 是一个具体的叠加区域，可以是 PIC(图像)，COVER(遮挡)等。
- 对于 FrameSource，一个 Channel 输出一路原始图像，FrameSource 的 Channel 实际上就是 Group。

Channel 作为功能单元，通常需要 Register 到 Group 中(FrameSource 除外)，才能接收到数据。Channel 注册到 Group 中后，会得到 Group 输入的数据。

不同 Device 的 Group 可 Register 的 Channel 数也不同。

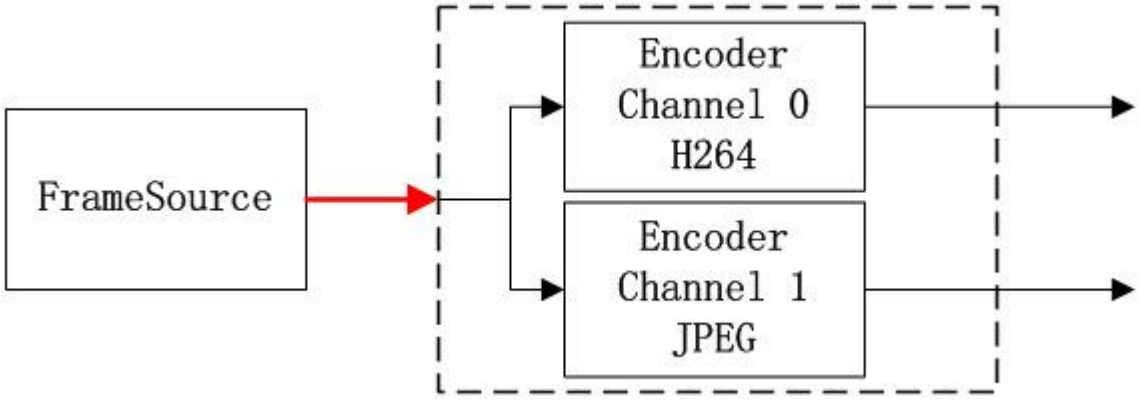
1.2 模块绑定（Bind）

两个 Group 经过 Bind 连接后，源 Group 的数据会自动发到目的 Group。

由于 Group 是数据输入的最小单元，Output 是数据输出的最小单元，因此 IMP_System_Bind(IMPCell *srcCell, IMPCell *dstCell)的两个参数中 srcCell 的 deviceID, groupID, outputID 是有效的 而 dstCell 仅 deviceID 和 groupID 有效，outputID 作为数据输入是无意义的。

下图是一个简单 Bind 的例子。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.



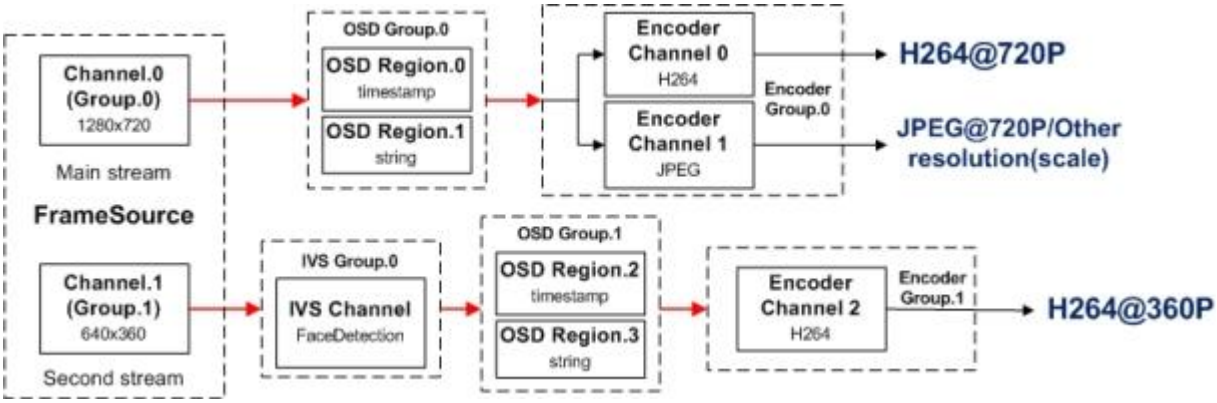
在上图中，实现了 FrameSource 的一路输出 Bind 到 Encoder 的一个 Group。 在 Encoder Group 中 Register 了两个 Channel，因此 Encoder Group 有 H264 和 JPEG 两路输出。 参考代码：

```
IMPCell fs_chn0 = {DEV_ID_FS, 0, 0};
//FrameSource deviceID:DEV_ID_FS groupID:0 outputID:0

IMPCell enc_grp0 = {DEV_ID_ENC, 0, 0};
//ENC deviceID:DEV_ID_ENC groupID:0 outputID:0,这里 enc_grp0 的第三个参数无意义。
int ret = IMP_System_Bind(&fs_chn0, &enc_grp0);
if (ret < 0)
printf("Bind FrameSource Channel0 and Encoder Group0 failed\n");
```

Bind 将系统的数据流串接起来，根据不同的产品功能需求，Bind 的策略也可能不同。

以下是典型双路码流产品应用 Bind 的示意图：

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.



上图中，FrameSource 有两路输出，分别是 Channel0 主码流(1280x720)和 Channel1 从码流(640x360)。

- 主码流：FrameSource 的 Channel0 Bind OSD Group.0， OSD Group.0 Bind Encoder Group.0。其中：

1. OSD Group.0 注册了两个 Region，分别用来显示时间戳和字符串信息；
2. Encoder Group.0 注册了两个 Channel，分别进行 H264 编码和 JPEG 编码。其中 JPEG 编码通道的图像大小如果不等于输入设置（FrameSource 的 Channel0），那么就会进行缩放(Software at T10)，达到任意分辨率抓拍的目的。

- 从码流：FrameSource 的 Channel1 Bind IVS Group.0，IVS Group.0 Bind OSD Group.1，OSD Group.1 Bind Encoder Group.1。其中：

1. IVS Group.0 注册了一个 Channel，用来进行移动侦测；
2. OSD Group.1 注册了两个 Region，分别用来显示时间戳和字符串信息；
3. Encoder Group.1 注册了一个 Channel，进行 H264 编码；
4. 这里值得注意的一点是，IVS Bind 在 OSD 之前，是因为 OSD 的时间戳可能造成 IVS 移动侦测的误判。

参考代码：

主码流数据流 Bind：

```
IMPCell fs_chn0 = {DEV_ID_FS, 0, 0};
IMPCell osd_grp0 = {DEV_ID_OSD, 0, 0};
IMPCell enc_grp0 = {DEV_ID_ENC, 0, 0};
int ret = IMP_System_Bind(&fs_chn0, &osd_grp0);
if (ret < 0)
printf("Bind FrameSource Channel0 and OSD Group0 failed\n");
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
int ret = IMP_System_Bind(&osd_grp0, &enc_grp0);
if (ret < 0)
printf("Bind OSD Group0 and Encoder Group0 failed\n");
```

从码流数据流 Bind：

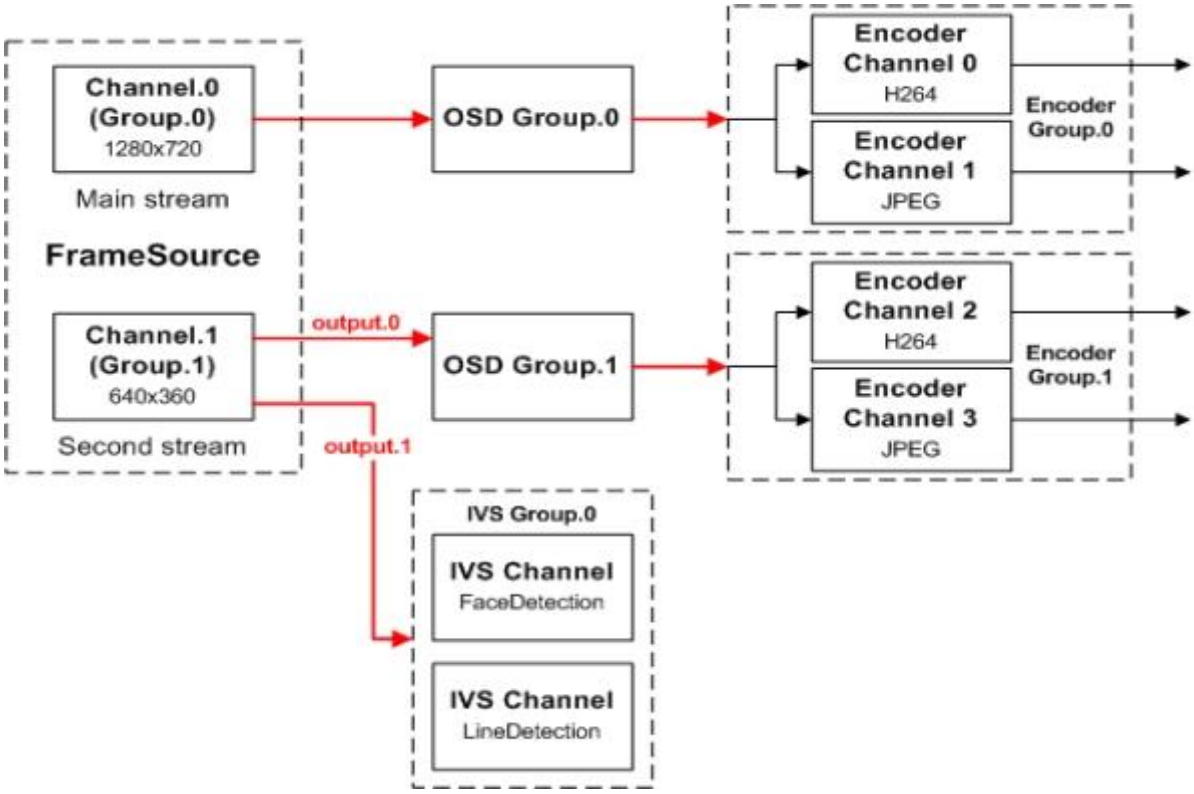
```
 IMPCell fs_chn1_output0 = {DEV_ID_FS, 1, 0};
IMPCell ivs_grp0 = {DEV_ID_IVS, 0, 0};
IMPCell osd_grp1 = {DEV_ID_OSD, 1, 0};
IMPCell enc_grp1 = {DEV_ID_ENC, 1, 0};

int ret = IMP_System_Bind(&fs_chn1_output0, &ivs_grp0);
if (ret < 0)
printf("Bind FrameSource Channel1 and IVS Group0 failed\n");

int ret = IMP_System_Bind(&ivs_grp0, &osd_grp1);
if (ret < 0)
printf("Bind IVS Group0 and OSD Group1 failed\n");

int ret = IMP_System_Bind(&osd_grp1, &enc_grp1);
if (ret < 0)
printf("Bind OSD Group1 and Encoder Group1 failed\n");
```

【注意】

- 建议所有的 Bind 的操作在系统初始化时进行。
- 在 FrameSource 使能后 Bind 和 UnBind 操作不能动态调用，需要 Disable FrameSource 后才可进行 UnBind。
- DestroyGroup 要在 UnBind 之后才能进行。

Bind 可以呈树状结构，下图是一个例子：

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.



上图中，FrameSource 的 Channel 1(Group.1)后端分别 Bind 了两个 Group，分别从 Output.0 和 Output.1 输出数据。本例中这样 Bind 的好处是，IVS Group 可以与 OSD Group.1 并行工作。

【注意】

- 此例中的 Bind 方式可能对普通移动侦测造成影响，因此普通移动侦测不建议采用这种方式。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

1.3 System 模块 API

表 1-1 T23 System API 功能表

API 名称	功能
IMP_System_Init	IMP 系统初始化.
IMP_System_Exit	IMP 系统去初始化.
IMP_System_GetTimeStamp	获得 IMP 系统的时间戳，单位为微秒.
IMP_System_RebaseTimeStamp	设置 IMP 系统的时间戳，单位为微秒.
IMP_System_ReadReg32	读取 32 位寄存器的值.
IMP_System_WriteReg32	向 32 位寄存器中写值.
IMP_System_GetVersion	获取 IMP 系统版本号.
IMP_System_GetCPUInfo	获取 CPU 型号信息.
IMP_System_Bind	绑定源 Cell 和目的 Cell.
IMP_System_UnBind	解除源 Cell 和目的 Cell 的绑定.
IMP_System_GetBindbyDest	获取绑定在目的 Cell 的源 Cell 信息.
IMP_System_MemPoolRequest	在 Rmem 上申请 mempool.

IMP_System_Init

【功能】

IMP 系统初始化。

【语法】

int IMP_System_Init(void);

【形参】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

- 此 API 调用后会初始化基础的数据结构，但不会初始化硬件单元。
- 在 IMP 的任何操作之前必须先调用此接口进行初始化。

IMP_System_Exit

【功能】

IMP 系统去初始化。

【语法】

int IMP_System_Exit(void);

【形参】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

- 此函数调用后会释放 IMP 所有的内存以及句柄，并关闭硬件单元。
- 在调用此 API 后，若要再次使用 IMP 则需重新进行 IMP 系统初始化。

IMP_System_GetTimeStamp

【功能】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

获得 IMP 系统的时间戳，单位为微秒。

【语法】

int64_t IMP_System_GetTimeStamp(void);

【形参】

无。

【返回值】

时间(usec)。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

- 系统初始化后时间戳自动被初始化，系统去初始化后时间戳失效。

IMP_System_RebaseTimeStamp

【功能】

设置 IMP 系统的时间戳，单位为微秒。

【语法】

uint32_t IMP_System_ReadReg32(uint32_t u32Addr);

【形参】

参数名称	描述	输入/输出
basets	基础时间	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

- 此 API 与 IMP_AI_Enable 配套使用,在系统休眠前必须执行 IMP_AI_Disable。

IMP_System_ReadReg32

【功能】

读取 32 位寄存器的值。

【语法】

uint32_t IMP_System_ReadReg32(uint32_t regAddr);

【形参】

参数名称	描述	输入/输出
regAddr	寄存器的物理地址	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_System_WriteReg32

【功能】

向 32 位寄存器中写值。

【语法】

void IMP_System_WriteReg32(uint32_t regAddr, uint32_t value);

【形参】

参数名称	描述	输入/输出
regAddr	寄存器的物理地址	输入
value	要写入的值	输入

【返回值】

无。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

- 在不明确寄存器的含义之前请谨慎调用此 API，否则可能会导致系统错误。

IMP_System_GetVersion

【功能】

获取 IMP 系统版本号。

【语法】

int IMP_System_GetVersion(IMPVersion *pstVersion);

【形参】

参数名称	描述	输入/输出
pstVersion	IMP 系统版本号结构体指针.	输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

无。

IMP_System_GetCPUInfo

【功能】

获取 CPU 型号信息。

【语法】

const char* IMP_System_GetCPUInfo(void);

【形参】

无。

【返回值】

CPU 型号字符串。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

- 返回值是 CPU 型号类型的字符串,例如对于 T10 来说,有"T10"及"T10-Lite"。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_System_Bind

【功能】

绑定源 Cell 和目的 Cell。

【语法】

int IMP_System_Bind(IMPCell *srcCell, IMPCell *dstCell);

【形参】

参数名称	描述	输入/输出
srcCell	源 Cell 指针.	输入
dstCell	目的 Cell 指针.	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

- 根据 Device、Group 和 Output 的概念，每个 Device 可能有多个 Group，每个 Group 可能有多个 Output，Group 作为 Device 的输入接口，而 Output 作为 Device 的输出接口.因此绑定实际上是将输出 Device 的某个 Output 连接到输入 Device 的某个 Group 上。
- 绑定关系成功后，源 Cell(Output)产生的数据会自动传送到目的 Cell(Group)。

IMP_System_UnBind

【功能】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

解除源 Cell 和目的 Cell 的绑定。

【语法】

int IMP_System_UnBind(IMPCell *srcCell, IMPCell *dstCell);

【形参】

参数名称	描述	输入/输出
srcCell	源 Cell 指针.	输入
dstCell	目的 Cell 指针.	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

无。

IMP_System_GetBindbyDest

【功能】

获取绑定在目的 Cell 的源 Cell 信息。

【语法】

int IMP_System_GetBindbyDest(IMPCell *dstCell, IMPCell *srcCell);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

dstCell	目的 Cell 指针.	输入
srcCell	源 Cell 指针.	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

IMP_System_MemPoolRequest

【功能】

在 Rmem 上申请 mempool。

【语法】

int IMP_System_MemPoolRequest(int poolId,size_t size,const char *name);

【形参】

参数名称	描述	输入/输出
poolId	申请的 poolID	输入
size	申请大小	输入
name	pool 名称	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_system.h

库文件：libimp.a / libimp.so

【注意】

- 视频内存池主要向媒体业务提供大块物理连续内存管理功能,负责内存的分配和回收,充分发挥内存缓存池的作用,让物理内存资源在各个媒体处理模块中合理使用。
- 视频内存池建立在原保留内存 RMEM（不由内核建立页表）的基础上进行大块内存管理,每申请一个内存池都是物理连续的内存，之后在该内存池上申请内存同样是申请物理连续的内存。
- 若使用内存池，必须在系统初始化之前配置内存池大小，根据业务的不同,内存池申请的大小和数量不同。

1.4 System 数据类型

名称	定义
IMPVersion	IMP 系统版本号定义

表 1-2 T23 System 数据类型

IMPVersion

【说明】

IMP 系统版本号定义。

【定义】

```
typedef struct {
        char aVersion[64];
} IMPVersion;
```

【成员】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

成员名称	描述
aVersion[64]	IMP 系统版本号

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

目录

2 音频模块.....1
2.1 音频简介.....1
2.2 内核配置.....1
2.3 Audio 驱动.....2
2.4 关键词说明.....3
2.5 功能模块介绍.....4
2.6 功能描述.....4
2.7 AI 模块 API.....7
2.8 AO 模块 API.....38
2.9 AENC 模块 API.....87
2.10 ADEC 模块 API.....96
2.11 AI/AO 数据类型.....107
2.12 AENC 数据类型.....123
2.13 ADEC 数据类型.....126

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2 音频模块

2.1 音频简介

音频使用内部 codec ，内部 codec 只支持模拟 mic。

2.2 内核配置

在内核源码根目录下执行 make menuconfig 命令进入配置界面，按需求进行下面配置。

- Amic 配置(默认已配置)

```
Device Drivers --->
        <*>Sound card support --->
             <*>Open Sound System --->
                [*]Open Sound System of Xburst --->
                  [*]Compile jzsound driver into ko
                  [*] Jz On-Chip I2S driver
                  [*] xburst internal coedc --->
                         [*] t10 internal codec
```

2.3 Audio 驱动

驱动代码位于/opensource/drivers/audio/oss3；在编译前需要确认 Makefile 中的 kernel 路径正确性。驱动参数使用如下。

- Audio 驱动加载时提供了参数 spk_gpio 用于配置功放的使能，驱动默认配置 的是 PB31；如果不需要驱动控制该引脚配置该参数为 spk_gpio=-1。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
# insmod audio.ko spk_gpio=-1
```

- Audio 只支持使用 Amic 功能。如果使用需要内核配置上 Amic 。
- Codec 支持输入 ALC 功能，用于自动控制输入增益；默认开启。驱动加载时提供参数 alc_mode 设置 ALC 开启和关闭。该参数配置成 0 时，关闭输入 ALC 功能，使用手动模式配置输入增益。

ALC 开启后，使用 IMP_AI_SetAlcGain 接口设置输入相对增益。ALC 关闭后，使用 IMP_AI_SetGain 接口手动设置输入增益。

```
# insmod audio.ko alc_mode=0
```

2.4 关键词说明

HPF

HPF(High-Pass Filtering)，高通滤波。

AGC

AGC(Automatic Gain Control)，自动增益控制，用于控制输入输出增益。

AEC

AEC(Acoustic Echo Cancellation)，回声消除。

NS

NS(Nosie Suppression)，噪声抑制。

AO

AO(Audio output),音频输出。

AI

AI(Audio Input)，音频输入。

AENC

AENC(Audio Encode)，音频编码。

ADEC

ADEC(Audio Dncode)，音频解码。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2.5 功能模块介绍

音频功能包含音频输入，音频输出，回音消除，音频编码和音频解码 5 个模块。

其中音频输入和音频输出存在设备和通道的概念。其中一个 MIC 认为是一个 Device，而一个 MIC 可以有多路 Channel 输入。同样的一个 SPK 认为是一个放音 Device，而一个 SPK 也可以有多路 Channel 输出。当前版本的音频 API 一个 Device 只支持一个 Channel。

回音消除位于音频输入接口中，具体说明在功能描述中体现。

音频编码当前音频 API 中支持 PT_G711A、PT_G711U 和 PT_G726 格式音频编码，如需要增加新的编码方式，需要注册编码器。

音频解码当前音频 API 中支持 PT_G711A、PT_G711U 和 PT_G726 格式音频解码，如需要增加新的解码方式，需要注册解码器。

2.6 功能描述

以下是对每个模块的具体功能描述。

2.6.1 音频输入

音频输入 Device ID 对应关系，0: 对应数字 MIC(t23 不支持)　1: 对应模拟 MIC

音频输入 Channel 当前 API 只支持 1 个通道。

音频输入的音量设置，音量的取值范围为[-30 ~ 120]。-30 代表静音，120 表示将声音放大 30dB，步长 0.5dB。其中 60 是音量设置的一个临界点，在这个值上软件不对音量做增加或减小，当音量值小于 60 时，每下降 1，音量减小 0.5dB；当音量值大于 60 时，每增加 1，音量增加 0.5dB。

图 2-1 音频输入增益控制流程图

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2.6.2 音频输出

音频输出 Device ID 对应关系，0：对应默认 SPK　1：对应其他 SPK

音频输出 Channel 当前 API 只支持 1 个通道。

音频输出的音量设置，音量的取值范围为[-30 ~ 120]。-30 代表静音，120 表示将声音放大 30dB，步长 0.5dB。其中 60 是音量设置的一个临界点，在这个值上软件不对音量做增加或减小，当音量值小于 60 时，每下降 1，音量减小 0.5dB；当音量值大于 60 时，每增加 1，音量增加 0.5dB。

图 2-2 音频输出增益控制流程图

2.6.3 回音消除

回音消除属于音频输入接口中的一项功能，所以在使能回音消除时必须先使能音频输入设备和通道。

回音消除当前支持音频采样率为 8K 和 16K，一帧数据采样数为 10ms 音频数据的整数倍（如：8K 采样率，送入的数据为：8000 × 2 / 100 = 160byte 的整数倍）。

回音消除针对不同的设备，不同的封装，回音消除会有不同的效果。

回音消除目前不支持自适应，所以针对不同设备有单独的回音消除参数，回音消除的参数文件位于/etc/webrtc_profile.ini 配置文件中。配置文件格式如下(以下列出主要需要调试的三个参数)：

[Set_Far_Frame]

```
Frame_V=0.3
```

[Set_Near_Frame]

```
Frame_V=0.1
delay_ms=150
```

其中第一个标签[Set_Far_Frame]中的内容代表远端参数，即 SPK 端放音数据参数。Fram_V 代表音频幅值比例，调节该参数可以调节放音数据的幅度(此幅度只用于回音

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

消除)。

第一个标签[Set_Near_Frame]中的内容代表近端参数，即 MIC 端录音数据参数。

Fram_V 代表音频幅值比例，调节该参数可以调节录音数据的幅度(此幅度只用于回音消除)。delay_ms 由于软件和硬件具有延时性，且 SPK 与 MIC 的安放有一定距离，SPK 放音数据会重新被 MIC 采样，所以 SPK 数据在 MIC 数据中体现会存在一定延时。该时间代表放音数据在录音数据中的时间差。

【注意】

- 对于回声消除来说，所有的配置参数全部都在开发板中的/etc/webrtc_profile.ini 文件中（每个参数含义，请参考具体的手册），应用层调用了 IMP_AI_EnableAec 函数之后回声消除即生效（喇叭有没有开启无所谓，只要此函数被调用了，就会进行回音消除）。回声消除之中包含了 AGC（可以控制录音的音量值），NS（降噪），HPF（高通滤波）三个功能，对应于配置文件中的[AGC]，[HP]，[NS]三项，true 即为开启此功能，false 即为关闭此功能，默认都开启，NS 的等级在配置文件中分别为：kLow，kModerate，kHigh，kVeryHigh 四个等级。AGC，NS，HPF 有单独的函数可以开启，函数只用于单工情况，这里要注意，如果 AEC 开启了，就无需再调用单独开启 AGC，NS，HPF 的函数了，因为配置文件中使能了的话，就直接在 AEC 中进行处理数据。

2.6.4 音频编码

音频编码目前音频 API 支持 PT_G711A、PT_G711U 和 PT_G726 格式音频编码，如果需要增加新的编码方式，需要调用 IMP_AENC_RegisterEncoder 接口进行注册编码器。

2.6.5 音频解码

音频解码目前音频 API 支持 PT_G711A、PT_G711U 和 PT_G726 格式音频解码，

如果需要增加新的解码方式，需要调用 IMP_ADEC_RegisterDecoder 接口进行注册解码器。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2.7 AI 模块 API

表 2-1 T23 AI API 功能表

API 名称	功能
IMP_AI_SetPubAttr	设置 AI 设备属性。
IMP_AI_GetPubAttr	获取 AI 设备属性。
IMP_AI_Enable	启用 AI 设备。
IMP_AI_Disable	禁用 AI 设备。
IMP_AI_EnableChn	启用 AI 通道。
IMP_AI_DisableChn	禁用 AI 通道。
IMP_AI_PollingFrame	Polling AI 通道音频流缓存。
IMP_AI_GetFrame	获取 AI 通道音频帧。
IMP_AI_ReleaseFrame	释放 AI 通道音频帧。
IMP_AI_SetChnParam	设置 AI 通道参数。
IMP_AI_GetChnParam	获取 AI 通道参数。
IMP_AI_EnableAec	启用 AI 通道和 AO 通道的回声抵消功能
IMP_AI_DisableAec	禁用 AI 通道回声抵消功能
IMP_AI_EnableNs	启用 AI 通道的噪声抑制功能
IMP_AI_DisableNs	禁用 AI 通道噪声抑制功能
IMP_AI_SetAgcMode	设置 agc 工作模式
IMP_AI_EnableAgc	启用 AI 通道的自动增益功能
IMP_AI_DisableAgc	禁用 AI 通道自动增益功能
IMP_AI_EnableHpf	启用 AI 通道的高通滤波功能
IMP_AI_SetHpfCoFrequency	设置 AI 通道的高通滤波器的截止频率。
IMP_AI_DisableHpf	禁用 AI 通道高通滤波功能
IMP_AI_SetVol	设置 AI 通道的音量。
IMP_AI_GetVol	获取 AI 通道的音量。
IMP_AI_SetVolMute	设置 AI 通道静音。
IMP_AI_SetGain	设置 AI 通道的增益。
IMP_AI_GetGain	获取 AI 通道的增益。
IMP_AI_GetFrameAndRef	获取 AI 通道的音频帧和输出参考帧。
IMP_AI_EnableAecRefFrame	启用 AI 通道获取参考帧功能。
IMP_AI_DisableAecRefFrame	禁用 AI 通道获取参考帧功能。
IMP_AI_EnableAlgo	启用 AI 通道算法功能
IMP_AI_DisableAlgo	禁用 AI 通道算法功能

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AI_SetPubAttr

【功能】

设置 AI 设备属性。

【语法】

int IMP_AI_SetPubAttr(int audioDevId, IMPAudioIOAttr *attr);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
attr	音频输入设备属性指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 需要在 IMP_AI_Enable 前调用。
- 采样率

采样率，也称位采样频率，定义每秒钟从连续信号提取并组成离散信号的个数，单位用赫兹 Hz 来表示。音频的采样率表示每秒的采样点数目。采样率越高，获得的音频数据失真度越低，原始数据量相对越大。T23 内置 codec 支持录音采样率有 8K / 16K / 24K / 32K / 44.1K / 48K / 96K。

- 采样精度

采样精度指的是采样通道中每个采样点的数据宽度。T23 目前只支持 16bit 采样精度录音。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 音频声道模式

  T23 内置 codec 录音支持单声道。

- 缓存帧数目

  内部申请的缓存帧个数。

- 每帧采样点个数

  每个缓存帧采样点个数。

- 支持的通道数目

  单声道录音设为 1。

【举例】

下面的代码实现了初始化 AI 设备，获取音频帧，反初始化 AI 设备。

```
/* Step 1: set public attribute of AI device. */
int devID = 1;
IMPAudioIOAttr attr;
attr.samplerate = AUDIO_SAMPLE_RATE_8000;
attr.bitwidth = AUDIO_BIT_WIDTH_16;
attr.soundmode = AUDIO_SOUND_MODE_MONO;
attr.frmNum = 20;
attr.numPerFrm = 400;
attr.chnCnt = 1;
ret = IMP_AI_SetPubAttr(devID, &attr);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "Set Audio in %d attr err: %d\n", devID, ret);
    return set;
}
```

IMP_AI_GetPubAttr

【功能】

获取 AI 设备属性。

【语法】

int IMP_AI_GetPubAttr(int audioDevId, IMPAudioIOAttr *attr);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

audioDevId	音频设备号	输入
attr	音频输入设备属性指针	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_AI_SetPubAttr 举例。

IMP_AI_Enable

【功能】

启用 AI 设备。

【语法】

int IMP_AI_Enable(int audioDevId);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 调用此函数之前必须调用 IMP_AI_SetPubAttr。

【举例】

请参见 IMP_AI_SetPubAttr 举例。

IMP_AI_Disable

【功能】

禁用 AI 设备。

【语法】

int IMP_AI_Disable(int audioDevId);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 此 API 与 IMP_AI_Enable 配套使用,在系统休眠前必须执行 IMP_AI_Disable。

【举例】

请见 IMP_AI_SetPubAttr 举例。

IMP_AI_EnableChn

【功能】

启用 AI 通道。

【语法】

int IMP_AI_EnableChn(int audioDevId, int aiChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 必须先调用 IMP_AI_EnableDevice，使能设备，才可以启用 AI 通道。

【举例】

请参见 IMP_AI_SetPubAttr 举例。

IMP_AI_DisableChn

【功能】

禁用 AI 通道。

【语法】

int IMP_AI_DisableChn(int audioDevId, int aiChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 与 IMP_AI_EnableChn 配套使用。

【举例】

请参见 IMP_AI_SetPubAttr 举例。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AI_PollingFrame

【功能】

Polling AI 通道音频流缓存。

【语法】

int IMP_AI_PollingFrame(int audioDevId, int aiChn, unsigned int timeout_ms);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
timeout_ms	Polling 超时时间	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 在使用 IMP_AI_GetFrame 之前使用该接口，当该接口调用成功之后表示音频数据已经准备完毕，可以使用 IMP_AI_GetFrame 获取音频数据。

【举例】

请参见 IMP_AI_SetPubAttr 举例。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AI_GetFrame

【功能】

获取 AI 通道音频帧。

【语法】

int IMP_AI_GetFrame(int audioDevId, int aiChn,

IMPAudioFrame *frm, IMPBlock block);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
frm	音频帧结构体指针	输入
block	阻塞和非阻塞标识	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 获取的音频帧之后需要及时释放音频帧，可调用 IMP_AI_ReleaseFrm。

【举例】

示例代码：

```
IMPAudioFrame frm;
  // 获取音频帧
  ret = IMP_AI_GetFrame(devID, chnID, &frm, BLOCK);
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
if(ret != 0) {
    IMP_LOG_ERR(TAG, "Audio Get Frame Data error\n");
    return ret;
}
fwrite(frm.virAddr, 1, frm.len, record_file); // 使用音频帧数据
// 释放音频帧
ret = IMP_AI_ReleaseFrame(devID, chnID, &frm);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "Audio release frame data error\n");
    return ret;
}
```

IMP_AI_ReleaseFrame

【功能】

释放 AI 通道音频帧。

【语法】

int IMP_AI_ReleaseFrame(int audioDevId, int aiChn, IMPAudioFrame *frm);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
frm	音频帧结构体指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

- 与 IMP_AI_GetFrame 配套使用。

【举例】

请参见 IMP_AI_SetPubAttr 举例。

IMP_AI_SetChnParam

【功能】

设置 AI 通道参数。

【语法】

int IMP_AI_SetChnParam(int audioDevId, int aiChn, IMPAudioIChnParam *chnParam);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
chnParam	AI 通道参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 在 IMP_AI_EnableChn 前调用。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

示例代码：

```
1.  int chnID = 0;
2.    IMPAudioIChnParam chnParam;
3.    chnParam.usrFrmDepth = 20;   // the range of valid value is [2, MAX_AUDIO_FRAME_
    NUM].
4.    ret = IMP_AI_SetChnParam(devID, chnID, &chnParam);
5.    if(ret != 0) {
6.          IMP_LOG_ERR(TAG, "set ai %d channel %d attr err: %d\n", devID, chnID, ret);

7.        return ret;
8.   }
```

IMP_AI_GetChnParam

【功能】

获取 AI 通道参数。

【语法】

int IMP_AI_GetChnParam(int audioDevId, int aiChn, IMPAudioIChnParam *chnParam);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
chnParam	AI 通道参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

请参见 IMP_AI_SetPubAttr 举例。

IMP_AI_EnableAec

【功能】

启用 AI 通道和 AO 通道的回声抵消功能。

【语法】

int IMP_AI_EnableAec(int aiDevId, int aiChn, int aoDevId, int aoChn);

【形参】

参数名称	描述	输入/输出
audioDevId	需要进行回声抵消的音频输入设备号	输入
aiChn	需要进行回声抵消的音频输入通道号	输入
aoDevId	需要进行回声抵消的音频输出设备号	输入
aoChn	需要进行回声抵消的音频输出通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 回音消除针对不同的设备，不同的封装，回音消除会有不同的效果。

【举例】

请参见 sample-Ai-AEC 举例。

IMP_AI_DisableAec

【功能】

禁用 AI 通道回声抵消功能。

【语法】

int IMP_AI_DisableAec(int aiDevId, int aiChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频输入设备号	输入
aiChn	音频输入通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 与 IMP_AI_EnableAec 配合使用。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

请参见 sample-Ai-AEC 举例。

IMP_AI_EnableNs

【功能】

启用 AI 通道的噪声抑制功能。

【语法】

int IMP_AI_EnableNs(IMPAudioIOAttr *attr, int mode);

【形参】

参数名称	描述	输入/输出
attr	需要进行噪声抑制的音频属性	输入
mode	噪声抑制的级别 0 ～ 3	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ； libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 噪声抑制的 mode 参数表示噪声抑制的级别,范围为[0 ~ 3],级别越高,噪声抑制的越干净。然而,噪声抑制的越干净同时也就会丢失更多的声音细节,所以这里有一个矛盾点,需要在使用的时候进行权衡。
- 回声消除包含了噪声抑制功能,如果启用回声抵消,就不需要做噪声抑制。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

请参见 sample-Audio 举例。

IMP_AI_DisableNs

【功能】

禁用 AI 通道噪声抑制功能。

【语法】

int IMP_AI_DisableNs(void);

【形参】

无

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 与 IMP_AI_EnableNs 配合使用。

【举例】

请参见 sample-Audio 举例。

IMP_AI_SetAgcMode

【功能】

设置 agc 工作模式。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_AI_SetAgcMode(int mode);

【形参】

参数名称	描述	输入/输出
mode	agc 工作模式选择	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ； libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 需要注意 agc mode 设置需要在使能 agc 功能之前。

【举例】

无。

IMP_AI_EnableAgc

【功能】

获取 AI 设备属性。

【语法】

int IMP_AI_EnableAgc(IMPAudioIOAttr *attr, IMPAudioAgcConfig agcConfig);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

attr	需要进行自动增益控制的音频属性	输入
agcConfig	自动增益的参数配置,配置放大倍数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 需要注意的是,AGC 可以将声音的增益放大,但是如果增益的参数不合适,就会导致破音等情况,请在具体使用时自行调整。
- 回声消除包含了 AGC 功能,如果启用回声抵消,就不需要做自动增益。

【举例】

请参见 sample-Audio 举例。

IMP_AI_DisableAgc

【功能】

禁用 AI 通道自动增益功能。

【语法】

int IMP_AI_DisableAgc(void);

【形参】

无。

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 与 IMP_AI_EnableAgc 配合使用。

【举例】

请参见 sample-Audio 举例。

IMP_AI_EnableHpf

【功能】

启用 AI 通道的高通滤波功能。

【语法】

int IMP_AI_EnableHpf(IMPAudioIOAttr *attr);

【形参】

参数名称	描述	输入/输出
attr	需要启用高通滤波【功能】的音频属性	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 回声消除包含了 HPF 功能,如果启用回声抵消,就不需要做 HPF。

【举例】

请参见 sample-Audio 举例。

IMP_AI_DisableHpf

【功能】

禁用 AI 通道高通滤波功能。

【语法】

```
int IMP_AI_DisableHpf(void);
```

【形参】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 此接口与 IMP_AI_EnableHpf 配合使用。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

请参见 sample-Audio 举例。

IMP_AI_SetHpfCoFrequency

【功能】

设置 AI 通道的高通滤波器的截止频率。

【语法】

int IMP_AI_SetHpfCoFrequency(int cofrequency);

【形参】

参数名称	描述	输入/输出
cofrequency	截止频率	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 此接口需要在 IMP_AI_EnableHpf 之前调用。

【举例】

请参见 sample-Audio 举例。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AI_SetVol

【功能】

设置 AI 通道的音量。

【语法】

int IMP_AI_SetVol(int audioDevId, int aiChn, int aiVol);

【形参】

参数名称	描述	输入/输出
audioDevId	音频输入设备号	输入
aiChn	音频输入通道号	输入
aiVol	音频输入音量大小	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 音量的取值范围为[-30 ~ 120]. -30 代表静音,120 表示将声音放大 30dB,步长 0.5dB。其中 60 是音量设置的一个临界点，在这个值上软件不对音量做增加或减小，当音量值小于 60 时，每下降 1，音量减小 0.5dB；当音量值大于 60 时，上增加 1，音量增加 0.5dB。
- 如果输入的 aiVol 超过了[-30 ~ 120]的范围,小于-30 的将会取-30,大于 120 的取 120。

【举例】

示例代码：

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
int volume = 60;
 ret = IMP_AI_SetVol(devID, chnID, volume);  if(ret != 0) {
      IMP_LOG_ERR(TAG, "Audio Record set volume failed\n");
      return ret;
  }
```

IMP_AI_GetVol

【功能】

获取 AI 通道的音量。

【语法】

int IMP_AI_GetVol(int audioDevId, int aiChn, int *vol);

【形参】

参数名称	描述	输入/输出
audioDevId	音频输入设备号	输入
aiChn	音频输入通道号	输入
aiVol	音频输入音量大小	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

请参见 IMP_AI_SetPubAttr 举例。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AI_SetVolMute

【功能】

设置 AI 通道静音。

【语法】

int IMP_AI_SetVolMute(int audioDevId, int aiChn, int mute);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
mute	音频输入静音标志，mute = 0:关闭静音, mute = 1:打开静音。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

无。

IMP_AI_SetGain

【功能】

设置 AI 通道的增益。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_AI_SetGain(int audioDevId, int aiChn, int aigain);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
aigain	音频通道输入增益，范围[0 ~ 31],对应[-18dB ~ 28.5dB],步长 1.5dB。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 如果使用的时双声道的话，两个通道增益一致。aiGain 的范围为[0 ~ 31]，如果输入的值小于 0，则 aiGain 的值将会被设置为 0。如果值大于 31,aiGain 的值会被设置为 10。

【举例】

请参见 IMP_AI_SetPubAttr 举例。

IMP_AI_GetGain

【功能】

获取 AI 通道的增益。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_AI_GetGain(int audioDevId, int aiChn, int *aigain);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
aigain	音频通道输入增益，范围[0 ~ 31],对应[-18dB ~ 28.5dB],步长 1.5dB。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

请参见 IMP_AI_SetPubAttr 举例。

IMP_AI_GetFrameAndRef

【功能】

获取 AI 通道的音频帧和输出参考帧。

【语法】

int IMP_AI_GetFrameAndRef(int audioDevId, int aiChn,

IMPAudioFrame *frm, IMPAudioFrame *ref, IMPBlock block);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
frm	音频帧结构体指针	输出
ref	参考帧结构体指针	输出
block	阻塞/非阻塞标识	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

下面的代码实现了获取 AI 通道的音频帧和输出参考帧。

```
//......
int ret = -1;
ret = IMP_AI_EnableAecRefFrame(devID, chnID, 0, 0);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "Audio Record enable channel failed\n");
    return NULL;
}

IMPAudioFrame frm;
IMPAudioFrame ref;
// 获取音频帧和输出参考帧
ret = IMP_AI_GetFrameAndRef(devID, chnID, &frm, &ref, BLOCK);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "Audio Get Frame Data error\n");
    return ret;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
16. }
17.
18. fwrite(frm.virAddr, 1, frm.len, record_file); // 使用音频帧数据
19. fwrite(ref.virAddr, 1, ref.len, ref_file); // 使用音频参考帧
20.
21. // 释放音频帧
22. ret = IMP_AI_ReleaseFrame(devID, chnID, &frm);
23. if(ret != 0) {
24.     IMP_LOG_ERR(TAG, "Audio release frame data error\n");
25.     return ret;
26. }
```

IMP_AI_EnableAecRefFrame

【功能】

打开获取参考帧。

【语法】

int IMP_AI_EnableAecRefFrame(int audioDevId,

int aiChn, int audioAoDevId, int aoChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
aoDevId	音频输出设备号	输入
aoChn	音频输出通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_AI_GetFrameAndRef 举例。

IMP_AI_DisableAecRefFrame

【功能】

关闭获取参考帧。

【语法】

int IMP_AI_DisableAecRefFrame(int audioDevId,

int aiChn, int audioAoDevId, int aoChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入
aoDevId	音频输出设备号	输入
aoChn	音频输出通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 此接口和 IMP_AI_EnableAecRefFrame 配合使用。

【举例】

请参见 IMP_AI_GetFrameAndRef 举例。

IMP_AI_EnableAlgo

【功能】

使能 AI 算法功能。

【语法】

int IMP_AI_EnableAlgo(int audioDevId, int aiChn,);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 此接口可开启 NS、AGC、HPF 功能，是否开启及参数的配置信息在 webrtc.ini 中设置。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AI_DisableAlgo

【功能】

禁止 AI 算法功能。

【语法】

int IMP_AI_DisableAlgo(int audioDevId, int aiChn,);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aiChn	音频输入通道号	输入

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 此接口和 IMP_AI_EnableAlgo 配合使用。

2.8 AO 模块 API

表 2-2 T23 AO API 功能表

API 名称	功能
IMP_AO_SetPubAttr	设置 AO 设备属性
IMP_AO_GetPubAttr	获取 AO 设备属性
IMP_AO_Enable	启用 AO 设备
IMP_AO_Disable	禁用 AO 设备
IMP_AO_EnableChn	启用 AO 通道
IMP_AO_DisableChn	禁用 AO 通道
IMP_AO_SendFrame	向 AO 通道发送音频帧
IMP_AO_PauseChn	暂停 AO 通道
IMP_AO_ResumeChn	恢复 AO 通道
IMP_AO_ClearChnBuf	清除 AO 通道中的音频数据缓存
IMP_AO_QueryChnStat	查询 AO 通道音频缓存状态
IMP_AO_SetVol	设置 AO 通道音量
IMP_AO_GetVol	获取 AO 通道音量

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AO_SetGain	设置 AO 通道增益
IMP_AO_GetGain	获取 AO 通道增益
IMP_AO_Soft_Mute	设置 AO 通道软件静音
IMP_AO_Soft_UNMute	撤销 AO 通道软件静音
IMP_AO_CacheSwitch	关闭 AO 通道音频缓存机制
IMP_AO_FlushChnBuf	等待 AO 通道音频数据播放完
IMP_AO_EnableAgc	启用 AO 通道音频自动增益功能
IMP_AO_DisableAgc	禁用 AO 通道音频自动增益功能
IMP_AO_EnableHpf	启用 AO 通道高通滤波功能
IMP_AO_SetHpfCoFrequency	设置 AO 通道高通滤波器的截止频率
IMP_AO_DisableHpf	禁用 AO 通道高通滤波功能
IMP_AO_EnableAlgo	启用 AO 算法功能
IMP_AO_DisableAlgo	禁用 AO 算法功能

IMP_AO_SetPubAttr

【功能】

设置 AO 设备属性。

【语法】

int IMP_AO_SetPubAttr(int audioDevId, IMPAudioIOAttr *attr);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
attr	音频 AO 输出设备属性指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

- 音频输出设备属性包括音频采样率、采样精度、音频声道模式、codec 编码通道配置、缓存帧的数目、每帧采样点个数和支持的通道数目。
- 采样率

  T23 内置 codec 支持放音采样率有 8K / 16K / 24K / 32K / 44.1K / 48K / 96K。
- 采样精度

  T23 目前只支持 16bit 采样精度放音。
- 音频声道模式

  T23 放音只支持单声道。
- Codec 编码通道

  T23 放音只支持单声道。设置为 1 即可。
- 缓存帧数目

  内部申请的缓存帧个数。
- 每帧采样点个数

  每个缓存帧采样点个数。
- 支持的通道数目

  默认设为 1。

【举例】

下面的代码实现了初始化 AO 设备，发送音频帧，反初始化 AO 设备

```
unsigned char *buf = NULL;
int size = 0;
int ret = -1;

buf = (unsigned char *)malloc(640);
if (buf == NULL) {
    IMP_LOG_ERR(TAG, "[ERROR] %s: malloc audio buf error\n", __func__);
    return NULL;
}

FILE *play_file = fopen("play.pcm", "rb");
if (play_file == NULL) {
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
13.     IMP_LOG_ERR(TAG, "[ERROR] %s: fopen %s failed\n", __func__, AO_BASIC_TEST_PLAY_FI
    LE);
14.     return NULL;
15. }
16.
17. /* Step 1: set public attribute of AO device. */
18. int devID = 0;
19. IMPAudioIOAttr attr;
20. attr.samplerate = AUDIO_SAMPLE_RATE_16000;
21. attr.bitwidth = AUDIO_BIT_WIDTH_16;
22. attr.soundmode = AUDIO_SOUND_MODE_MONO;
23. attr.frmNum = 20;
24. attr.numPerFrm = 640;
25. attr.chnCnt = 1;
26. ret = IMP_AO_SetPubAttr(devID, &attr);
27. if (ret != 0) {
28.     IMP_LOG_ERR(TAG, "set ao %d attr err: %d\n", devID, ret);
29.     return NULL;
30. }
31.
32. memset(&attr, 0x0, sizeof(attr));
33. ret = IMP_AO_GetPubAttr(devID, &attr);
34. if (ret != 0) {
35.     IMP_LOG_ERR(TAG, "get ao %d attr err: %d\n", devID, ret);
36.     return NULL;
37. }
38.
39. /* Step 2: enable AO device. */
40. ret = IMP_AO_Enable(devID);
41. if (ret != 0) {
42.     IMP_LOG_ERR(TAG, "enable ao %d err\n", devID);
43.     return NULL;
44. }
45.
46. /* Step 3: enable AI channel. */
47. int chnID = 0;
48. ret = IMP_AO_EnableChn(devID, chnID);
49. if (ret != 0) {
50.     IMP_LOG_ERR(TAG, "Audio play enable channel failed\n");
51.     return NULL;
52. }
53.
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
/* Step 4: Set audio channel volume. */
int chnVol = 80;
ret = IMP_AO_SetVol(devID, chnID, chnVol);
if (ret != 0) {
    IMP_LOG_ERR(TAG, "Audio Play set volume failed\n");
    return NULL;
}

ret = IMP_AO_GetVol(devID, chnID, &chnVol);
if (ret != 0) {
    IMP_LOG_ERR(TAG, "Audio Play get volume failed\n");
    return NULL;
}
IMP_LOG_INFO(TAG, "Audio Out GetVol    vol:%d\n", chnVol);

int aogain = 28;
ret = IMP_AO_SetGain(devID, chnID, aogain);
if (ret != 0) {
    IMP_LOG_ERR(TAG, "Audio Record Set Gain failed\n");
    return NULL;
}

ret = IMP_AO_GetGain(devID, chnID, &aogain);
if (ret != 0) {
    IMP_LOG_ERR(TAG, "Audio Record Get Gain failed\n");
    return NULL;
}
IMP_LOG_INFO(TAG, "Audio Out GetGain    gain : %d\n", aogain);

  int i = 0;
  while (1) {
    size = fread(buf, 1, AO_TEST_BUF_SIZE, play_file);
    if (size < AO_TEST_BUF_SIZE)
        break;

    /* Step 5: send frame data. */
    IMPAudioFrame frm;
    frm.virAddr[0] = (uint32_t *)buf;
    frm.virAddr[1] = NULL;
    frm.len = size;
    ret = IMP_AO_SendFrame(devID, chnID, &frm, BLOCK);
    if (ret != 0) {
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
        IMP_LOG_ERR(TAG, "send Frame Data error\n");
        return NULL;
    }
  }
ret = IMP_AO_FlushChnBuf(devID, chnID);
if (ret != 0) {
    IMP_LOG_ERR(TAG, "IMP_AO_FlushChnBuf error\n");
    return NULL;
}
/* Step 6: disable the audio channel. */
ret = IMP_AO_DisableChn(devID, chnID);
if (ret != 0) {
    IMP_LOG_ERR(TAG, "Audio channel disable error\n");
    return NULL;
}

/* Step 7: disable the audio devices. */
ret = IMP_AO_Disable(devID);
if (ret != 0) {
    IMP_LOG_ERR(TAG, "Audio device disable error\n");
    return NULL;
}
fclose(play_file);
free(buf);
```

IMP_AO_GetPubAttr

【功能】

设置 AO 设备属性。

【语法】

int IMP_AO_GetPubAttr(int audioDevId, IMPAudioIOAttr *attr);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
attr	音频 AO 输出设备属性指针	输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

请参见 MP_AO_SetPubAttr 举例。

IMP_AO_Enable

【功能】

启用 AO 设备。

【语法】

int IMP_AO_Enable(int audioDevId);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_AO_SetPubAttr 举例。

IMP_AO_Disable

【功能】

禁用 AO 设备。

【语法】

int IMP_AO_Disable(int audioDevId);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
attr	音频 AO 输出设备属性指针	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 此接口需要和 IMP_AO_Enable 配合使用。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

请参见 IMP_AO_SetPubAttr 举例。

IMP_AO_EnableChn

【功能】

启用 AO 通道。

【语法】

int IMP_AO_EnableChn(int audioDevId, int aoChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_AO_SetPubAttr 举例。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AO_DisableChn

【功能】

禁用 AO 通道。

【语法】

int IMP_AO_DisableChn(int audioDevId, int aoChn)

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 此接口需要和 IMP_AO_EnableChn 配合使用。

【举例】

请参见 IMP_AO_SetPubAttr 举例。

IMP_AO_SendFrame

【功能】

向 AO 通道发送音频帧。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_AO_SendFrame(int audioDevId, int aoChn,

IMPAudioFrame *data, IMPBlock block);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入
data	音频帧结构体指针	输入
block	阻塞/非阻塞标识	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

示例代码：

```
while(1) {
        size = fread(buf, 1, IMP_AUDIO_BUF_SIZE, play_file);
      if(size < IMP_AUDIO_BUF_SIZE)
            break;

        IMPAudioFrame frm;
        frm.virAddr = (uint32_t *)buf;
        frm.len = size;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
        ret = IMP_AO_SendFrame(devID, chnID, &frm, BLOCK);
        if(ret != 0) {
            IMP_LOG_ERR(TAG, "send Frame Data error\n");
            return ret;
        }
    }
```

IMP_AO_PauseChn

【功能】

暂停 AO 通道。

【语法】

int IMP_AO_PauseChn(int audioDevId, int aoChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

无。

IMP_AO_ResumeChn

【功能】

恢复 AO 通道。

【语法】

int IMP_AO_ResumeChn(int audioDevId, int aoChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

无。

IMP_AO_ClearChnBuf

【功能】

清除 AO 通道中的音频数据缓存。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_AO_ClearChnBuf(int audioDevId, int aoChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_AO_QueryChnStat

【功能】

查询 AO 通道音频缓存状态。

【语法】

int IMP_AO_QueryChnStat(int audioDevId, int aoChn, IMPAudioOChnState *status);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入
status	缓存状态结构体指针	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_AO_SetVol

【功能】

设置 AO 通道音量。

【语法】

```
int IMP_AO_SetVol(int audioDevId, int aoChn, int aoVol);
```

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

aoChn	音频 AO 设备通道号	输入
aoVol	音频输出音量	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

请参见 IMP_AO_SetPubAttr 举例。

IMP_AO_GetVol

【功能】

获取 AO 通道音量。

【语法】

```
int IMP_AO_GetVol(int audioDevId, int aoChn, int *vol);
```

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入
vol	音频输出音量指针	输出

【返回值】

返回值	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

请参见 IMP_AO_SetPubAttr 举例。

IMP_AO_SetGain

【功能】

设置 AO 通道模拟增益。

【语法】

int IMP_AO_SetGain(int audioDevId, int aoChn, int aoGain);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入
aoGain	音频 AO 输出模拟增益	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_AO_SetPubAttr 举例。

IMP_AO_GetGain

【功能】

获取 AO 通道模拟增益。

【语法】

```
int IMP_AO_GetGain(int audioDevId, int aoChn, int *aoGain);
```

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入
aoGain	音频 AO 输出模拟增益指针	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

请参见 IMP_AO_SetPubAttr 举例。

IMP_AO_Soft_Mute

【功能】

设置 AO 通道软件静音。

【语法】

int IMP_AO_SetVolMute(int audioDevId, int aoChn, int mute);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 调用该接口不会立马静音，会从正常放音状态缓慢将音量降低，直到真正静音。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AO_Soft_UNMute

【功能】

撤销 AO 通道软件静音。

【语法】

int IMP_AO_Soft_UNMute(int audioDevId, int aoChn);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_AO_CacheSwitch

【功能】

关闭 AO 通道音频缓存机制。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_AO_CacheSwitch(int audioDevId,int aoChn, int cache_en);

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入
cache_en	缓存机制开关	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_AO_FlushChnBuf

【功能】

等待 AO 通道音频数据播放完。

【语法】

int IMP_AO_FlushChnBuf(int audioDevId, int aoChn);

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【形参】

参数名称	描述	输入/输出
audioDevId	音频 AO 设备号	输入
aoChn	音频 AO 设备通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【举例】

请参见 IMP_AO_SetPubAttr 举例。

IMP_AO_EnableAgc

【功能】

启用 AO 通道音频自动增益功能。

【语法】

int IMP_AO_EnableAgc(IMPAudioIOAttr *attr, IMPAudioAgcConfig agcConfig);

【形参】

参数名称	描述	输入/输出
attr	需要进行自动增益的音频属性	输入
agcConfig	自动增益的参数配置,配置放大倍数	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 需要注意 agcConfig 的配置，AGC 的放大倍数主要有该参数配置，具体增益见 IMPAudioAgcConfig 说明。AGC 可以将声音的增益放大，但是如果增益的参数不合适，就会导致破音等情况，请在具体使用时自行调整。回声消除包含了 AGC 功能，如果启用回声抵消，就不需要做自动增益。

【举例】

无。

IMP_AO_DisableAgc

【功能】

禁用 AO 通道音频自动增益控制功能。

【语法】

int IMP_AO_DisableAgc(void);

【形参】

无。

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 此接口与 IMP_AO_DisableAgc 配合使用。

【举例】

无。

IMP_AO_EnableHpf

【功能】

启用 AO 通道高通滤波功能

【语法】

int IMP_AI_EnableHpf(IMPAudioIOAttr *attr);

【形参】

参数名称	描述	输入/输出
attr	需要进行高通滤波的音频属性	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ； libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 回声消除包含了 HPF 功能,如果启用回声抵消,就不需要做 HPF。

【举例】

无。

IMP_AO_SetHpfCoFrequency

【功能】

设置 AO 通道高通滤波器的截止频率。

【语法】

int IMP_AI_SetHpfCoFrequency(int cofrequency);

【形参】

参数名称	描述	输入/输出
cofrequency	高通滤波的截止频率	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ； libaudioProcess.so

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

配置文件：webrtc_profile.ini

【注意】

- 开启高通滤波器之前先设置截止频率。

【举例】

无。

IMP_AO_DisableHpf

【功能】

禁用 AO 通道高通滤波功能。

【语法】

int IMP_AI_DisableHpf(void);

【形参】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so ；libaudioProcess.so

配置文件：webrtc_profile.ini

【注意】

- 此接口与 IMP_AO_EnableHpf 配合使用。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AO_EnableAlgo

【功能】

使能 AO 算法功能。

【语法】

int IMP_AO_EnableAlgo(int audioDevId, int aoChn,);

【形参】

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aoChn	音频输出通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 此接口可开启 NS、AGC、HPF 功能，是否开启及参数的配置信息在 webrtc.ini 中设置。

IMP_AO_DisableAlgo

【功能】

禁止 AO 算法功能。

【语法】

int IMP_AO_DisableAlgo(int audioDevId, int aoChn,);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
audioDevId	音频设备号	输入
aoChn	音频输入通道号	输入

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 此接口和 IMP_AO_EnableAlgo 配合使用。

2.9 AENC 模块 API

表 2-4 T23 AENC API 功能表

API 名称	功能
IMP_AENC_CreateChn	创建音频编码通道
IMP_AENC_DestroyChn	销毁音频编码通道
IMP_AENC_SendFrame	发送音频编码音频帧
IMP_AENC_PollingStream	Polling 编码音频流缓存
IMP_AENC_GetStream	获取编码后码流
IMP_AENC_ReleaseStream	释放从音频编码通道获取的码流
IMP_AENC_RegisterEncoder	注册编码器
IMP_AENC_UnRegisterEncoder	注销编码器

IMP_AENC_CreateChn

【功能】

创建音频编码通道。

【语法】

int IMP_AENC_CreateChn(int aeChn, IMPAudioEncChnAttr *attr);

【形参】

参数名称	描述	输入/输出
aeChn	通道号	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

attr	音频编码通道属性指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 目前 SDK 支持 PT_G711A、PT_G711U 和 PT_G726 编码. 所以使用 SDK 中的编码,只需要 attr.type = PT_G711A 即可.
- 如果需要使用自定义的编码器,则需要注册编码器,示例代码在 IMP_AENC_RegisterEncoder 中说明。

【举例】

下面代码实现了创建编码通道，销毁编码通道的操作。

```
int AeChn = 0;
IMPAudioEncChnAttr attr;
attr.type = PT_G711A;
attr.bufSize = 20;
ret = IMP_AENC_CreateChn(AeChn, &attr);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "Audio encode create channel failed\n");
    return ret;
}
//........
ret = IMP_AENC_DestroyChn(AeChn);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "Audio encode destroy channel failed\n");
    return ret;
}
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_AENC_DestroyChn

【功能】

创建音频编码通道。

【语法】

int IMP_AENC_DestroyChn(int aeChn);

【形参】

参数名称	描述	输入/输出
aeChn	通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

此接口与 IMP_AENC_CreateChn 配合使用。

【举例】

请参见 IMP_AENC_CreateChn 举例。

IMP_AENC_SendFrame

【功能】

创建音频编码通道。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_AENC_SendFrame(int aeChn, IMPAudioFrame *frm);

【形参】

参数名称	描述	输入/输出
aeChn	通道号	输入
frm	音频帧结构体指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

下面代码实现了读取音频原始数据，发送音频原始数据编码，获取编码后的数据，使用编码码流，释放编码码流的操作。

```
1.   while(1) {
2.  // 读取一帧数据
3.  ret = fread(buf_pcm, 1, IMP_AUDIO_BUF_SIZE, file_pcm);
4.  if(ret < IMP_AUDIO_BUF_SIZE)
5.      break;
6.
7.  // 编码
8.  IMPAudioFrame frm;
9.  frm.virAddr = (uint32_t *)buf_pcm;
10. frm.len = ret;
11. ret = IMP_AENC_SendFrame(AeChn, &frm);
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
if(ret != 0) {
    IMP_LOG_ERR(TAG, "imp audio encode send frame failed\n");
    return ret;
}
// 获取编码码流
IMPAudioStream stream;
ret = IMP_AENC_GetStream(AeChn, &stream, BLOCK);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "imp audio encode get stream failed\n");
    return ret;
}

// 使用编码码流
fwrite(stream.stream, 1, stream.len, file_g711);

// 释放编码码流
ret = IMP_AENC_ReleaseStream(AeChn, &stream);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "imp audio encode release stream failed\n");
    return ret;
}
```

IMP_AENC_PollingStream

【功能】

创建音频编码通道。

【语法】

int IMP_AENC_PollingStream(int AeChn, unsigned int timeout_ms);

【形参】

参数名称	描述	输入/输出
AeChn	音频编码输入通道号	输入
timeout_ms	Polling 超时时间	输入

【返回值】

返回值	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 在使用 IMP_AENC_GetStream 之前使用该接口，当该接口调用成功之后表示音频编码数据已经准备完毕，可以使用 IMP_AENC_GetStream 获取编码完毕的数据。

【举例】

请参见 IMP_AENC_SendFrame 中举例。

IMP_AENC_GetStream

【功能】

创建音频编码通道。

【语法】

```
int IMP_AENC_GetStream(int aeChn, IMPAudioStream *stream ,IMPBlock block);
```

【形参】

参数名称	描述	输入/输出
aeChn	通道号	输入
stream	获取音频码流	输出
block	非阻塞标识	输入

【返回值】

返回值	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_AENC_SendFrame 中举例。

IMP_AENC_ReleaseStream

【功能】

创建音频编码通道。

【语法】

```
int IMP_AENC_ReleaseStream(int aeChn,IMPAudioStream *stream);
```

【形参】

参数名称	描述	输入/输出
aeChn	通道号	输入
stream	获取音频码流指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_AENC_SendFrame 中举例。

IMP_AENC_RegisterEncoder

【功能】

创建音频编码通道。

【语法】

```
int IMP_AENC_RegisterEncoder(int *handle, IMPAudioEncEncoder *encoder);
```

【形参】

参数名称	描述	输入/输出
handle	注册句柄	输出
encoder	编码器属性结构体	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 注册之后使用方法和使用 SDK 自带编码器一样。

【举例】

下面代码实现了注册编码器，使用编码器，创建编码通道的操作。

```
int handle_g711a = 0;
IMPAudioEncEncoder my_encoder;
my_encoder.maxFrmLen = 1024;
sprintf(my_encoder.name, "%s", "MY_G711A");
my_encoder.openEncoder = NULL; // 编码器回调函数
my_encoder.encoderFrm = MY_G711A_Encode_Frm; // 编码器回调函数
my_encoder.closeEncoder = NULL; // 编码器回调函数

ret = IMP_AENC_RegisterEncoder(&handle_g711a, &my_encoder);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "IMP_AENC_RegisterEncoder failed\n");
    return ret;
}

// 使用编码器
int AeChn = 0;
IMPAudioEncChnAttr attr;
attr.type = handle_g711a;
// 编码器 type 等于注册成功返回的 handle_g711a 的值即可.
attr.bufSize = 20;
ret = IMP_AENC_CreateChn(AeChn, &attr);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "imp audio encode create channel failed\n");
    return ret;
}
```

IMP_AENC_UnRegisterEncoder

【功能】

创建音频编码通道。

【语法】

int IMP_AENC_UnRegisterEncoder(int *handle);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
handle	注册句柄(注册编码器时获得的句柄)	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 与 IMP_AENC_RegisterEncoder 配合使用。

【举例】

请参见 IMP_AENC_RegisterEncoder 举例。

2.10 ADEC 模块 API

表 2-5 T23 ADEC API 功能表

API 名称	功能
IMP_ADEC_CreateChn	创建音频解码通道
IMP_ADEC_DestroyChn	销毁音频解码通道
IMP_ADEC_SendStream	发送音频码流到音频解码通道
IMP_ADEC_PollingStream	Polling 解码音频流缓存
IMP_ADEC_GetStream	获取解码后码流
IMP_ADEC_ReleaseStream	释放从音频解码通道获取的码流
IMP_ADEC_ClearChnBuf	清除音频解码通道中当前的音频数据缓存
IMP_ADEC_RegisterDecoder	注册解码器
IMP_ADEC_UnRegisterDecoder	注销解码器

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_ADEC_CreateChn

【功能】

创建音频编码通道。

【语法】

int IMP_ADEC_CreateChn(int adChn, IMPAudioDecChnAttr *attr);

【形参】

参数名称	描述	输入/输出
adChn	通道号	输入
attr	通道属性指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

下面代码实现了创建解码通道，销毁解码通道的操作。

```
int adChn = 0;
IMPAudioDecChnAttr attr;
attr.type = PT_G711A;
attr.bufSize = 20;
attr.mode = ADEC_MODE_PACK;
ret = IMP_ADEC_CreateChn(adChn, &attr);
if(ret != 0) {
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
        IMP_LOG_ERR(TAG, "imp audio decoder create channel failed\n");
        return ret;
}
///..........
ret = IMP_ADEC_DestroyChn(adChn, &attr);
if(ret != 0) {
        IMP_LOG_ERR(TAG, "imp audio decoder destroy channel failed\n");
        return ret;
}
```

IMP_ADEC_DestroyChn

【功能】

销毁音频解码通道。

【语法】

int IMP_ADEC_DestroyChn(int adChn)

【形参】

参数名称	描述	输入/输出
adChn	通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

请参见 IMP_ADEC_CreateChn 举例。

IMP_ADEC_SendFrame

【功能】

发送音频码流到音频解码通道。

【语法】

int IMP_ADEC_SendStream(int adChn, IMPAudioStream *stream, IMPBlock block);

【形参】

参数名称	描述	输入/输出
adChn	通道号	输入
stream	音频码流	输入
block	阻塞/非阻塞标识.	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

下面代码实现了获取解码数据，发送给需要解码的数据，获取解码后的数据，使用解码后的数据，释放解码后的数据操作。

```
1.    while(1) {
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
// 获取需要解码的数据
ret = fread(buf_g711, 1, IMP_AUDIO_BUF_SIZE/2, file_g711);
if(ret < IMP_AUDIO_BUF_SIZE/2)
    break;

// 发送解码数据
IMPAudioStream stream_in;
stream_in.stream = (uint8_t *)buf_g711;
stream_in.len = ret;
ret = IMP_ADEC_SendStream(adChn, &stream_in, BLOCK);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "imp audio encode send frame failed\n");
    return ret;
}

// 获取解码后的数据
IMPAudioStream stream_out;
ret = IMP_ADEC_GetStream(adChn, &stream_out, BLOCK);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "imp audio decoder get stream failed\n");
    return ret;
}

// 使用解码后的数据
fwrite(stream_out.stream, 1, stream_out.len, file_pcm);

// 释放解码后的数据
ret = IMP_ADEC_ReleaseStream(adChn, &stream_out);
if(ret != 0) {
    IMP_LOG_ERR(TAG, "imp audio decoder release stream failed\n");
    return ret;
}
```

IMP_ADEC_PollingStream

【功能】

Polling 解码音频流缓存。

【语法】

int IMP_ADEC_PollingStream(int AdChn, unsigned int timeout_ms);

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【形参】

参数名称	描述	输入/输出
AdChn	音频解码输入通道号	输入
timeout_ms	Polling 超时时间	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

- 在使用 IMP_ADEC_GetStream 之前使用该接口，当该接口调用成功之后表示音频解码数据已经准备完毕，可以使用 IMP_ADEC_GetStream 获取解码完毕的数据。

【举例】

请参见 IMP_ADEC_SendFrame 举例。

IMP_ADEC_GetStream

【功能】

获取解码后码流。

【语法】

```
int IMP_ADEC_GetStream(int adChn, IMPAudioStream *stream ,IMPBlock block);
```

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

adChn	通道号	输入
stream	获取解码码流	输入
block	阻塞/非阻塞标识	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_ADEC_SendFrame 举例。

IMP_ADEC_ReleaseStream

【功能】

释放从音频解码通道获取的码流。

【语法】

int IMP_ADEC_ReleaseStream(int adChn,IMPAudioStream *stream);

【形参】

参数名称	描述	输入/输出
adChn	通道号	输入
stream	音频码流指针	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_ADEC_SendFrame 举例。

IMP_ADEC_ClearChnBuf

【功能】

清除音频解码通道中当前的音频数据缓存。

【语法】

```
int IMP_ADEC_ClearChnBuf(int adChn);
```

【形参】

参数名称	描述	输入/输出
adChn	通道号	输入

【返回值】

返回值	描述
0	成功。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_ADEC_RegisterEncoder

【功能】

注册解码器。

【语法】

```
int IMP_ADEC_RegisterDecoder(int *handle, IMPAudioDecDecoder *decoder);
```

【形参】

参数名称	描述	输入/输出
handle	注册句柄	输出
decoder	解码器属性结构体	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

- 注册之后使用方法和使用 SDK 自带解码器一样。

【举例】

下面代码实现了注册解码器，使用解码器和创建解码通道的操作。

```
int handle_g711a = 0;
IMPAudioDecDecoder my_decoder;
sprintf(my_decoder.name, "%s", "MY_G711A");
my_decoder.openDecoder = NULL; // 解码器回调函数
my_decoder.decodeFrm = MY_G711A_Decode_Frm; // 解码器回调函数
my_decoder.getFrmInfo = NULL; // 解码器回调函数
my_decoder.closeDecoder = NULL; // 解码器回调函数

// 注册解码器
ret = IMP_ADEC_RegisterDecoder(&handle_g711a, &my_decoder);
if(ret != 0) {
        IMP_LOG_ERR(TAG, "IMP_ADEC_RegisterDecoder failed\n");
        return ret;
}

 // 使用解码器
int adChn = 0;
IMPAudioDecChnAttr attr;
attr.type = handle_g711a; // 解码 type 等于解码器注册返回的 handle_g711a.
attr.bufSize = 20;
attr.mode = ADEC_MODE_PACK;
// 创建解码通道
ret = IMP_ADEC_CreateChn(adChn, &attr);
 if(ret != 0) {
        IMP_LOG_ERR(TAG, "imp audio decoder create channel failed\n");
        return ret;
}
```

IMP_ADEC_UnRegisterEncoder

【功能】

注销解码器。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_ADEC_UnRegisterDecoder(int *handle);

【形参】

参数名称	描述	输入/输出
handle	注册句柄(注册解码器时获得的句柄).	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_audio.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

请参见 IMP_ADEC_RegisterEncoder 举例。

2.11 AI/AO 数据类型

AI 和 AO 模块相关数据类型定义如下：

表 2-6 T23 AI/AO 数据类型

名称	定义
IMPAudioSampleRate	音频采样率定义
IMPAudioBitWidth	音频采样深度定义
IMPAudioSoundMode	音频声道模式定义
IMPAudioIOAttr	音频输入输出设备属性结构体
IMPAudioFrame	音频帧结构体

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPAudioIChnParam	音频通道参数结构体
IMPAudioAgcConfig	自动增益控制结构体
IMPAudioOChnState	音频输出通道的数据缓存状态结构体

IMPAudioSampleRate

【说明】

音频采样率定义。

【定义】

```
typedef enum {
    AUDIO_SAMPLE_RATE_8000  = 8000,
    AUDIO_SAMPLE_RATE_16000 = 16000,
    AUDIO_SAMPLE_RATE_24000 = 24000,
    AUDIO_SAMPLE_RATE_32000 = 32000,
    AUDIO_SAMPLE_RATE_44100 = 44100,
    AUDIO_SAMPLE_RATE_48000 = 48000,
    AUDIO_SAMPLE_RATE_96000 = 96000,
} IMPAudioSampleRate;
```

【成员】

成员名称	描述
AUDIO_SAMPLE_RATE_8000	8K 采样率
AUDIO_SAMPLE_RATE_16000	16K 采样率
AUDIO_SAMPLE_RATE_24000	24K 采样率
AUDIO_SAMPLE_RATE_32000	32K 采样率
AUDIO_SAMPLE_RATE_44100	44.1K 采样率
AUDIO_SAMPLE_RATE_48000	48K 采样率
AUDIO_SAMPLE_RATE_96000	96K 采样率

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【相关数据类型和接口】

IMPAudioIOAttr

IMPAudioBitWidth

【说明】

音频采样精度定义。

【定义】

```
typedef enum {
    AUDIO_BIT_WIDTH_16 = 16,
} IMPAudioBitWidth;
```

【成员】

成员名称	描述
AUDIO_BIT_WIDTH_16	16bit 采样精度

【注意】

无。

【相关数据类型和接口】

IMPAudioIOAttr

IMPAudioFrame

IMPAudioSoundMode

【说明】

音频声道模式定义。

【定义】

```
typedef enum {
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
    AUDIO_SOUND_MODE_MONO     = 1,
    AUDIO_SOUND_MODE_STEREO = 2,
} IMPAudioSoundMode;
```

【成员】

成员名称	描述
AUDIO_SOUND_MODE_MONO	单声道
AUDIO_SOUND_MODE_STEREO	双声道

【注意】

无。

【相关数据类型和接口】

IMPAudioIOAttr

IMPAudioFrame

IMPAudioAecChn

【说明】

回声消除通道选择定义。

【定义】

```
typedef enum {
    AUDIO_AEC_CHANNEL_FIRST_LEFT     = 1,
    AUDIO_AEC_CHANNEL_SECOND_RIGHT = 2,
    AUDIO_AEC_CHANNEL_THIRD = 3,
    AUDIO_AEC_CHANNEL_FOURTH = 4,
} IMPAudioAecChn;
```

【成员】

成员名称	描述
AUDIO_AEC_CHANNEL_FIRST_LEFT	第一个通道/左

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

AUDIO_AEC_CHANNEL_SECOND_RIGHT	第二个声道/右
AUDIO_AEC_CHANNEL_THIRD	第三个声道
AUDIO_AEC_CHANNEL_FOURTH	第四个声道

【注意】

- AI 只能选择左声道或者右声道。

【相关数据类型和接口】

IMPAudioIChnParam

IMPAudioIOAttr

【说明】

音频输入输出设备属性结构体。

【定义】

```
typedef struct {
    IMPAudioSampleRate samplerate;
    IMPAudioBitWidth bitwidth;
    IMPAudioSoundMode soundmode;
    int frmNum;
    int numPerFrm;
    int chnCnt;
} IMPAudioIOAttr;
```

【成员】

成员名称	描述
samplerate	音频采样率
bitwidth	音频采样精度

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

soundmode	音频声道模式
frmNum	缓存帧的数目，[0-50]
numPerFrm	每帧的采样点个数
chnCnt	支持的通道数目

【注意】

无。

【相关数据类型和接口】

IMP_AI_SetPubAttr

IMP_AI_GetPubAttr

IMP_AO_SetPubAttr

IMP_AO_GetPubAttr

IMPAudioFrame

【说明】

音频输入输出设备属性结构体。

【定义】

```
typedef struct {
    IMPAudioBitWidth bitwidth;
    IMPAudioSoundMode soundmode;
    uint32_t *virAddr;
    uint32_t phyAddr;
    int64_t timeStamp;
    int seq;
    int len;
} IMPAudioFrame;
```

【成员】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

成员名称	描述
bitwidth	音频采样精度
soundmode	音频声道模式
virAddr	音频帧数据虚拟地址
phyAddr	音频帧数据物理地址
timeStamp	音频帧数据时间戳，以 us 为单位
seq	音频帧序号
len	音频帧长度，以 byte 为单位

【注意】

无。

【相关数据类型和接口】

IMP_AI_GetFrame

IMP_AI_GetFrameAndRef

IMP_AI_ReleaseFrame

IMP_AO_SendFrame

IMPAudioIChnParam

【说明】

音频通道参数结构体。

【定义】

```
typedef struct {
    int usrFrmDepth;
    IMPAudioAecChn aecChn;
    int Rev;
} IMPAudioIChnParam;
```

【成员】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

成员名称	描述
usrFrmDepth	音频帧缓存深度
aecChn	选择回音消除的通道
Rev	保留参数

【注意】

无。

【相关数据类型和接口】

IMP_AI_SetChnParam

IMP_AI_GetChnParam

IMPAudioAgcConfig

【说明】

自动增益控制结构体。

【定义】

```
typedef struct {
    int TargetLevelDbfs;
    int CompressionGaindB;
} IMPAudioAgcConfig;
```

【成员】

成员名称	描述
TargetLevelDbfs	增益级别,取值为[0, 31]，这指目标音量级别
CompressionGaindB	设置最大的增益值,[0, 90],0 代表无增益,值越大,增益越高。值越大，放大的倍数

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

	越接近目标增益级别

【注意】

无。

【相关数据类型和接口】

IMP_AI_EnableAgc

IMP_AO_EnableAgc

IMPAudioOChnState

【说明】

音频输出通道的数据缓存状态结构体。

【定义】

```
typedef struct {
    int chnTotalNum;
    int chnFreeNum;
    int chnBusyNum;
} IMPAudioOChnState;
```

【成员】

成员名称	描述
chnTotalNum	输出通道总的缓存块数
chnFreeNum	空闲缓存块数
chnBusyNum	被占用的缓存块数

【注意】

无。

【相关数据类型和接口】

IMP_AO_QueryChnStat

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2.12 AENC 数据类型

AENC 相关数据类型定义如下：

表 2-7 T23 AENC 数据类型

名称	定义
IMPAudioPalyloadType	音频净荷数据类型定义
IMPAudioEncChnAttr	音频编码通道属性结构体
IMPAudioEncEncoder	编码器属性结构体

IMPAudioPalyloadType

【说明】

音频净荷类型枚举定义。

【定义】

```
typedef enum {
    PT_PCM      = 0,
    PT_G711A    = 1,
    PT_G711U    = 2,
    PT_G726     = 3,
    PT_AEC      = 4,
    PT_ADPCM    = 5,
    PT_MAX      = 6,
} IMPAudioPalyloadType;
```

【成员】

成员名称	描述
PT_PCM	PCM
PT_G711A	G711A
PT_G711U	G711U

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

PT_G726	G726
PT_AEC	AEC
PT_ADPCM	ADPCM
PT_PT_MAX	保留

【注意】

无。

【相关数据类型和接口】

IMPAudioEncChnAttr

IMPAudioDecChnAttr

IMPAudioEncChnAttr

【说明】

音频编码通道属性结构体。

【定义】

```
typedef struct {
    IMPAudioPalyloadType type;
    int bufSize;
    uint32_t *value;
} IMPAudioEncChnAttr;
```

【成员】

成员名称	描述
type	音频净荷数据类型
bufSize	buf 大小，以帧为单位，[2～MAX_AUDIO_FRAME_NUM]
value	协议属性指针

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

无。

【相关数据类型和接口】

IMP_ADEC_CreateChn

IMPAudioEncEncoder

【说明】

编码器属性结构体。

【定义】

```
typedef struct {
    IMPAudioPalyloadType type;
    int maxFrmLen;
    char name[16];
    int (*openEncoder)(void *encoderAttr, void
            *encoder);
    int (*encoderFrm)(void *encoder, IMPAudioFrame
            *data, unsigned char *outbuf,int *outLen);
    int (*closeEncoder)(void *encoder);
} IMPAudioEncEncoder;
```

【成员】

成员名称	描述
type	编码协议类型
maxFrmLen	最大码流长度
name	编码器名称
openEncoder	打开编码器回调接口
encoderFrm	编码码流回调接口

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

closeEncoder	关闭编码器回调接口

【注意】

无。

【相关数据类型和接口】

IMP_AENC_RegisterEncoder

2.13 ADEC 数据类型

ADEC 相关数据类型定义如下：

表 2-7 T23 ADEC 数据类型

名称	定义
IMPAudioDecMode	解码方式定义
IMPAudioDecChnAttr	解码通道属性结构体
IMPAudioDecDecoder	解码器属性结构体

IMPAudioDecMode

【说明】

解码方式定义。

【定义】

```
typedef enum {
    ADEC_MODE_PACK     = 0,
    ADEC_MODE_STREAM = 1,
} IMPAudioDecMode;
```

【成员】

成员名称	描述
ADEC_MODE_PACK	Pack 方式解码
ADEC_MODE_STREAM	Stream 方式解码

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

无。

【相关数据类型和接口】

IMPAudioDecChnAttr

IMPAudioDecChnAttr

【说明】

解码通道属性结构体。

【定义】

```
typedef struct {
    IMPAudioPalyloadType type;
    int bufSize;
    IMPAudioDecMode mode;
    void *value;
} IMPAudioDecChnAttr;
```

【成员】

成员名称	描述
type	音频解码协议类型
bufSize	音频解码缓存大小
mode	解码方式
value	具体协议属性指针

【注意】

无。

【相关数据类型和接口】

IMP_ADEC_CreateChn

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPAudioDecDecoder

【说明】

解码器属性结构体。

【定义】

```
typedef struct {
    IMPAudioPalyloadType type;
    char name[16];
    int (*openDecoder)(void *decoderAttr, void *decoder);
    int (*decodeFrm)(void *decoder, unsigned char
            *inbuf,int inLen, unsigned short *outbuf,int
            *outLen,int *chns);
    int (*getFrmInfo)(void *decoder, void *info);
    int (*closeDecoder)(void *decoder);
} IMPAudioDecDecoder;
```

【成员】

成员名称	描述
type	音频解码协议类型
name	音频解码器名字
openDecoder	打开解码器回调函数
decodeFrm	解码码流回调函数
getFrmInfo	获取码流信息回调函数
closeDecoder	关闭解码器回调函数

【注意】

无。

【相关数据类型和接口】

IMP_ADEC_RegisterEncoder

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2.14 SDK sample 介绍

Sample 程序位于/samples/libimp-samples，目录下是依赖 SDK 库的应用程序，包括录音放音、回应消除等应用程序。用户可以参考提供的 sample 程序，编写自己相应工程代码。

完成编译后，拷贝可执行程序到开发板上测试即可。详细介绍请参考下表：

应用文件	功能	执行命令	执行结果
sample-Ai.c	模拟 mic 录音	./sample-Ai	生成录音文件 ai_record.pcm
sample-Ao.c	音频文件播放	./sample-Ao	播放采样率 16K 的音频文件 ao_paly.pcm
sample-Ai-AEC.c	音频回声消除	./sample-Ai-AEC	程序运行后，生成了 test_for_play.pcm 和 test_aec_record.pcm 两个录音文件
sample-Ai-Ref.c	验证模拟 mic 音频获取回声消除算法参考帧	./sample-Ai-Ref	程序运行后，生成了 test_for_play.pcm 和 test_aec_record.pcm 两个录音文件

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

目录

3 视频编码模块结构 1
3.1 模块简介 1
3.2 结构框图 1
3.3 关键词说明 2
3.4 码率控制 2
3.5 Encoder 模块 API 3
3.6 VENC 数据类型 43

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

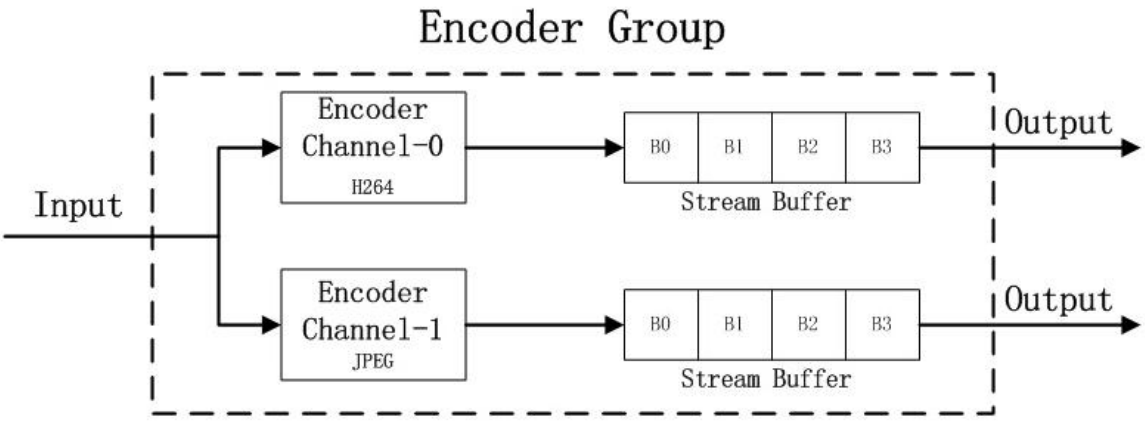
3 视频编码模块

3.1 模块简介

视频编码（JPEG, H264）模块，主要提供视频编码通道的创建和销毁、开启和停止接收图像、设置和获取编码通道属性、获取和释放 码流等功能本模块支持多路实时编码，每路独立，且可以设置不同编码协议和 profile。

3.2 结构框图

图 3-1 T23 编码模块结构



如上图所示，编码模块由若干个 Group 组成，每个 Group 由编码 Channel 组成。每个编码 Channel 附带一个输出码流缓冲区，这个缓冲区由多个 buffer 组成。

目前T40上支持6个Group，每个Group由2个Channel组成。最多使用6个Channel。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

3.3 关键词说明

QP

QP(Quantization Parameter)，QP 值对应量化步长的序号，值越小，量化步长越小，量化的精度就越高，画质也就越好，编码出来的 size 也越大。

GOP

GOP(Group Of Pictures)，指的是两个 I 帧的间隔。视频图像序列由一个或多个图像组 GOP 组成，GOP 之间是独立的。

MB

MB(Macroblock)，编码宏块，H.264 编码的基本单元。

MCU

MCU(Minimum code unit)，JPEG 编码的基本单元。

SPS

SPS(Sequence Parameter Set)，序列参数集，包含了一个 GOP 中所有图像的公有信息。

PPS

PPS(Picture Parameter Set)，图像参数集，包含了一张图像编码所用的参数。

SEI

SEI(Supplemental Enhancement information)，辅助增强信息。

3.4 码率控制

3.4.1 CBR

CBR（Constent Bit Rate）恒定比特率，即在码率统计时间内编码码率恒定。以 H.264 编码为例，用户可设置 maxQp，minQp，bitrate 等。maxQp，minQp 用于控制图像的质量范围，bitrate 用于钳位码率统计时间内的平均编码码率。

当编码码率大于恒定码率时，图像 QP 会逐步向 maxQp 调整，当编码码率远小于恒定码率时，图像 QP 会逐步向 minQp 调整。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

当图像 QP 达到 maxQp 时，QP 被钳位到最大值，bitrate 的钳位效果失效，编码码率有可能会超出 bitrate。

当图像 QP 达到 minQp 时，QP 被钳位到最小值，此时编码的码率已经达到最大值，而且图像质量最好。

3.4.2 VBR

VBR 可变比特率。

3.4.3 Smart

智能编码，推荐使用。

3.4.4 FixQP

Fix Qp 固定 Qp 值。在码率统计时间内，编码图像所有宏块 Qp 值相同，采用用户设定的图像 Qp 值。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

3.5 Encoder 模块 API

表 3-1 T23 AI API 功能表

API 名称	功能
IMP_Encoder_CreateGroup	创建编码 Group
IMP_Encoder_DestroyGroup	销毁编码 Grouop
IMP_Encoder_CreateChn	创建编码 Channel
IMP_Encoder_DestroyChn	销毁编码 Channel
IMP_Encoder_GetChnAttr	获取编码 Channel 的属性
IMP_Encoder_RegisterChn	注册编码 Channel 到 Group
IMP_Encoder_UnRegisterChn	反注册编码 Channel 到 Group
IMP_Encoder_StartRecvPic	开启编码 Channel 接收图像
IMP_Encoder_StopRecvPic	停止编码 Channel 接收图像
IMP_Encoder_Query	查询编码 Channel 状态
IMP_Encoder_GetStream	获取编码的码流
IMP_Encoder_ReleaseStream	释放码流缓存
IMP_Encoder_PollingStream	Polling 码流缓存
IMP_Encoder_GetFd	获取编码 Channel 对应的设备文件句柄
IMP_Encoder_SetMaxStreamCnt	设置码流缓存 Buffer 个数
IMP_Encoder_GetMaxStreamCnt	获取码流 Buffer 数
IMP_Encoder_RequestIDR	请求 IDR 帧
IMP_Encoder_FlushStream	刷掉编码器里残留的码流，以 IDR 帧开始编码
IMP_Encoder_GetChnFrmRate	获取帧率控制属性
IMP_Encoder_SetChnFrmRate	动态设置帧率控制属性
IMP_Encoder_GetChnAttrRcMode	获取码率控制模式属性
IMP_Encoder_SetChnAttrRcMode	设置码率控制模式属性
IMP_Encoder_SetFisheyeEnableStatus	设置 Ingenic 提供的鱼眼矫正算法的使能状态
IMP_Encoder_GetFisheyeEnableStatus	获取 Ingenic 提供的鱼眼矫正算法的使能状态
IMP_Encoder_GetChnEncType	获取图像编码协议类型
IMP_Encoder_SetPool	绑定 chnnel 到内存池，编码从内存池申请 mem
IMP_Encoder_GetPool	获取内存池的 poolID
IMP_Encoder_SetChnROI	设置通道 ROI 属性
IMP_Encoder_GetChnROI	获取通道 ROI 属性
IMP_Encoder_SetGOPSize	设置通道 GOP 属性
IMP_Encoder_GetGOPSize	获取通道 GOP 属性
IMP_Encoder_SetChnColor2Grey	设置彩转灰功能
IMP_Encoder_GetChnColor2Grey	获取彩转灰功能属性

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_SetChnFrmUsedMode	设置通道输入帧使用模式属性
IMP_Encoder_GetChnFrmUsedMode	获取通道输入帧使用模式属性
IMP_Encoder_SetChnDenoise	设置通道去噪属性
IMP_Encoder_GetChnDenoise	获取通道去噪属性
IMP_Encoder_SetChnHSkip	设置通道高级跳帧属性
IMP_Encoder_GetChnHSkip	获取通道高级跳帧属性
IMP_Encoder_SetChnHSkipBlackEnhance	设置通道高级跳帧中 bBlackEnhance 属性
IMP_Encoder_InsertUserData	插入用户数据
IMP_Encoder_SetChangeRef	设置是否允许改变 BASE 帧参考方式的状态
IMP_Encoder_GetChangeRef	获取是否允许改变 BASE 帧参考方式的状态
IMP_Encoder_SetMbRC	设置是否开启宏块级 qp 控制
IMP_Encoder_GetMbRC	获取是否开启宏块级 qp 控制的状态
IMP_Encoder_SetSuperFrameCfg	设置编码超大帧配置
IMP_Encoder_GetSuperFrameCfg	获取编码超大帧配置
IMP_Encoder_SetH264TransCfg	设置 H.264 协议编码通道的色度量化属性
IMP_Encoder_GetH264TransCfg	获取 H.264 协议编码通道的色度量化属性
IMP_Encoder_SetJpegeQl	设置 JPEG 协议编码通道的量化表配置参数
IMP_Encoder_GetJpegeQl	获取 JPEG 协议编码通道的量化表配置参数
IMP_Encoder_InputJpege	IVPU 外部输入 NV12 编码 JPEG

IMP_Encoder_CreateGroup

【功能】

创建编码 Group。

【语法】

int IMP_Encoder_CreateGroup(int encGroup);

【形参】

参数名称	描述	输入/输出
encGroup	Group 号，取值范围:[0, NR_MAX_ENC_GROUPS - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 一路 Group 仅支持一路分辨率，不同分辨率需启动新的 Group，一路 Group 允许最多注册 2 个编码 channel。
- 如果指定的 Group 已经存在，则返回失败。

【举例】

```
1.   for (i = 0; i < FS_CHN_NUM; i++) {
2.    if (chn[i].enable) {
3.        ret = IMP_Encoder_CreateGroup(chn[i].index);
4.        if (ret < 0) {
5.            IMP_LOG_ERR(TAG, "IMP_Encoder_CreateGroup(%d) error !\n", chn[i].
index);
6.            return -1;
7.        }
8.    }
9.}
```

IMP_Encoder_DestroyGroup

【功能】

销毁编码 Grouop。

【语法】

int IMP_Encoder_DestroyGroup(int encGroup);

【形参】

参数名称	描述	输入/输出
encGroup	Group 号，取值范围:[0, NR_MAX_ENC_GROUPS - 1]	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 销毁 Group 时，必须保证 Group 为空，即没有任何 Channel 在 Group 中注册，或注册到 Group 中 的 Channel 已经反注册，否则返回失败。
- 销毁并不存在的 Group，则返回失败。

【举例】

无。

IMP_Encoder_CreateChn

【功能】

创建编码 Channel。

【语法】

int IMP_Encoder_CreateChn(int encChn,const IMPEncoderChnAttr *attr);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入
attr	编码 Channel 属性指针	输入

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 编码 Channel 属性由两部分组成，编码器属性和码率控制属性。
- 编码器属性首先需要选择编码协议，然后分别对各种协议对应的属性进行赋值。

【举例】

无。

IMP_Encoder_DestroyChn

【功能】

销毁编码 Channel。

【语法】

int IMP_Encoder_DestroyChn(int encChn);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 销毁并不存在的 Channel，则返回失败。
- 销毁前必须保证 Channel 已经从 Group 反注册，否则返回失败。

【举例】

详情请参考 IMP_Encoder_UnRegisterChn 举例。

IMP_Encoder_GetChnAttr

【功能】

获取编码 Channel 的属性。

【语法】

int IMP_Encoder_GetChnAttr(int encChn, IMPEncoderCHNAttr * const attr);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入
attr	编码 Channel 属性指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_Encoder_RegisterChn

【功能】

注册编码 Channel 到 Group。

【语法】

int IMP_Encoder_RegisterChn(int encGroup, int encChn);

【形参】

参数名称	描述	输入/输出
encGroup	编码 Group 号,取值范围: [0, NR_MAX_ENC_GROUPS - 1]	输入
encChn	编码 Channel 号,取值范围: [0, NR_MAX_ENC_CHN - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 注册并不存在的 Channel，则返回失败。
- 注册 Channel 到不存在的 Group，否则返回失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 同一个编码 Channel 只能注册到一个 Group，如果该 Channel 已经注册到某个 Group，则返回失败。
- 如果一个 Group 已经被注册，那么这个 Group 就不能再被其他的 Channel 注册，除非之前注册关系被解除。

【举例】

无。

IMP_Encoder_UnRegisterChn

【功能】

反注册编码 Channel 到 Group。

【语法】

int IMP_Encoder_UnRegisterChn(int encChn);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 注销未创建的 Channel，则返回失败。
- 注销未注册的 Channel，则返回失败。
- 如果编码 Channel 未停止接收图像编码，则返回失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- Channel 注销之后，编码 Channel 会被复位，编码 Channel 里的码流 buffer 都会被清空，如果用户还在使用 未及时释放的码流 buffer，将不能保证 buffer 数据的正确性，用户可以使用 IMP_Encoder_Query 接口来查询编 码 Channel 码流 buffer 状态，确认码流 buffer 里的码流取完之后再反注册 Channel。

【举例】

```
int ret = 0, i = 0, chnNum = 0;
IMPEncoderChnStat chn_stat;

for (i = 0; i <  FS_CHN_NUM; i++) {
    if (chn[i].enable) {
        chnNum = chn[i].index;
        memset(&chn_stat, 0, sizeof(IMPEncoderChnStat));
        ret = IMP_Encoder_Query(chnNum, &chn_stat);
        if (ret < 0) {
                IMP_LOG_ERR(TAG, "IMP_Encoder_Query(%d) error: %d\n", chnNum, ret);
                return -1;
            }
            if (chn_stat.registered) {
                ret = IMP_Encoder_UnRegisterChn(chnNum);
                if (ret < 0) {
                    IMP_LOG_ERR(TAG, "IMP_Encoder_UnRegisterChn(%d) error: %d\n", chnNum, ret);
                    return -1;
                }
                ret = IMP_Encoder_DestroyChn(chnNum);
                if (ret < 0) {
                    IMP_LOG_ERR(TAG, "IMP_Encoder_DestroyChn(%d) error: %d\n", chnNum, ret);
                    return -1;
                }
            }
        }
    }
```

IMP_Encoder_StartRecvPic

【功能】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

开启编码 Channel 接收图像。

【语法】

int IMP_Encoder_StartRecvPic(int encChn);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 开启编码 Channel 接收图像后才能开始编码。
- 如果 Channel 未创建，则返回失败。
- 如果 Channel 没有注册到 Group，则返回失败

【举例】

```
int val, i, chnNum, ret;
char stream_path[64];
IMPEncoderEncType encType;
int stream_fd = -1, totalSaveCnt = 0;

val = (int)args;
chnNum = val & 0xffff;
encType = (val >> 16) & 0xffff;

    ret = IMP_Encoder_StartRecvPic(chnNum);
     if (ret < 0) {
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
12.         IMP_LOG_ERR(TAG, "IMP_Encoder_StartRecvPic(%d) failed\n", chnNum);
13.         return ((void *)-1);
14.       }
15.
16.       for (i = 0; i < totalSaveCnt; i++) {
17.         ret = IMP_Encoder_PollingStream(chnNum, 1000);
18.         if (ret < 0) {
19.           IMP_LOG_ERR(TAG, "IMP_Encoder_PollingStream(%d) timeout\n", chnNum)
;
20.           continue;
21.         }
22.
23.         IMPEncoderStream stream;
24.         ret = IMP_Encoder_GetStream(chnNum, &stream, 1);
25.         if (ret < 0) {
26.           IMP_LOG_ERR(TAG, "IMP_Encoder_GetStream(%d) failed\n", chnNum);
27.           return ((void *)-1);
28.         }
29.
30.         if (encType == IMP_ENC_TYPE_JPEG) {
31.           ret = save_stream_by_name(stream_path, i, &stream);
32.           if (ret < 0) {
33.             return ((void *)ret);
34.           }
35.         }
36.         else {
37.           ret = save_stream(stream_fd, &stream);
38.           if (ret < 0) {
39.             close(stream_fd);
40.             return ((void *)ret);
41.           }
42.         }
43.         IMP_Encoder_ReleaseStream(chnNum, &stream);
44.       }
45.
46.       close(stream_fd);
47.
48.       ret = IMP_Encoder_StopRecvPic(chnNum);
49.       if (ret < 0) {
50.         IMP_LOG_ERR(TAG, "IMP_Encoder_StopRecvPic(%d) failed\n", chnNum);
51.         return ((void *)-1);
52.       }
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_StopRecvPic

【功能】

停止编码 Channel 接收图像。

【语法】

int IMP_Encoder_StopRecvPic(int encChn);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 此接口并不判断当前是否停止接收，即允许重复停止接收不返回错误，调用此接口仅停止接收原始数据编码，码流 buffer 并不会被消除。
- 如果 Channel 未创建，则返回失败。
- 如果 Channel 没有注册到 Group，则返回失败。

【举例】

详情请参考 IMP_Encoder_StartRecvPic 举例。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_Query

【功能】

查询编码 Channel 状态。

【语法】

int IMP_Encoder_Query(int encChn, IMPEncoderChnStat *stat);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入
stat	编码 Channel 状态	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

详情请参考 IMP_Encoder_UnRegisterChn 举例。

IMP_Encoder_GetStream

【功能】

获取编码的码流。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_Encoder_GetStream(int encChn,

IMPEncoderStream *stream, bool blockFlag);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围: [0, NR_MAX_ENC_CHN - 1]	输入
stream	码流结构体指针	输入
blockFlag	是否使用阻塞方式获取，0 非阻塞；1 阻塞	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 每次获取一帧码流的数据，如果用户长时间不获取码流,码流缓冲区就会满。一个编码 Channel 如果发生码流缓冲区满,就会把后 面接收的图像丢掉,直到用户获取码流,从而有足够的码流缓冲可以用于编码时,才开始继续编码。建议用户 获取码流接口调用与释放码流的接口调用成对出现,且尽快释放码流,防止出现由于用户态获取码流,释放不 及时而导致的码流 buffer 满,停止编码。
- 对于 H264 类型码流，一次调用成功获取一帧的码流，这帧码流可能包含多个包。
- 对于 JPEG 类型码流，一次调用成功获取一帧的码流，这帧码流只包含一个包，这一帧包含了 JPEG 图片文件的完整信息。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

详情请参考 IMP_Encoder_StartRecvPic 举例。

IMP_Encoder_ReleaseStream

【功能】

释放码流缓存。

【语法】

int IMP_Encoder_ReleaseStream(int encChn,IMPEncoderStream *stream);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围: [0, NR_MAX_ENC_CHN - 1]	输入
stream	码流结构体指针	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 此接口应当和 IMP_Encoder_GetStream 配对起来使用，用户获取码流后必须及时释放已经获取的码流缓存，否则可能会导致码流 buffer 满，影响编码器编码。并且用户必须按先获取先 释放的顺序释放已经获取的码流缓存；在编码 Channel 反注册后，所有未释放的码流包均无效，不能再使用或者释放这部分无效的码流缓存。
- 如果 pstStream 为 NULL,则返回失败。
- 如果 Channel 未创建，则返回失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 释放无效的码流会返回失败。

【举例】

详情请参考 IMP_Encoder_StartRecvPic 举例。

IMP_Encoder_PollingStream

【功能】

Polling 码流缓存。

【语法】

int IMP_Encoder_PollingStream(int encChn,uint32_t timeoutMsec);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围: [0, NR_MAX_ENC_CHN - 1]	输入
timeoutMsec	超时时间，单位：毫秒	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 在获取码流之前可以用过此 API 进行 Polling，当码流缓存不为空时或超时时函数返回。

【举例】

详情请参考 IMP_Encoder_StartRecvPic 举例。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_GetFd

【功能】

获取编码 Channel 对应的设备文件句柄。

【语法】

int IMP_Encoder_GetFd(int encChn);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 在使用 IMP_Encoder_PollingStream 不合适的场合，比如在同一个地方 Polling 多个编码 channel 的编码完成情况时，可以使用此文件句柄调用 select, poll 等类似函数来阻塞等待编码完成事件。
- 调用此 API 需要通道已经存在。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_SetMaxStreamCnt

【功能】

设置码流缓存 Buffer 个数。

【语法】

int IMP_Encoder_SetMaxStreamCnt(int encChn, int nrMaxStream);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围: [0, NR_MAX_ENC_CHN - 1]	输入
nrMaxStream	码流 Buffer 数,取值范围: [1, NR_MAX_ENC_CHN_STREAM]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 由于码流缓存 Buffer 个数在通道创建时就已经固定，因此次 API 需要在通道创建之前调用。
- 若通道创建之前不调用此 API 设置码流缓存 Buffer 个数，则使用 SDK 默认的 buffer 个数。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_GetMaxStreamCnt

【功能】

获取码流缓存 Buffer 个数。

【语法】

int IMP_Encoder_GetMaxStreamCnt(int encChn, int *nrMaxStream);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围: [0, NR_MAX_ENC_CHN - 1]	输入
nrMaxStream	码流 Buffer 数,取值范围: [1, NR_MAX_ENC_CHN_STREAM]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_RequestIDR

【功能】

请求 IDR 帧。

【语法】

int IMP_Encoder_RequestIDR(int encChn);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 在调用此 API 后，会在最近的编码帧申请 IDR 帧编码。
- 此 API 只适用于 H264 编码 channel。

【举例】

无。

IMP_Encoder_FlushStream

【功能】

刷掉编码器里残留的旧码流，并以 IDR 帧开始编码。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_Encoder_FlushStream(int encChn);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 在调用此 API 后，会在最近的编码帧申请 IDR 帧编码。

【举例】

无。

IMP_Encoder_GetChnFrmRate

【功能】

获取帧率控制属性。

【语法】

int IMP_Encoder_GetChnFrmRate(int encChn, IMPEncoderFrmRate *pstFps);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入
pstFps	帧率控制属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 调用此 API 会获取通道的帧率控制属性，调用此 API 需要通道已经存在。
- 此 API 只适用于 H264 编码 channel。

【举例】

无。

IMP_Encoder_SetChnFrmRate

【功能】

动态设置帧率控制属性。

【语法】

int IMP_Encoder_SetChnFrmRate(int encChn, const IMPEncoderFrmRate *pstFps);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入
pstFps	帧率控制属性参数	输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 调用此 API 会重新设置编码器帧率属性，帧率属性在下一个 GOP 生效，最大延时 1 秒钟生效，调用此 API 需要通道已经存在。
- 如果调用 IMP_FrameSource_SetChnFPS()函数动态改变系统帧率，那么需要调用该函数修改编码器帧率，完成正确参数配置。
- 此 API 只适用于 H264 编码 channel。

【举例】

无。

IMP_Encoder_GetChnAttrRcMode

【功能】

获取码率控制模式属性。

【语法】

```
int IMP_Encoder_GetChnAttrRcMode(int encChn,
IMPEncoderAttrRcMode *pstRcModeCfg);
```

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入
pstRcModeCfg	码率控制模式属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 调用此 API 会获取通道的码率控制模式属性，调用此 API 需要通道已经存在。
- 此 API 只适用于 H264 编码 channel。

【举例】

无。

IMP_Encoder_SetChnAttrRcMode

【功能】

设置码率控制模式属性。

【语法】

int IMP_Encoder_SetChnAttrRcMode(int encChn, const IMPEncoderAttrRcMode *pstRcModeCfg);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN -	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

	1]	
pstRcModeCfg	码率控制模式属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 调用此 API 会设置通道的码率控制模式属性，下一个 IDR 生效,调用此 API 需要通道已经存在。
- 目前，码率控制模式支持 ENC_RC_MODE_FIXQP, ENC_RC_MODE_CBR, ENC_RC_MODE_VBR 与 ENC_RC_MODE_SMART。
- 此 API 只适用于 H264 编码 channel。

【举例】

无。

IMP_Encoder_SetFisheyeEnableStatus

【功能】

设置 Ingenic 提供的鱼眼矫正算法的使能状态。

【语法】

int IMP_Encoder_SetFisheyeEnableStatus(int encChn, int enable);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入
enable	0:不使能(默认),1:使能	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 由于鱼眼矫正算法的使能状态在通道创建时就已经固定，因此次 API 需要在通道创建之前调用。
- 若通道创建之前不调用此 API 设置 Ingenic 提供的鱼眼矫正算法的使能状态,则默认不使能，即不能使用君正提供的鱼眼矫正算法。
- 此 API 只适用于 H264 编码 channel。

【举例】

无。

IMP_Encoder_GetFisheyeEnableStatus

【功能】

获取 Ingenic 提供的鱼眼矫正算法的使能状态。

【语法】

int IMP_Encoder_GetFisheyeEnableStatus(int encChn,int *enable);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入
enable	返回设置的 Ingenic 提供的鱼眼矫正算法的使能状态,0:未使能,1:已使能	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 此 API 只适用于 H264 编码 channel。

【举例】

无。

IMP_Encoder_GetChnEncType

【功能】

获取图像编码协议类型。

【语法】

int IMP_Encoder_GetChnEncType(int encChn, IMPPayloadType *payLoadType);

【形参】

参数名称	描述	输入/输出
encChn	编码 Channel 号,取值范围:[0, NR_MAX_ENC_CHN - 1]	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

payLoadType	返回获取图像编码协议类型	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 如果通道未创建，则返回失败。

【举例】

无。

IMP_Encoder_SetPool

【功能】

绑定 chnnel 到内存池中，即 Encoder 从 pool 申请 mem。

【语法】

int IMP_Encoder_SetPool(int chnNum, int poolID);

【形参】

参数名称	描述	输入/输出
chnNum	通道编号	输入
poolID	内存池编号	输入

【返回值】

返回值	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 为了解决 rmem 碎片化，将该 channel Encoder 绑定到对应的 mempool 中, Encoder 申请 mem 就在 mempool 中申请，若不调用，Encoder 会在 rmem 中申请 此时对于 rmem 来说会存在碎片的可能。

【举例】

无。

IMP_Encoder_GetPool

【功能】

通过 channel ID 获取 poolID

【语法】

int IMP_Encoder_GetPool(int chnNum);

【形参】

参数名称	描述	输入/输出
chnNum	通道编号	输入

【返回值】

返回值	描述
0	成功。
非 0	当前 ChannelId 的 PoolId

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_Encoder_SetChnROI

【功能】

设置通道 ROI 属性

【语法】

int IMP_Encoder_SetChnROI(int encChn, const IMPEncoderROICfg *pstVencRoiCfg);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstVencRoiCfg	ROI 属性参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会设置通道的 ROI 属性，调用此 API 需要通道已经存在。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

无。

IMP_Encoder_GetChnROI

【功能】

获取通道 ROI 属性

【语法】

int IMP_Encoder_GetChnROI(int encChn, IMPEncoderROICfg *pstVencRoiCfg);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstVencRoiCfg	ROI 属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会获取通道的 ROI 属性，调用此 API 需要通道已经存在。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_SetGOPSize

【功能】

设置通道 GOP 属性

【语法】

int IMP_Encoder_SetGOPSize(int encChn, const IMPEncoderGOPSizeCfg *pGOPSize);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pGOPSize	GOP 属性参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会设置通道的 GOPSize 属性，调用此 API 需要通道已经存在。

【举例】

无。

IMP_Encoder_GetGOPSize

【功能】

获取通道 GOP 属性

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_Encoder_GetGOPSize(int encChn, IMPEncoderGOPSizeCfg *pstGOPSize);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstGOPSize	GOP 属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会获取通道的 GOPSize 属性，调用此 API 需要通道已经存在。

【举例】

无。

IMP_Encoder_SetChnColor2Grey

【功能】

设置彩转灰功能

【语法】

int IMP_Encoder_SetChnColor2Grey(int encChn, const IMPEncoderColor2GreyCfg *pstColor2Grey);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

encChn	通道编号	输入
pstColor2Grey	彩转灰功能的参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 设置通道的彩转灰功能，在下一个 IDR 或者 P 帧生效。调用此 API 需要通道已经存在。

【举例】

无。

IMP_Encoder_GetChnColor2Grey

【功能】

设置彩转灰功能

【语法】

int IMP_Encoder_GetChnColor2Grey(int encChn, IMPEncoderColor2GreyCfg *pstColor2Grey);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstColor2Grey	彩转灰功能的参数	输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会获取通道的彩转灰功能属性，调用此 API 需要通道已经存在。

【举例】

无。

IMP_Encoder_SetChnFrmUsedMode

【功能】

设置通道输入帧使用模式属性

【语法】

int IMP_Encoder_SetChnFrmUsedMode(int encChn, const IMPEncoderAttrFrmUsed *pfrmUsedAttr);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pfrmUsedAttr	输入帧使用模式属性参数	输入

【返回值】

返回值	描述
0	成功。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会设置通道的输入帧使用模式属性，调用此 API 需要通道已经存在。

【举例】

无。

IMP_Encoder_GetChnFrmUsedMode

【功能】

获取通道输入帧使用模式属性

【语法】

int IMP_Encoder_GetChnFrmUsedMode(int encChn, IMPEncoderAttrFrmUsed *pfrmUsedAttr);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pfrmUsedAttr	输出帧使用模式属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会获取通道的输入帧使用模式属性，调用此 API 需要通道已经存在。

【举例】

无。

IMP_Encoder_SetChnDenoise

【功能】

设置通道去噪属性

【语法】

int IMP_Encoder_SetChnDenoise(int encChn, const IMPEncoderAttrDenoise *pdenoiseAttr);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pdenoiseAttr	去噪属性参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会设置通道的去噪属性，调用此 API 需要通道已经存在。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

无。

IMP_Encoder_GetChnDenoise

【功能】

获取通道去噪属性

【语法】

int IMP_Encoder_GetChnDenoise(int encChn, IMPEncoderAttrDenoise *pdenoiseAttr);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pdenoiseAttr	去噪属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会获取通道的去噪属性，调用此 API 需要通道已经存在。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_SetChnHSkip

【功能】

设置通道高级跳帧属性

【语法】

int IMP_Encoder_SetChnHSkip(int encChn, const IMPEncoderAttrHSkip *phSkipAttr);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
phSkipAttr	高级跳帧属性参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会设置通道高级跳帧属性，调用此 API 需要通道已经存在。

若创建通道时设置的高级跳帧类型是 IMP_Encoder_STYPE_N1X 到 IMP_Encoder_STYPE_N2X 中的一个，此 API 设置跳帧类型只能为 IMP_Encoder_STYPE_N1X 或 IMP_Encoder_STYPE_N2X 中的任意一个。

若创建通道时设置的高级跳帧类型是 IMP_Encoder_STYPE_N4X 到 IMP_Encoder_STYPE_H1M_TRUE 中的一个，则可以设置为任意一个高级跳帧类型。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_GetChnHSkip

【功能】

获取通道高级跳帧属性

【语法】

int IMP_Encoder_GetChnHSkip(int encChn, IMPEncoderAttrHSkip *phSkipAttr);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
phSkipAttr	高级跳帧属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会获取通道高级跳帧属性，调用此 API 需要通道已经存在。

【举例】

无。

IMP_Encoder_SetChnHSkipBlackEnhance

【功能】

设置通道高级跳帧中 bBlackEnhance 属性

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_Encoder_SetChnHSkipBlackEnhance(int encChn, const int bBlackEnhance);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
bBlackEnhance	逻辑值	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 会设置通道高级跳帧中 bBlackEnhance 属性，调用此 API 需要通道已经存在。

【举例】

无。

IMP_Encoder_InsertUserData

【功能】

插入用户数据

【语法】

int IMP_Encoder_InsertUserData(int encChn, void *userData, uint32_t userDataLen);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

encChn	通道编号	输入
userData	用户数据指针	输入
userDataLen	用户数据长度，取值范围:(0, 1024],以 byte 为单位	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

调用此 API 需要通道已经存在。

如果 userData 为空或 userDataLen 为 0,则返回失败。

插入用户数据,只支持 H.264 编码协议。

最多分配 2 块内存空间用于缓存用户数据,且每段用户数据大小不超过 1k byte。

如果用户插入的数据多余 2 块,或插入的一段用户数据大于 1k byte 时,此接口会返回错误。

每段用户数据以 SEI 包的形式被插入到最新的图像码流包之前。在某段用户数据包被编码发送之后，通道内缓存这段用户数据的内存空间被清零,用于存放新的用户数据。

【举例】

无。

IMP_Encoder_SetChangeRef

【功能】

设置是否允许改变 BASE 帧参考方式的状态

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_Encoder_SetChangeRef(int encChn, int bEnable);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
bEnable	返回是否允许改变 BASE 帧参考方式的状态，0：不允许改变，1，允许改变	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_Encoder_GetChangeRef

【功能】

获取是否允许改变 BASE 帧参考方式的状态

【语法】

int IMP_Encoder_GetChangeRef(int encChn, int *bEnable);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

encChn	通道编号	输入
bEnable	返回是否允许改变 BASE 帧参考方式的状态，0：不允许改变，1，允许改变	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_Encoder_SetMbRC

【功能】

设置是否开启宏块级 qp 控制

【语法】

int IMP_Encoder_SetMbRC(int encChn, int bEnable);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
bEnable	0:不开启(默认), 1:开启	输入

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

此 API 必须在创建编码通道后调用，设置完后编码下一帧就会生效。

【举例】

无。

IMP_Encoder_GetMbRC

【功能】

获取是否开启宏块级 qp 控制

【语法】

int IMP_Encoder_GetMbRC(int encChn, int *bEnable);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
bEnable	返回开启宏块级 qp 控制的状态, 0:不开启, 1:开启	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_Encoder_SetSuperFrameCfg

【功能】

设置编码超大帧配置

【语法】

int IMP_Encoder_SetSuperFrameCfg(int encChn, const IMPEncoderSuperFrmCfg *pstSuperFrmParam);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstSuperFrmParam	编码超大帧配置	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

无。

【举例】

无。

IMP_Encoder_GetSuperFrameCfg

【功能】

获取编码超大帧配置

【语法】

int IMP_Encoder_GetSuperFrameCfg(int encChn, IMPEncoderSuperFrmCfg *pstSuperFrmParam);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstSuperFrmParam	返回编码超大帧配置	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_SetH264TransCfg

【功能】

设置 H.264 协议编码通道的色度量化属性

【语法】

int IMP_Encoder_SetH264TransCfg(int encChn, const IMPEncoderH264TransCfg *pstH264TransCfg);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstH264TransCfg	H.264 协议编码通道的色度量化属性	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

建议在创建编码 channel 之后，startRecvPic 之前调用，设置时先 GetH264TransCfg，然后再 SetH264TransCfg。

此 API 只适用于 H264。

如果通道未创建,则返回失败。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_Encoder_GetH264TransCfg

【功能】

获取 H.264 协议编码通道的色度量化属性

【语法】

int IMP_Encoder_GetH264TransCfg(int encChn, IMPEncoderH264TransCfg *pstH264TransCfg);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstH264TransCfg	返回 H.264 协议编码通道的色度量化属性	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

此 API 只适用于 H264。

如果通道未创建,则返回失败。

【举例】

无。

IMP_Encoder_SetJpegeQl

【功能】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

设置 JPEG 协议编码通道的量化表配置参数

【语法】

int IMP_Encoder_SetJpegeQl(int encChn, const IMPEncoderJpegeQl *pstJpegeQl);

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstJpegeQl	JPEG 协议编码通道的量化表配置参数,128 个字节填入量化表。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

此 API 只适用于 JPEG。

如果通道未创建,则返回失败。

【举例】

无。

IMP_Encoder_GetJpegeQl

【功能】

获取 JPEG 协议编码通道的用户量化表配置参数

【语法】

int IMP_Encoder_GetJpegeQl(int encChn, IMPEncoderJpegeQl *pstJpegeQl);

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【形参】

参数名称	描述	输入/输出
encChn	通道编号	输入
pstJpegeQl	返回 JPEG 协议编码通道的量化表配置参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

此 API 只适用于 JPEG。

如果通道未创建,则返回失败。

【举例】

无。

IMP_Encoder_InputJpege

【功能】

IVPU 外部输入 NV12 编码 JPEG

【语法】

int IMP_Encoder_InputJpege(uint8_t *src, uint8_t *dst, int src_w, int src_h, int q,int *stream_length);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

src	源数据地址指针	输入
dst	码流数据地址指针	输入
src_w	图像宽	输入
src_h	图像高	输入
q	图像质量控制<暂不支持>	输入
stream_length	码流数据长度	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

此 API 只适用于 JPEG。

此 API 只适用于宽 32 对齐,高 8 对齐的 NV12 输入编码 JPEG。

【举例】

无。

3.6 VENC 数据类型

VENC 相关数据类型定义如下：

表 3-2 T23 VENC 数据类型

名称	定义
IMPEncoderPack	编码帧码流包结构体
IMPEncoderStream	编码帧信息结构体

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPEncoderFrmRate	编码 Channel 的输出帧率
IMPEncoderAttrH264FixQP	Fixqp 模式属性
IMPEncoderAttrH264CBR	Cbr 模式属性
IMPEncoderAttrH264VBR	Vbr 模式属性
IMPEncoderAttrH264Smart	Smart 模式属性
IMPEncoderAttrRcMode	编码器编码模式属性结构体
IMPEncoderAttrFrmUsed	输入帧使用模式属性
IMPEncoderAttrDenoise	去噪属性
IMPEncoderAttrHSkip	高级跳帧属性
IMPEncoderAttrInitHSkip	高级跳帧初始化属性
IMPEncoderRcAttr	码率控制器属性结构体
IMPEncoderAttr	编码器属性结构体
IMPEncoderCHNAttr	编码 Channel 属性

IMPEncoderPack

【说明】

定义编码帧码流包结构体。

【定义】

```
typedef struct {
    uint32_t        phyAddr;
    uint32_t        virAddr;
    uint32_t        length;
    int64_t         timestamp;
    bool            frameEnd;
    IMPEncoderDataType    dataType;
} IMPEncoderPack;
```

【成员】

成员名称	描述
phyAddr	码流包物理地址
virAddr	码流包虚拟地址
length	码流包长度
timestamp	时间戳，单位 us

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

frameEnd	帧结束标识
dataType	H.264 编码码流 NAL 类型

【注意】

无。

IMPEncoderStream

【说明】

定义编码帧码流类型结构体。

【定义】

```
typedef struct {
    IMPEncoderPack    *pack;
    uint32_t          packCount;
    uint32_t          seq;
    IMPRefType        refType;
} IMPEncoderStream;
```

【成员】

成员名称	描述
pack	帧码流包结构
packCount	一帧码流的所有包的个数
seq	编码帧码流序列号
refType	编码帧参考类型，只针对 H264

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPEncoderFrmRate

【说明】

编码 Channel 的输出帧率。

【定义】

```
typedef struct {
    uint32_t    frmRateNum;
    uint32_t    frmRateDen;
} IMPEncoderFrmRate;
```

【成员】

成员名称	描述
frmRateNum	帧率的分子
frmRateDen	帧率的分母

【注意】

无。

IMPEncoderAttrH264FixQP

【说明】

Fixqp 模式属性。

【定义】

```
typedef struct {
    uint32_t    qp;
} IMPEncoderAttrH264FixQP;
```

【成员】

成员名称	描述
qp	帧级 Qp 值

【注意】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

无。

IMPEncoderAttrH264CBR

【说明】

CBR 模式属性。

【定义】

```
typedef struct {
    uint32_t        maxQp;
    uint32_t        minQp;
    uint32_t        outBitRate;
    int             iBiasLvl;
    uint32_t        frmQPStep;
    uint32_t        gopQPStep;
    bool            adaptiveMode;
    bool            gopRelation;
} IMPEncoderAttrH264CBR;
```

【成员】

成员名称	描述
maxQp	编码器支持图像最大 QP
minQp	编码器支持图像最小 QP
outBitRate	编码器输出码率,以 kbps 为单位
iBiasLvl	调整 I 帧 QP 以调节 I 帧的图像质量及其码流大小,范围:[-3,3]
frmQPStep	帧间 QP 变化步长
gopQPStep	GOP 间 QP 变化步长
adaptiveMode	自适应模式
gopRelation	GOP 是否关联

【注意】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

无。

IMPEncoderAttrH264VBR

【说明】

VBR 模式属性。

【定义】

```
typedef struct {
    uint32_t        maxQp;
    uint32_t        minQp;
    uint32_t        staticTime;
    uint32_t        maxBitRate;
    int32_t         iBiasLvl;
    uint32_t        changePos;
    uint32_t        qualityLvl;
    uint32_t        frmQPStep;
    uint32_t        gopQPStep;
    bool            gopRelation;
} IMPEncoderAttrH264VBR;
```

【成员】

成员名称	描述
maxQp	编码器支持图像最大 QP
minQp	编码器支持图像最小 QP
staticTime	码率统计时间,以秒为单位
maxBitRate	编码器输出最大码率,以 kbps 为单位
iBiasLvl	调整 I 帧 QP 以调节 I 帧的图像质量及其码流大小,范围:[-3,3]
changePos	VBR 开始调整 Qp 时的码率相对于最大码率的比例，取值

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

	范围[50,100]
qualityLvl	视频质量最低水平，范围[0-7]。值越低图像质量越高。
frmQPStep	帧间 QP 变化步长
gopQPStep	GOP 间 QP 变化步长
gopRelation	gop 是否关联

【注意】

无。

IMPEncoderAttrH264Smart

【说明】

Smart 模式属性。

【定义】

```
typedef struct {
    uint32_t        maxQp;
    uint32_t        minQp;
    uint32_t        staticTime;
    uint32_t        maxBitRate;
    int32_t         iBiasLvl;
    uint32_t        changePos;
    uint32_t        qualityLvl;
    uint32_t        frmQPStep;
    uint32_t        gopQPStep;
    bool            gopRelation;
} IMPEncoderAttrH264Smart;
```

【成员】

成员名称	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

maxQp	编码器支持图像最大 QP
minQp	编码器支持图像最小 QP
staticTime	码率统计时间,以秒为单位
maxBitRate	编码器输出最大码率,以 kbps 为单位
iBiasLvl	调整 I 帧 QP 以调节 I 帧的图像质量及其码流大小,范围:[-3,3]
changePos	VBR 开始调整 Qp 时的码率相对于最大码率的比例, 取值范围[50,100]
qualityLvl	视频质量最低水平, 范围[0-7]。值越低图像质量越高。
frmQPStep	帧间 QP 变化步长
gopQPStep	GOP 间 QP 变化步长
gopRelation	gop 是否关联

【注意】

无。

IMPEncoderAttrRcMode

【说明】

定义编码器编码模式属性结构体。

【定义】

```
typedef struct {
    IMPEncoderRcMode                rcMode;
    union {
        IMPEncoderAttrH264FixQP         attrH264FixQp;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
            IMPEncoderAttrH264CBR          attrH264Cbr;
            IMPEncoderAttrH264VBR          attrH264Vbr;
            IMPEncoderAttrH264Smart        attrH264Smart;
            IMPEncoderAttrH265FixQP        attrH265FixQp;
            IMPEncoderAttrH265CBR          attrH265Cbr;
            IMPEncoderAttrH265VBR          attrH265Vbr;
            IMPEncoderAttrH265Smart        attrH265Smart;
        };
    } IMPEncoderAttrRcMode;
```

【成员】

成员名称	描述
rcMode	编码模式
attrH264FixQp	FixQP 模式
attrH264Cbr	CBR 模式
attrH264Vbr	VBR 模式
attrH264Smart	Smart 模式
attrH265FixQp	不支持
attrH265Cbr	不支持
attrH265Vbr	不支持
attrH265Smart	不支持

【注意】

无。

IMPEncoderAttrFrmUsed

【说明】

输入帧使用模式属性。

【定义】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
typedef struct {
    bool            enable;
    EncFrmUsedMode  frmUsedMode;
    uint32_t        frmUsedTimes;
} IMPEncoderAttrFrmUsed;
```

【成员】

成员名称	描述
enable	是否使能输入帧使用模式
frmUsedMode	输入帧使用模式
frmUsedTimes	在重复帧或丢帧模式下每次使用的帧间隔

【注意】

无。

IMPEncoderAttrDenoise

【说明】

去噪属性。

【定义】

```
typedef struct {
    bool            enable;
    int             dnType;
    int             dnIQp;
    int             dnPQp;
} IMPEncoderAttrDenoise;
```

【成员】

成员名称	描述
enable	否使能去噪功能

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

dnType	噪类型,0:忽略，不降噪,1:使用 IP 帧类型降噪,2:使用 I 帧类型降噪
dnIQp	去噪 I 帧量化参数
dnPQp	去噪 P 帧量化参数

【注意】

无。

IMPEncoderAttrHSkip

【说明】

高级跳帧属性。

【定义】

```
typedef struct {
    IMPSkipType     skipType;
    int             m;
    int             n;
    int             maxSameSceneCnt;
    int             bEnableScenecut;
    int             bBlackEnhance;
} IMPEncoderAttrHSkip;
```

【成员】

成员名称	描述
skipType	高级跳帧属性
m	增强帧间隔数。
n	参考帧间隔数
maxSameSceneCnt	同一场景占用 gop 最大数目，仅对 H1M Skip 类型有效,若设

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

	为未大于 0,则 m 值不起作用
bEnableScenecut	是否使能场景切换,仅对 H1M Skip 类型有效
bBlackEnhance	是否使得增强帧以空码流输出

【注意】

无。

IMPEncoderAttrInitHSkip

【说明】

高级跳帧初始化属性。

【定义】

```
typedef struct {
    IMPEncoderAttrHSkip    hSkipAttr;
    IMPSkipType            maxHSkipType;
} IMPEncoderAttrInitHSkip;
```

【成员】

成员名称	描述
hSkipAttr	高级跳帧属性
maxHSkipType	需要使用的最大跳帧类型，影响 rmem 内存大小。

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPEncoderRcAttr

【说明】

码率控制器属性。

【定义】

```
typedef struct {
    IMPEncoderFrmRate          outFrmRate;
    uint32_t                   maxGop;
    IMPEncoderAttrRcMode       attrRcMode;
    IMPEncoderAttrFrmUsed      attrFrmUsed;
    IMPEncoderAttrDenoise      attrDenoise;
    IMPEncoderAttrInitHSkip    attrHSkip;
} IMPEncoderRcAttr;
```

【成员】

成员名称	描述
outFrmRate	编码 Channel 的输出帧率
maxGop	gop 值，必须是帧率的整数倍
attrRcMode	码率控制模式属性
attrFrmUsed	输入帧使用模式属性
attrDenoise	去噪属性
attrHSkip	高级跳帧初始化属性

【注意】

无。

IMPEncoderAttr

【说明】

编码器属性结构体。

【定义】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
typedef struct {
    IMPPayloadType          enType;
    uint32_t                bufSize;
    uint32_t                profile;
    uint32_t                picWidth;
    uint32_t                picHeight;
    IMPEncoderCropCfg       crop;
    IMPEncoderUserDataCfg   userData;
} IMPEncoderAttr;
```

【成员】

成员名称	描述
enType	编码协议类型
bufSize	配置 buffer 大小
profile	编码的等级
picWidth	编码图像宽度
picHeight	码图像高度
crop	编码器裁剪属性
userData	插入用户数据属性

【注意】

无。

IMPEncoderCHNAttr

【说明】

编码 Channel 属性。

【定义】

```
typedef struct {
    IMPEncoderAttr      encAttr;
    IMPEncoderRcAttr    rcAttr;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
        bool                    bEnableIvdc;
    } IMPEncoderCHNAttr;
```

【成员】

成员名称	描述
encAttr	编码器属性结构体
rcAttr	码率控制器属性结构体
bEnableIvdc	直通使能标志

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

目录

4 视频源模块 1
4.1 功能介绍 1
4.2 模块操作流程 2
4.3 FrameSource 模块 API 4
4.4 FrameSource 数据类型 25

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

4 视频源模块

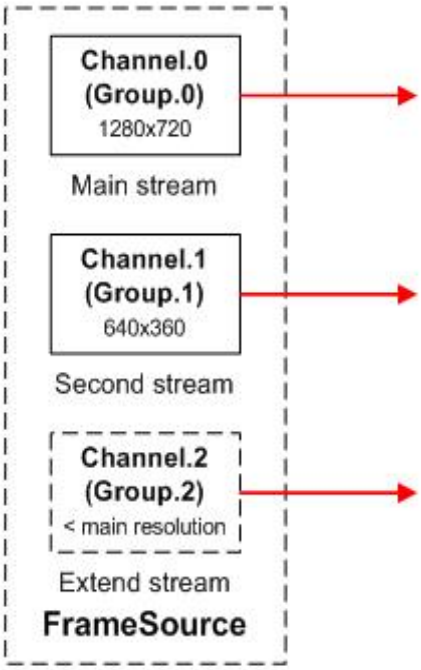
4.1 功能介绍

视频源，是 IMP 系统的图像数据源，可设置图像的分辨率、裁减、缩放等属性，以及后端降噪功能

FrameSource 是一个数据流相关概念，可以设置图像分辨率，格式等，并向后端提供原始图像。

FrameSource 的结构如下图：

图 4-1 FrameSource



如左图所示，FrameSource 有三种（主码流、次码流和扩展码流）输出，三种输出均可用来编码，其中：

- Channel 0 一般作为高清视频流
- Channel 1 一般作为标清视频流，或者 IVS 智能算法的数据源
- Channel 2 是拓展通道，在特殊应用下使用，一般不建议使用
- 其实还有一个 Channel 3 是拓展通道，同 Channel 2，所以在这里就没有画出来，在特殊应用下使用，一般情况下不建议使用。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

- 在这里扩展通道的码流其实是从主码流（Channel 0）或次码流（Channel 1）拷贝过来的，作用就是把拷贝过来的视频进行一些其他的处理，来达到一些主码流或次码流不方便做的功能，（比如 CSC）或并行运行。但是扩展通道的码流尺寸一般不建议大于或小于所拷贝码流通道的尺寸，这样会额外消耗 CPU，较低 CPU 效率。

图 4-2 T23 Framesource 获得流程

FrameSource 是 ISP 出图功能的软件抽象。Sensor 输出的一定分辨率的图像经过 fcrop、sclaer 和 crop 三级处理后输出。fcrop 是前级裁剪功能，sclaer 是缩放功能，crop 是后级裁剪功能。这些功能在 FrameSource 通道参数中都有对应。注意：T30/T21 的区别是没有 fcrop 功能。

4.2 模块操作流程

4.2.1 Init 流程

1) 创建通道
2) 设置通道
3) 使能通道

4.2.2 Exit 流程

1) 失能通道
2) 注销通道

FrameSource 的初始化步骤如下（以两路输出为例）：

```
1. IMPFSChnAttr fs_chn_attr;
2. fs_chn_attr.pixFmt = PIX_FMT_NV12;
3. fs_chn_attr.outFrmRateNum = SENSOR_FRAME_RATE;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
4.  fs_chn_attr.outFrmRateDen = 1;
5.  fs_chn_attr.nrVBs = 3;
6.  fs_chn_attr.type = FS_PHY_CHANNEL;
7.
8.  fs_chn_attr.crop.enable = 1;
9.  fs_chn_attr.crop.top = 0;
10. fs_chn_attr.crop.left = 0;
11. fs_chn_attr.crop.width = 960;
12. fs_chn_attr.crop.height = 640;
13.
14. fs_chn_attr.scaler.enable = 1;
15. fs_chn_attr.scaler.outwidth = 320;
16. fs_chn_attr.scaler.outheight = 240;
17.
18. ret = IMP_FrameSource_CreateChn(0, &fs_chn_attr);
19. //step.1 创建 Channel 0
20. if (ret < 0) {
21. printf("FrameSource_CreateChn(0) error\n");
22. goto createchn_err;
23. }
24.
25. ret = IMP_FrameSource_EnableChn(0);
26. //step.2 Enable Channel 0，channel 0 开始输出图像
27. if (ret < 0) {
28. printf("EnableChn(0) error\n";
29. return enablechn_err;
30. }
31.
32. //至此，FrameSource 开始向后端 Group 传递数据
33.
34. ret = IMP_FrameSource_DisableChn(0);
35. //step.3 Disable channel 0，channel 0 停止输出图像
36. if (ret < 0) {
37. printf("FrameSource_DisableChn(0) error\n");
38. return disablechn_err;
39. }
40.
41. ret = IMP_FrameSource_DestroyChn(0); //step.4 销毁 channel 0
42. if (ret < 0) {
43. printf("FrameSource_DestroyChn error\n");
44. return destorychn_err;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
}
```

更多使用方法请参考 Samples。

4.3 FrameSource 模块 API

表 4-1 T23 FrameSource API 功能表

API 名称	功能
IMP_FrameSource_CreateChn	创建通道
IMP_FrameSource_DestroyChn	销毁通道
IMP_FrameSource_EnableChn	使能通道
IMP_FrameSource_DisableChn	关闭通道
IMP_FrameSource_SetSource	指定扩展通道源通道接口
IMP_FrameSource_GetChnAttr	获得通道属性
IMP_FrameSource_SetChnAttr	设置通道属性
IMP_FrameSource_SetFrameDepth	设置可获取的图像最大深度
IMP_FrameSource_GetFrameDepth	获取的图像最大深度
IMP_FrameSource_GetFrame	获取的图像
IMP_FrameSource_GetTimedFrame	获取指定时间的图像
IMP_FrameSource_ReleaseFrame	释放获取的图像
IMP_FrameSource_SnapFrame	获取图像
IMP_FrameSource_SetMaxDelay	设置最大延迟帧数
IMP_FrameSource_GetMaxDelay	获取最大延迟帧数
IMP_FrameSource_SetDelay	设置延迟帧数
IMP_FrameSource_GetDelay	获取延迟帧数
IMP_FrameSource_SetChnFifoAttr	设置通道最大缓存 FIFO 属性
IMP_FrameSource_GetChnFifoAttr	获取通道最大缓存 FIFO 属性
IMP_FrameSource_SetPool	绑定 chnnel 到内存池中，即 FrameSource 申请 mem 从 pool 申请
IMP_FrameSource_GetPool	通过 channel ID 获取 poolID
IMP_FrameSource_ChnStatQuery	通过 channel ID 获取 framesource 通道的创建状态
IMP_FrameSource_SetDirectModeAttr	设置双摄直通时，根据编码数据缓存阈值决定编码或丢帧
IMP_FrameSource_GetDirectModeAttr	获取双摄直通时编码数据缓存值

IMP_FrameSource_CreateChn

【功能】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

创建通道。

【语法】

int IMP_FrameSource_CreateChn(int chnNum, IMPFSChnAttr *chn_attr);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入
chnAttr	通道属性结构体指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 创建通道，给后端模块提供数据源。
- 可以设置通道的相关属性，包括：图片宽度，图片高度，图片格式，通道的输出帧率，缓存 buf 数，裁剪和缩放属性；
- 对于 T10，通道 0、1 只能被设置为物理通道，通道 2,3 只能被设置为拓展通道。

IMP_FrameSource_DestroyChn

【功能】

销毁通道。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_FrameSource_DestroyChn(int chnNum);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 如果程序调用过 IMP_FrameSource_EnableChn，一定要调用 IMP_FrameSource_DisableChn 之后，再使用此函数。

IMP_FrameSource_EnableChn

【功能】

使能通道。

【语法】

int IMP_FrameSource_EnableChn(int chnNum);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须确保所使能的通道已经创建。

IMP_FrameSource_DisableChn

【功能】

关闭通道。

【语法】

int IMP_FrameSource_DisableChn(int chnNum);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

无。

IMP_FrameSource_SetSource

【功能】

指定扩展通道源通道接口。

【语法】

int IMP_FrameSource_SetSource(int extchnNum, int sourcechnNum);

【形参】

参数名称	描述	输入/输出
extchnNum	扩展通道号	输入
sourcechnNum	源通道号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 在 IMP_FrameSource_CreateChn 之后, IMP_FrameSource_EnableChn 之前调用。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_FrameSource_GetChnAttr

【功能】

获得通道属性。

【语法】

int IMP_FrameSource_GetChnAttr(int chnNum, IMPFSChnAttr *chnAttr);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入
chnAttr	通道属性结构体指针	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 可以获得通道的相关属性，包括：图片宽度，图片高度，图片格式，通道的输出帧率，缓存 buf 数，裁剪和缩放属性。

IMP_FrameSource_SetChnAttr

【功能】

设置通道属性。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_FrameSource_SetChnAttr(int chnNum,const IMPFSChnAttr *chnAttr);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入
chnAttr	通道属性结构体指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 可以设置通道的相关属性，包括：图片宽度，图片高度，图片格式，通道的输出帧率，缓存 buf 数，裁剪和缩放属性。

IMP_FrameSource_SetFrameDepth

【功能】

设置可获取的图像最大深度。

【语法】

int IMP_FrameSource_SetFrameDepth(int chnNum, int depth);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

depth	设置可获取的图像最大深度值	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 此接口用于设置某一通道缓存的视频图像帧数。当用户设置缓存多帧视频图像时，用户可以获取到一定数目的连续图像数据；
- 若指定 depth 为 0，表示不需要系统为该通道缓存图像，故用户获取不到该通道图像数据。系统默认不为通道缓存图像，即 depth 默认为 0；
- 系统将自动更新最旧的图像数据，保证用户一旦开始获取，就可获取到最近最新的图像；
- 系统因获取不到图像而自动停止缓存新的图像，用户也不能获取新的图像。因此建议用户保证获取和释放接口配对使用；
- 系统将自动更新用户仍未获取的最旧的图像数据，保证缓存的图像队列为最近最新的图像。由于用户不能保证获取速度，导致获取的可能不是连续的图像；
- 此函数可以调用位置，没有要求。但是只能使用一次。

IMP_FrameSource_GetFrameDepth

【功能】

获取的图像最大深度。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_FrameSource_GetFrameDepth(int chnNum, int *depth);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入
depth	设置可获取的图像最大深度值	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

无。

IMP_FrameSource_GetFrame

【功能】

获取的图像。

【语法】

int IMP_FrameSource_GetFrame(int chnNum, IMPFrameInfo **frame);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入
frame	获取的图像	输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 此接口可以获取指定通道的视频图像信息。图像信息主要包括：图像的宽度、高度、像素格式以及图片数据起始地址；
- 此接口需在通道已启用后才有效；
- 支持多次获取后再释放，但建议获取和释放接口配对使用；
- 该接口默认超时时间为 2s，即 2s 内仍未获取到图像，则超时返回。

IMP_FrameSource_GetTimedFrame

【功能】

获取指定时间的图像。

【语法】

int IMP_FrameSource_GetTimedFrame(int chnNum,

IMPFrameTimestamp *framets, int block, void *framedata, IMPFrameInfo *frame);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入
framets	时间信息	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

block	阻塞属性	输入
framedata	拷贝图像的内存指针	输入
frame	获取到图像信息	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 此接口可以获取指定通道指定时间的视频图像信息。图像信息主要包括：图像的宽度、高度、像素格式以及图片数据；
- 此接口需在通道已启用后才有效；
- 此接口需要先设置 IMP_FrameSource_SetMaxDelay 和 IMP_FrameSource_SetDelay。

IMP_FrameSource_ReleaseFrame

【功能】

释放获取的图像。

【语法】

int IMP_FrameSource_ReleaseFrame(int chnNum, IMPFrameInfo *frame);

【形参】

参数名称	描述	输入/输出
chnNum	通道号	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

frame	释放获取的图像	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

无。

IMP_FrameSource_SnapFrame

【功能】

获取图像。

【语法】

```
int IMP_FrameSource_SnapFrame(int chnNum,
IMPPixelFormat fmt, int width, int height, void *framedata, IMPFrameInfo *frame);
```

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入
fmt	图像格式	输入
width	图像宽度	输入
height	图像高度	输入
framedata	拷贝图像的内存指针	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 此接口可以获取一帧指定格式和大小的图像；
- 目前格式支持 NV12，YUYV422；
- 大小和通道分辨率一致；
- 不需要调用 IMP_FrameSource_SetFrameDepth 接口；
- 此接口需在通道已启用后才有效。

IMP_FrameSource_SetMaxDelay

【功能】

设置最大延迟帧数。

【语法】

int IMP_FrameSource_SetMaxDelay(int chnNum, int maxcnt);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入
maxcnt	最大延迟，单位帧	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 使用时需要在函数 IMP_FrameSource_CreateChn 与 IMP_FrameSource_EnableChn 之间调用。

IMP_FrameSource_GetMaxDelay

【功能】

获取最大延迟帧数。

【语法】

int IMP_FrameSource_GetMaxDelay(int chnNum, int *maxcnt);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入
maxcnt	最大延迟，单位帧	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 使用时需要在函数 IMP_FrameSource_CreateChn 之后。

IMP_FrameSource_SetDelay

【功能】

设置延迟帧数。

【语法】

int IMP_FrameSource_SetDelay(int chnNum, int cnt);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入
cnt	延迟，单位帧	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 使用时需要在函数 IMP_FrameSource_SetMaxDelay 之后调用。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_FrameSource_GetDelay

【功能】

获取延迟帧数。

【语法】

int IMP_FrameSource_GetDelay(int chnNum, int *cnt);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入
cnt	延迟，单位帧	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 使用时需要在函数 IMP_FrameSource_CreateChn 之后。

IMP_FrameSource_SetChnFifoAttr

【功能】

设置通道最大缓存 FIFO 属性。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_FrameSource_SetChnFifoAttr(int chnNum, IMPFSChnFifoAttr *attr);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入
attr	FIFO 属性，包括 FIFO 最大深度，单位帧；FIFO 类型	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 使用时需要在函数 IMP_FrameSource_CreateChn 与 IMP_FrameSource_EnableChn 之间调用。

IMP_FrameSource_GetChnFifoAttr

【功能】

获取通道最大缓存 FIFO 属性。

【语法】

int IMP_FrameSource_GetChnFifoAttr(int chnNum, IMPFSChnFifoAttr *attr);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

attr	FIFO 属性	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 使用时需要在函数 IMP_FrameSource_CreateChn 之后。

IMP_FrameSource_SetPool

【功能】

绑定 chnnel 到内存池中，即 FrameSource 申请 mem 从 pool 申请。

【语法】

int IMP_FrameSource_SetPool(int chnNum, int poolID);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入
poolID	内存池编号	输入

【返回值】

返回值	描述
0	成功。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 为了解决 rmem 碎片化，将该 channel FrameSource 绑定到对应的 mempool 中，FramSource 申请 mem 就在 mempool 中申请，若不调用，FramSource 会在 rmem 中申请此时对于 rmem 来说会存在碎片的可能。
- ChannelId 必须大于等于 0 且小于 32

IMP_FrameSource_GetPool

【功能】

通过 channel ID 获取 poolID。

【语法】

int IMP_FrameSource_GetPool(int chnNum);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入

【返回值】

返回值	描述
>=0 && <32	成功。
<0	失败。

【依赖】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

- 通过 ChannelId 获取 poolId，客户暂时用不到。

IMP_FrameSource_ChnStatQuery

【功能】

通过 channel ID 获取 framesource 通道的创建状态；0：未创建，1：通道创建，2：通道正在运行.。

【语法】

int IMP_FrameSource_ChnStatQuery(int chnNum, IMPFSChannelState *pstate);

【形参】

参数名称	描述	输入/输出
chnNum	通道的编号	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_framesource.h

库文件：libimp.a / libimp.so

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_FrameSource_SetDirectModeAttr

【功能】

设置双摄直通时，根据编码数据缓存阈值决定编码或丢帧。

【语法】

int IMP_FrameSource_SetDirectModeAttr(int chnNum, int data_threshold);

【形参】

参数名称	描述	输入/输出
chnNum	通道编号	输入
data_threshold	编码缓存阈值	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 使用时需要在函数 IMP_FrameSource_CreateChn 之后。

【举例】

无。

IMP_FrameSource_GetDirectModeAttr

【功能】

获取双摄直通时编码数据缓存值。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_FrameSource_GetDirectModeAttr(int chnNum, int *data_threshold);

【形参】

参数名称	描述	输入/输出
chnNum	通道编号	输入
data_threshold	编码缓存阈值	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_encoder.h

库文件：libimp.a / libimp.so

【注意】

- 使用时需要在函数 IMP_FrameSource_CreateChn 之后。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

4.4 FrameSource 数据类型

表 4-2 T23 FrameSource 数据类型

名称	定义
IMPFSChnCrop	通道裁剪结构体
IMPFSChnScaler	通道缩放结构体
IMPFSChnType	通道缩放结构体
IMPFSChnFifoType	通道 FIFO 类型
IMPFSChannelState	通道 FIFO 类型
IMPFSChnFifoAttr	通道 FIFO 属性结构体
IMPFSChnAttr	通道属性结构体

IMPFSChnCrop

【说明】

通道裁剪结构体。

【定义】

```
typedef struct {
        int enable;
        int left;
        int top;
        int width;
        int height;
} IMPFSChnCrop;
```

【成员】

成员名称	描述
enable	使能裁剪功能
left	裁剪左起始点
top	裁剪上起始点
width	图片裁剪宽度
height	图片裁剪高度

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

无。

IMPFSChnScaler

【说明】

通道缩放结构体。

【定义】

```
typedef struct {
        int enable;
        int outwidth;
        int outheight;
} IMPFSChnScaler;
```

【成员】

成员名称	描述
enable	使能缩放功能
outwidth	缩放后图片宽度
outheight	缩放后图片高度

【注意】

无。

IMPFSChnType

【说明】

通道缩放结构体。

【定义】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
typedef enum {
        FS_PHY_CHANNEL,
        FS_EXT_CHANNEL,
} IMPFSChnType;
```

【成员】

成员名称	描述
FS_PHY_CHANNEL	物理通道
FS_EXT_CHANNEL	拓展通道

【注意】

无。

IMPFSChnFifoType

【说明】

通道 FIFO 类型。

【定义】

```
typedef enum {
        FIFO_CACHE_PRIORITY = 0,
        FIFO_DATA_PRIORITY,
} IMPFSChnFifoType;
```

【成员】

成员名称	描述
FIFO_CACHE_PRIORITY	FIFO 优先缓存，然后输出数据
FIFO_DATA_PRIORITY	FIFO 优先输出数据，然后缓存

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPFSChannelState

【说明】

通道状态。

【定义】

```
typedef enum {
    IMP_FSCHANNEL_STATE_CLOSE,
    IMP_FSCHANNEL_STATE_OPEN,
    IMP_FSCHANNEL_STATE_RUN,
} IMPFSChannelState;
```

【成员】

成员名称	描述
FSCHANNEL_STATE_CLOSE	fs 通道未创建或已关闭销毁
FSCHANNEL_STATE_OPEN	fs 通道已创建，未使能
FSCHANNEL_STATE_RUN	fs 通道已创建，已使能

【注意】

无。

IMPFSChnFifoAttr

【说明】

通道 FIFO 属性结构体。

【定义】

```
typedef struct {
        int maxdepth;
        IMPFSChnFifoType type;
} IMPFSChnFifoAttr;
```

【成员】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

成员名称	描述
maxdepth	FIFO 最大深度
IMPFSChnFifoType type	通道 FIFO 类型

【注意】

无。

IMPFSChnAttr

【说明】

通道属性结构体。

【定义】

```
typedef struct {
        int picWidth;
        int picHeight;
        IMPPixelFormat pixFmt;
        IMPFSChnCrop crop;
        IMPFSChnScaler scaler;
        int outFrmRateNum;
        int outFrmRateDen;
        int nrVBs;
        IMPFSChnType type;
        IMPFSChnCrop fcrop;
} IMPFSChnAttr;
```

【成员】

成员名称	描述
picWidth	图片宽度
picHeight	图片高度

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPPixelFormat pixFmt	图片格式
IMPFSChnCrop crop;	图片裁剪属性
IMPFSChnScaler scaler;	图片缩放属性
outFrmRateNum	通道的输出帧率分子
outFrmRateDen	通道的输出帧率分母
nrVBs	Video buffer 数量
IMPFSChnType type	通道类型
IMPFSChnCrop fcrop	图片裁剪属性

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

目录

5 IPU 模块.....1
5.1 模块简介.....1
5.2 组织框图.....2
5.3 关键词说明.....2
5.4 IMP_OSD 图像信息叠加模块.....3
5.5 API 参考.....4
5.6 数据类型.....20
5.7 SDK Sample 介绍.....26

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

5 IPU 模块

5.1 模块简介

IPU 是图像处理单元，全称是（Image Processing Unit）。支持对从 ISP 模块采集的图像到视频显示模块前的数据处理。例如，将特定的信息叠加到视频中，如点阵数据、直线、矩形框、矩形遮挡、图片数据等。

IPU 模块对图像的操作主要包含 OSD 模块和 CSC 模块。OSD 模块主要在视频帧上叠加框线、矩形遮挡、图片等数据；CSC 可以实现把输入的视频帧转化成硬件支持的图像，如 HSV、 NV12、 NV21、RGB32、和 ARGB 模式中任意一种格式。

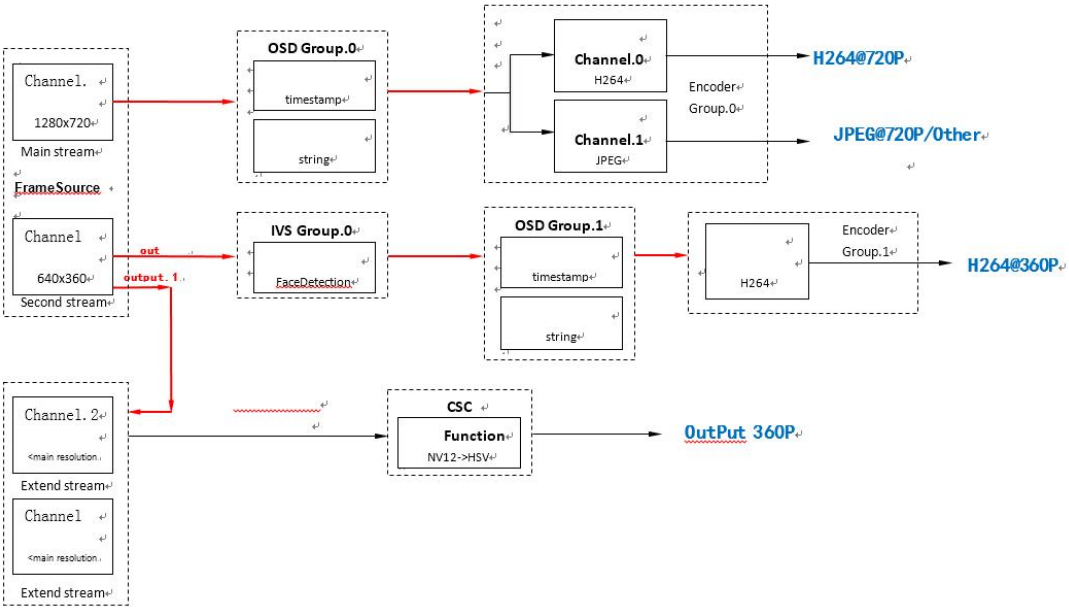
叠加在视频数据中的数据信息称为 OSD 区域（Region），系统中使用 OSD 设备模块（OSD device module）来管理这些 OSD 区域，OSD 设备模块支持区域创建，支持通过区域属性设置接口配置区域的类型（如：配置区域显示的属性、位置坐标、前景背景等），支持区域的销毁等操作。在 OSD 区域完成创建、注册、设置属性等操作后，OSD 区域内部会统一调用 IPU 进行绘制。

用户可以通过 IMP_OSD_CreateRgn 创建一个区域，之后使用 IMP_OSD_RegisterRgn 将区域注册到 OSD 的组中，再使用 IMP_OSD_SetRgnAttr 设置区域属性将叠加的信息添加到 OSD 模块中。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

5.2 组织框图

图 5-1 OSD、CSC 上下文关系图



5.3 关键词说明

- IPU

  IPU(Image Processing Unit)，图像处理单元

- OSD

  OSD(On-screen Display)，屏幕菜单式调节方式

- REGION

  REGION (Region)，模块区域

- DEVICE MOUDLE

  设备管理模块，系统内部通过 device 进行各个模块的管理

- CSC

  CSC 的全称是 color space conversion。实现的功能是把输入的图片格式转化成硬件支持的任意一种格式

- 区域属性

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

用户需要将待显示的区域信息配置好，这些属性中包含要显示的信息，如待显示的前景地址信息，前景在背景中的位置信息，前景像素格式等。

5.4 IMP_OSD 图像信息叠加模块

5.4.1 功能介绍

OSD 模块，可在视频流上叠加图片、位图、直线、矩形框。OSD 的全称是 On-Screen Display。模块的功能是在每帧视频中叠加线条、图片等信息。

在编译完成后，在设备中运行如下 sample 即可生成有叠加 sample 中配置的 OSD 文件。该文件可以放入 VLC 等播放软件进行播放。

应用文件	功能	执行命令	执行结果
sample-OSD.c	视频叠加 OSD 信息	./sample-OSD	生成视频文件 stream-0.h264

5.4.2 模块操作流程

5.4.2.1 Init 流程

1) 创建 OSD 组，使用 IMP_OSD_CreateGroup 创建一个 OSD 组；
2) 创建 OSD 区域，使用 IMP_OSD_CreateRgn 创建一个区域；
3) 注册 OSD 区域到 OSD 组中，通过 IMP_OSD_RegisterRgn 将创建的区域注册到 OSD 组中；
4) 设置 OSD 组区域属性和区域属性，通过 IMP_OSD_SetRgnAttr 设置区域属性，通过 IMP_OSD_SetGrpRgnAttr 设置区域组属性；
5) 设置 OSD 功能显示开关，通过 IMP_OSD_ShowRgn 开启 osd 显示；
6) 绑定 OSD 组到系统中，通过 IMP_System_Bind 将 OSD 模块和其他模块进行绑定。

5.4.2.2 Exit 流程

1) 解绑，通过 IMP_System_UnBind 进行解绑；

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2) 关闭 OSD 功能显示开关，IMP_OSD_ShowRgn 关闭 osd 显示；

3) 注销 OSD 组中的 OSD 区域，通过 IMP_OSD_UnRegisterRgn 注销区域

4) 销毁 OSD 区域，通过 IMP_OSD_DestroyRgn 销毁区域

5) 销毁 OSD 组，最后通过 IMP_OSD_DestroyGroup 销毁组

注：其他具体流程细节请参看 sample

5.5 API 参考

表 5-1 T23 IPU API 功能表

API 名称	功能
IMP_OSD_CreateGroup	创建一个 OSD 组
IMP_OSD_DestroyGroup	销毁一个 OSD 组
IMP_OSD_CreateRgn	创建一个 OSD 区域
IMP_OSD_DestroyRgn	销毁一个 OSD 区域
IMP_OSD_RegisterRgn	注册区域
IMP_OSD_UnRegisterRgn	注销区域
IMP_OSD_SetRgnAttr	设置区域属性
IMP_OSD_GetRgnAttr	获取区域属性
IMP_OSD_SetRgnAttrWithTimestamp	设置区域属性生效时间
IMP_OSD_UpdateRgnAttrData	更新区域数据属性
IMP_OSD_SetGrpRgnAttr	设置 OSD 组区域属性
IMP_OSD_GetGrpRgnAttr	获取 OSD 组区域属性
IMP_OSD_ShowRgn	设置组区域是否显示
IMP_OSD_Start	设置开始 OSD 组的显示
IMP_OSD_Stop	设置停止 OSD 组的显示

IMP_OSD_CreateGroup

【功能】

创建 OSD 组。

【语法】

int IMP_OSD_CreateGroup(int grpNum);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
grpNum	OSD 组号; 取值范围:[0, NR_MAX_OSD_GROUPS - 1]	输入

【返回值】

【返回值】	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【举例】

```
int grpNum = 0;
IMPRgnHandle rHanderLogo;
IMPOSDRgnAttr rAttrLogo;
IMPOSDGrpRgnAttr grAttrLogo;
IMPOSDRgnCreateStat stStatus;
IMPOSDRgnRegisterStat stRigStatus;
memset(&stRigStatus,0,sizeof(IMPOSDRgnRegisterStat));
memset(&stStatus,0,sizeof(IMPOSDRgnCreateStat));
memset(&rAttrLogo, 0, sizeof(IMPOSDRgnAttr));
memset(&grAttrLogo, 0, sizeof(IMPOSDGrpRgnAttr));

if (IMP_OSD_CreateGroup(grpNum) < 0) {
        IMP_LOG_ERR(TAG, "IMP_OSD_CreateGroup(%d) error !\n", grpNum);
        return -1;
}

rHanderLogo = IMP_OSD_CreateRgn(NULL);
if (rHanderLogo == INVHANDLE) {
    IMP_LOG_ERR(TAG, "IMP_OSD_CreateRgn Logo error !\n");
    return -1;
}
//query osd rgn create status
ret = IMP_OSD_RgnCreate_Query(rHanderLogo,&stStatus);
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
24. if(ret < 0){
25.     IMP_LOG_ERR(TAG, "IMP_OSD_RgnCreate_Query error !\n");
26.     return NULL;
27. }
28. ret = IMP_OSD_RegisterRgn(rHanderLogo, grpNum, NULL);
29. if (ret < 0) {
30.     MP_LOG_ERR(TAG, "IVS IMP_OSD_RegisterRgn failed\n");
31.     return -1;
32. }
33.
34. ret = IMP_OSD_RgnRegister_Query(rHanderFont, grpNum,&stRigStatus);
35. if (ret < 0) {
36.     IMP_LOG_ERR(TAG, "IMP_OSD_RgnRegister_Query failed\n");
37.     return NULL;
38. }
39. int picw = 100;
40. int pich = 100;
41. rAttrLogo.type = OSD_REG_PIC;
42. rAttrLogo.rect.p0.x = SENSOR_WIDTH - 100;
43. rAttrLogo.rect.p0.y = SENSOR_HEIGHT - 100;
44. rAttrLogo.rect.p1.x  =  rAttrLogo.rect.p0.x+picw-1;          /*p0  is  start ,
    and p1 well be epual p0+width(or heigth)-1*/
45. rAttrLogo.rect.p1.y = rAttrLogo.rect.p0.y+pich-1;
46. rAttrLogo.fmt = PIX_FMT_BGRA;
47. rAttrLogo.data.picData.pData = logodata_100x100_bgra;
48. ret = IMP_OSD_SetRgnAttr(rHanderLogo, &rAttrLogo);
49. if (ret < 0) {
50.     IMP_LOG_ERR(TAG, "IMP_OSD_SetRgnAttr Logo error !\n");
51.     return NULL;
52. }
53.
54. if (IMP_OSD_GetGrpRgnAttr(rHanderLogo, grpNum, &grAttrLogo) < 0) {
55.     IMP_LOG_ERR(TAG, "IMP_OSD_GetGrpRgnAttr Logo error !\n");
56.     return NULL;
57.
58. }
59. memset(&grAttrLogo, 0, sizeof(IMPOSDGrpRgnAttr));
60. grAttrLogo.show = 0;
61.
62. /* Set Logo global alpha to 0x7f, it is semi-transparent. */
63. grAttrLogo.gAlphaEn = 1;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
grAttrLogo.fgAlhpa = 0x7f;
grAttrLogo.layer = 2;

if (IMP_OSD_SetGrpRgnAttr(rHanderLogo, grpNum, &grAttrLogo) < 0) {
    IMP_LOG_ERR(TAG, "IMP_OSD_SetGrpRgnAttr Logo error !\n");
    return NULL;
}

/*other operation*/
```

IMP_OSD_DestroyGroup

【功能】

销毁 OSD 组。

【语法】

int IMP_OSD_DestroyGroup(int grpNum);

【形参】

参数名称	描述	输入/输出
grpNum	OSD 组号; 取值范围:[0, NR_MAX_OSD_GROUPS - 1]	输入

【返回值】

【返回值】	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【注意】

- 此接口需要对应 grpNum 的组已经创建。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

示例代码：

```
int ret = 0;
int grpNum = 0;
ret = IMP_OSD_ShowRgn(rHanderLogo, grpNum, 0);
if (ret < 0) {
    IMP_LOG_ERR(TAG, "IMP_OSD_ShowRgn close timeStamp error\n");
}

ret = IMP_OSD_UnRegisterRgn(rHanderLogo, grpNum);
if (ret < 0) {
    IMP_LOG_ERR(TAG, "IMP_OSD_UnRegisterRgn timeStamp error\n");
}

IMP_OSD_DestroyRgn(rHanderLogo);

ret = IMP_OSD_DestroyGroup(grpNum);
if (ret < 0) {
    IMP_LOG_ERR(TAG, "IMP_OSD_DestroyGroup(0) error\n");
    return -1;
}
return ret;
```

IMP_OSD_CreateRgn

【功能】

创建 OSD 区域。

【语法】

IMPRgnHandle IMP_OSD_CreateRgn(IMPOSDRgnAttr *prAttr);

【形参】

参数名称	描述	输入/输出
prAttr	Osd 区域属性结构体指针	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
大于等于 0	成功，对应相应的区域 取值范围:[0, NR_MAX_OSD_REGION-1]
小于 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【注意】

此接口需要对应 grpNum 的组已经创建。

【举例】

参见 IMP_OSD_CreateGroup 举例。

IMP_OSD_DestroyRgn

【功能】

销毁 OSD 区域。

【语法】

void IMP_OSD_DestroyRgn(IMPRgnHandle handle);

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入

【返回值】

无。

【依赖】

头文件：imp_osd.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

此接口需要对应 handle 号的区域已经创建。

【举例】

参见 IMP_OSD_DestroyGroup 举例。

IMP_OSD_RegisterRgn

【功能】

注册 OSD 区域。

【语法】

Int IMP_OSD_RegisterRgn(IMPRgnHandle handle, int grpNum,

IMPOSDGrpRgnAttr *pgrAttr);

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入
grpNum	Osd 组号； 取值范围:[0, NR_MAX_OSD_GROUPS - 1]	输入
pgrAttr	OSD 组显示属性结构体指针	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

- 此接口需要对应 handle 号的区域已经创建。

【举例】

参见 IMP_OSD_CreateGroup 举例。

IMP_OSD_UnRegisterRgn

【功能】

销毁 OSD 区域。

【语法】

int IMP_OSD_UnRegisterRgn(IMPRgnHandle handle, int grpNum);

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入
grpNum	Osd 组号	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【注意】

- 此接口需要对应 handle 号的区域已经创建。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参见 IMP_OSD_DestroyGroup 举例。

IMP_OSD_SetRgnAttr

【功能】

设置区域属性。

【语法】

int IMP_OSD_SetRgnAttr(IMPRgnHandle handle, IMPOSDRgnAttr *prAttr);

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入
prAttr	OSD 区域属性结构体指针	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【举例】

参见 IMP_OSD_CreateGroup 举例。

IMP_OSD_GetRgnAttr

【功能】

获取区域属性。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_OSD_GetRgnAttr(IMPRgnHandle handle, IMPOSDRgnAttr *prAttr)；

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入
prAttr	Osd 区域属性结构体指针	输出

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【举例】

参见 IMP_OSD_CreateGroup 举例。

IMP_OSD_SetRgnAttrWithTimestamp

【功能】

设置区域属性和生效时间。

【语法】

int IMP_OSD_SetRgnAttrWithTimestamp(IMPRgnHandle handle,

IMPOSDRgnAttr *prAttr, IMPOSDRgnTimestamp *prTs);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

handle	已创建区域的句柄	输入
prAttr	OSD 区域属性结构体指针	输入
prTs	生效时间	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【举例】

无。

IMP_OSD_UpdateRgnAttrData

【功能】

更新区域数据属性，只针对 OSD_REG_BITMAP 和 OSD_REG_PIC 、OSD_REG_PIC_RMEM 的区域类型。

【语法】

```
int IMP_OSD_UpdateRgnAttrData(IMPRgnHandle handle,
IMPOSDRgnAttrData *prAttrData);
```

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入
prAttrData	OSD 区域数据属性结构体指针	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【举例】

无。

IMP_OSD_SetGrpRgnAttr

【功能】

设置 OSD 组区域属性。

【语法】

int IMP_OSD_SetGrpRgnAttr(IMPRgnHandle handle, int grpNum,

IMPOSDGrpRgnAttr *pgrAttr);

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入
grpNum	OSD 组号	输入
pgrAttr	OSD 组区域属性	输入

【返回值】

【返回值】	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【举例】

参见 IMP_OSD_CreateGroup 举例。

IMP_OSD_GetGrpRgnAttr

【功能】

获取 OSD 组区域属性。

【语法】

```
int IMP_OSD_GetGrpRgnAttr(IMPRgnHandle handle, int grpNum,
IMPOSDGrpRgnAttr *pgrAttr);
```

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入
grpNum	OSD 组号	输入
pgrAttr	OSD 组区域属性结构体指针	输出

【返回值】

返回值	描述
0	成功
非 0	失败

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【注意】

- 调用此 API 时要求对应的 OSD 组已经创建,区域已经创建并注册。

【举例】

参见 IMP_OSD_CreateGroup 举例。

IMP_OSD_ShowRgn

【功能】

设置 OSD 显示开关。

【语法】

int IMP_OSD_ShowRgn(IMPRgnHandle handle, int grpNum, int showFlag);

【形参】

参数名称	描述	输入/输出
handle	已创建区域的句柄	输入
grpNum	OSD 组号	输入
showFlag	OSD 显示开关标志，0 为关闭，1 为开启	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【举例】

参见 IMP_OSD_DestroyGroup 举例。

IMP_OSD_Start

【功能】

设置开始 OSD 组的显示。

【语法】

int IMP_OSD_Start(int grpNum);

【形参】

参数名称	描述	输入/输出
grpNum	OSD 组号	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_OSD_Stop

【功能】

设置停止 OSD 组的显示。

【语法】

int IMP_OSD_Stop(int grpNum);

【形参】

参数名称	描述	输入/输出
grpNum	OSD 组号	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_osd.h

库文件：libimp.a / libimp.so

【注意】

- 调用此 API 时要求对应的 OSD 组已经创建。

【举例】

无。

5.6 数据类型

IMP_OSD 相关结构类型如下：

表 5-2　T23 IPU 数据类型表

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

数据类型	功能
IMPOsdColour	定义非 IPU 绘制的 OSD 颜色类型
IMPOsdRgnType	定义 OSD 区域类型
IMPOSDRgnAttrData	定义 OSD 区域的数据
IMPOSDRgnAttr	定义 OSD 区域属性
IMPOSDRgnTimestamp	定义 OSD 区域设置生效时间戳
IMPOSDGrpRgnAttr	定义 OSD 组区域属性

IMPOsdColour

【说明】

OSD 颜色类型，颜色格式为 bgra。

【定义】

```
typedef enum {
    OSD_BLACK  = 0xff000000,
    OSD_WHITE = 0xffffffff,
    OSD_RED          = 0xffff0000,
    OSD_GREEN = 0xff00ff00,
    OSD_BLUE    = 0xff0000ff,
}IMPOsdColour;
```

【元素】

元素名称	描述
OSD_BLACK	绘制区域内容颜色为黑色
OSD_WHITE	绘制区域内容颜色为白色
OSD_RED	绘制区域内容颜色为红色
OSD_GREEN	绘制区域内容颜色为绿色

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPOsdRgnType

【说明】

OSD 区域类型枚举。

【定义】

```
typedef enum {
    OSD_REG_INV          = 0,
    OSD_REG_LINE         = 1,
    OSD_REG_RECT         = 2,
    OSD_REG_BITMAP       = 3,
    OSD_REG_COVER        = 4,
    OSD_REG_PIC          = 5,
    OSD_REG_PIC_RMEM     = 6,
} IMPOsdRgnType;
```

【元素】

元素名称	描述
OSD_REG_INV	未定义的，保留
OSD_REG_HORIZONTAL_LINE	横线类型
OSD_REG_VERTICAL_LINE	竖线类型
OSD_REG_RECT	矩形类型
OSD_REG_FOUR_CORNER_RECT	四角矩形类型
OSD_REG_BITMAP	点阵图片类型
OSD_REG_COVER	矩形遮挡类型
OSD_REG_PIC	图片类型，适合用于 logo 或时间戳
OSD_REG_PIC_RMEM	图片类型，适合用于 logo 或时间戳，内存类型为 rmem 类型

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

- OSD_REG_PIC 和 OSD_REG_PIC_RMEM 都是调用 IPU 硬件进行绘制，OSD_REG_PIC 是来一个 OSD 区域中的内容就拷贝到 rmem 中，然后调用 IPU 硬件绘制，而 OSD_REG_PIC_RMEM 是将区域一次转给 IPU 做 OSD，因此使用 OSD_REG_PIC_RMEM 性能比 OSD_REG_PIC 更好。

IMPOSDRgnAttrData

【说明】

OSD 区域属性数据。

【定义】

```
typedef union {
        void                    *bitmapData;
        lineRectData            lineRectData;
        coverData               coverData;
        picData                 picData;
 } IMPOSDRgnAttrData;
```

【元素】

元素名称	描述
bitmapData	点阵数据
lineRectData	线、框数据
coverData	遮挡数据
picData	图片数据

【注意】

无。

IMPOSDRgnAttr

【说明】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

OSD 区域属性。

【定义】

```
typedef struct {
        IMPOsdRgnType          type;
        IMPRect                rect;
        IMPLine                line;
        IMPPixelFormat         fmt;
        IMPOSDRgnAttrData      data;
 } IMPOSDRgnAttr;
```

【元素】

元素名称	描述
type	OSD 区域类型
rect	矩形数据
line	直线数据
fmt	点格式
data	OSD 区域属性数据

【注意】

无。

IMPOSDRgnTimestamp

【说明】

OSD 区域生效时间戳。

【定义】

```
typedef struct {
        uint64_t ts;
        uint64_t minus;
        uint64_t plus;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

} IMPOSDRgnTimestamp;

【元素】

元素名称	描述
ts	时间戳
minus	下限
plus	上限

【注意】

无。

IMPOSDGrpRgnAttr

【说明】

OSD 组区域属性。

【定义】

```
typedef struct {
        int             show;
        IMPPoint        offPos;
        float           scalex;
        float           scaley;
        int             gAlphaEn;
        int             fgAlhpa;
        int             bgAlhpa;
        int             layer;
} IMPOSDGrpRgnAttr;
```

【元素】

元素名称	描述
show	osd 区域显示开关，0 为关闭显示，1 为开启显示
offPos	显示起始坐标

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

scalex	缩放 x 参数
scaley	缩放 y 参数
gAlphaEn	Alpha 开关
fgAlhpa	前景 Alpha
bgAlhpa	背景 Alpha
layer	显示层

【注意】

无。

5.7 SDK Sample 介绍

Sample 程序位于/samples/libimp-samples，目录下是依赖 SDK 库的应用程序。用户可根据提供的 sample 程序，编写调试相应工程代码。

完成编译后，拷贝可执行程序到开发板上测试即可。详细介绍请参考下表：

应用文件	功能	执行命令	执行结果
sample-OSD.c	视频流上叠加 OSD	./sample-OSD	默认在板子/tmp 路径下生成 stream-0.h264，用户可将该文件放入 VLC 等视频播放软件进行播放

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

目录

6 ISP 模块 1
6.1 模块简介 1
6.2 组织框图 1
6.3 关键词说明 1
6.4 驱动简介 2
6.5 驱动编译加载 2
6.6 IMP_ISP 图像信号处理单元 3
6.7 API 参考 4
6.8 数据类型 47

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

6 ISP 模块

6.1 模块简介

ISP 通过一系列数字图像处理算法完成对数字图像的效果处理。主要包括 3A、坏点校正、去噪、强光抑制、背光补偿、色彩增强、镜头阴影校正等处理。

6.2 组织框图

6.3 关键词说明

- ISP

  ISP(Image Signal Processin)，图像信号处理

- VIC

  VIC(VIDEO INPUT Control)，视频输入控制器

- DVP

  DVP (Digital Video Port)，数字视频端口

- MIPI CSI

  CSI(CMOS Sensor Interface)，相机串行接口

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

6.4 驱动简介

T23 ISP 驱动位于: opensource/drivers/isp-T23/tx-isp-T23。

6.5 驱动编译加载

6.5.1 驱动的编译

6.5.1.1 ISP 驱动编译

1) 进入驱动文件夹 /drivers/isp-T23/tx-isp-T23；首先修改 Makefile 中 ISVP_ENV_KERNEL_DIR 宏定义，使之能够索引到正确的 kernel 路径。

2) 然后进行执行 make clean; make

3) 最后生成的 tx-isp-T23.ko 拷贝到系统中。

6.5.1.2 Sensor 驱动编译

1) sensor 驱动依赖于 kernel 和 ISP 驱动，所以在编译 sensor 驱动之前 kernel 和 ISP 驱动必须已经编译完成。

2) 进入具体的 sensor 驱动文件夹，首先修改 Makefile 中 ISVP_ENV_KERNEL_DIR 和 ISP_DRIVER_DIR 宏定义，使之能够索引到正确的 kernel 和 ISP 驱动路径。

3) 然后进行执行 make clean;make

4) 最后将生成的 sensor_xxx.ko 拷贝到系统中。

6.5.2 驱动的加载

6.5.2.1 ISP 驱动加载

ISP 驱动注册时提供 module_param 参数有 isp_clk 参数，其参考配置：

例如 3M@25fps 使用 125Mhz；

```
$ insmod tx-isp-T23.ko isp_clk=125000000
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

6.5.2.2 Sensor 驱动加载

sensor 驱动注册时提供了多个 module_param 可配置参数，如果产品硬件遵循参考设计 insmod 时无需跟参数，使用默认值即可。

- reset，power_down 的配置：加载模块时加入参数，例如：

```
$ insmod sensor_xx.ko reset_gpio=18 pwdn_gpio=20
```

其中，gpio 的值为 GPIO 编号，规则为：num = 32 * n + bit，例如：PA18 的 GPIO 编号为 18，PC20 的 GPIO 编号为 84。

- DVP 数据端口配置：加载模块时加入参数，例如：

```
$ insmod sensor_xx.ko sensor_gpio_func=1
```

其中 sensor_gpio_func 为 DVP Port 的配置选项，0：PA Low-10bit，1：PA High-10bit，2：PA 12bit。

- Sensor 的数据接口有多种，目前支持 DVP 以及 MIPI CSI-2 两种接口；但有的 senosr 可能同时支持两种数据接口，为了区分加入了模块参数 data_interface，例如：

```
$ insmod sensor_xx.ko data_interface=1
```

其中，data_inteface 为 sensor 数据接口配置选项 1：MIPI　　2：DVP

- 基于降低功耗的考量，在 sensor 驱动中加入了配置最高帧率的参数 sensor_max_fps。一般 sensor 驱动中支持 30/25fps 与 15fps 的切换。例如：

```
$ insmod sensor_xx.ko sensor_max_fps=15
```

其中，sensor_max_fps 为 sensor 帧率配置选项。15:配置为 15fps，　25：配置为 25fps，默认为 30/25fps 模式。

6.6 IMP_ISP 图像信号处理单元

6.6.1 功能介绍

图像信号处理单元。主要包含 Sensor 的注册，添加，以及图像效果设置、模式切换等。

ISP 模块与数据流无关，不需要进行 Bind，仅作用于 Sensor 控制及效果参数设置。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

6.6.2 模块操作流程

6.6.2.1 Init 流程

1) 创建 ISP 模块。
2) 添加一个 sensor，在此操作之前 sensor 驱动已经添加到内核。
3) 使能 sensor，现在 sensor 开始传输图像。
4) 使能 ISP tuning，然后才能调用 ISP 调试接口。
5) 效果调试。

6.6.2.2 Exit 流程

1) 退出 System 相关配置
2) 关闭 Sensor，关闭后 Sensor 停止传输图像，在此之前 FrameSource 上对应通道必须全部关闭。
3) 删除 Sensor，在此操作之前 Sensor 必须关闭。
4) 清理 ISP 模块，在此操作之前所有的 Sensor 都必须被删除。
5) 关闭 ISP 节点。

6.7 API 参考

表 6-1 T23 ISP API 简介

API 名称	功能
IMP_ISP_Open	打开 ISP 模块
IMP_ISP_Close	关闭 ISP 模块
IMP_ISP_AddSensor	添加一个 sensor，用于向 ISP 模块提供数据源
IMP_ISP_DelSensor	删除一个 sensor
IMP_ISP_EnableSensor	使能一个 sensor
IMP_ISP_DisableSensor	关闭一个 sensor
IMP_ISP_Tuning_SetISPBypass	ISP 模块是否 bypass
IMP_ISP_WDR_ENABLE	使能 ISP WDR
IMP_ISP_WDR_ENABLE_GET	获取 ISP WDR 模式

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_ISP_EnableTuning	使能 ISP 效果调试功能
IMP_ISP_DisableTuning	不使能 ISP 效果调试功能
IMP_ISP_SetSensorRegister	设置 sensor 一个寄存器的值
IMP_ISP_GetSensorRegister	获取 sensor 一个寄存器的值
IMP_ISP_Tuning_GetSensorAttr	获取填充参数
IMP_ISP_Tuning_SetSensorFPS	设置摄像头输出帧率
IMP_ISP_Tuning_GetSensorFPS	获取摄像头输出帧率
IMP_ISP_Tuning_SetVideoDrop	设置视频丢失功能
IMP_ISP_Tuning_SetISPRunningMode	设置 ISP 工作模式
IMP_ISP_Tuning_GetISPRunningMode	获取 ISP 工作模式
IMP_ISP_Tuning_GetAntiFlickerAttr	设置 ISP 抗闪频属性
IMP_ISP_Tuning_SetAntiFlickerAttr	获得 ISP 抗闪频属性
IMP_ISP_Tuning_SetHVFLIP	设置 HV Flip 的模式
IMP_ISP_Tuning_GetHVFLIP	获取 HV Flip 的模式
IMP_ISP_Tuning_SetBrightness	设置 ISP 综合效果图片亮度
IMP_ISP_Tuning_GetBrightness	获取 ISP 综合效果图片亮度
IMP_ISP_Tuning_SetISPRunningMode	设置 ISP 综合效果图片亮度
IMP_ISP_Tuning_GetISPRunningMode	获取 ISP 综合效果图片亮度
IMP_ISP_Tuning_SetContrast	设置 ISP 综合效果图片对比度
IMP_ISP_Tuning_GetContras	获取 ISP 综合效果图片对比度
IMP_ISP_Tuning_SetSharpness	设置 ISP 综合效果图片锐度
IMP_ISP_Tuning_GetSharpness	获取 ISP 综合效果图片锐度
IMP_ISP_Tuning_SetSaturation	设置 ISP 综合效果图片饱和度
IMP_ISP_Tuning_GetSaturation	获取 ISP 综合效果图片饱和度
IMP_ISP_Tuning_SetBcshHue	设置图像的色调
IMP_ISP_Tuning_GetBcshHue	获取图像的色调值
IMP_ISP_Tuning_SetModuleControl	设置 ISP 各个模块 bypass 功能
IMP_ISP_Tuning_GetModuleControl	获取 ISP 各个模块 bypass 功能
IMP_ISP_Tuning_SetCCMAttr	设置 CCM 属性
IMP_ISP_Tuning_GetCCMAttr	获取 CCM 属性
IMP_ISP_Tuning_SetAeWeight	设置 AE 统计区域的权重
IMP_ISP_Tuning_GetAeWeight	获取 AE 统计区域的权重
IMP_ISP_Tuning_SetAwbWeight	设置 AWB 统计区域的权重
IMP_ISP_Tuning_GetAwbWeight	获取 AWB 统计区域的权重
IMP_ISP_Tuning_SetAfWeight	设置 AF 统计区域的权重
IMP_ISP_Tuning_GetAfWeight	获取 AF 统计区域的权重
IMP_ISP_Tuning_SetAutoZoom	设置自动对焦功能的参数
IMP_ISP_Tuning_SetMask	设置填充参数

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_ISP_Tuning_GetMask	获取填充参数

IMP_ISP_Open

【功能】

打开 ISP 模块。

【语法】

int IMP_ISP_Open(void);

【形参】

无。

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 创建 ISP 模块，准备向 ISP 添加 sensor，并开启 ISP 效果调试功能。
- 必须在添加 sensor 之前被调用。

【举例】

无。

IMP_ISP_Close

【功能】

关闭 ISP 模块。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_ISP_Close(void);

【形参】

无。

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证所有 FrameSource 和效果调试功能已经关闭，所有 sensor 都已被卸载。

【举例】

无。

IMP_ISP_AddSensor

【功能】

添加一个 sensor，用于向 ISP 模块提供数据源。

【语法】

int IMP_ISP_AddSensor(IMPSensorInfo *pinfo);

【形参】

参数名称	描述	输入/输出
pinfo	需要添加 sensor 的信息指针	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证摄像头驱动已经注册进内核。

【举例】

无。

IMP_ISP_DelSensor

【功能】

关闭 ISP 模块。

【语法】

int IMP_ISP_DelSensor(IMPSensorInfo *pinfo);

【形参】

参数名称	描述	输入/输出
pinfo	需要添加 sensor 的信息指针	输入

【返回值】

返回值	描述
0	成功
非 0	失败

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证摄像头已经停止工作，即调用了 IMP_ISP_DisableSensor 函数。

【举例】

无。

IMP_ISP_EnableSensor

【功能】

使能一个 sensor。

【语法】

int IMP_ISP_EnableSensor(void);

【形参】

无。

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证摄像头已经被添加进 ISP 模块。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

无。

IMP_ISP_DisableSensor

【功能】

不使能一个 sensor。

【语法】

int IMP_ISP_DisableSensor(void);

【形参】

无。

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证所有 FrameSource 都已停止输出图像，同时效果调试也在不使能态。

【举例】

无。

IMP_ISP_Tuning_SetISPBypass

【功能】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

ISP 模块是否 bypass。

【语法】

int IMP_ISP_Tuning_SetISPBypass(IMPISPTuningOpsMode enable);

【形参】

无。

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 模块是开启的。

【举例】

无。

IMP_ISP_Tuning_WDR_ENABLE

【功能】

使能 ISP WDR。

【语法】

int IMP_ISP_WDR_ENABLE(IMPISPTuningOpsMode mode);

【形参】

参数名称	描述	输入/输出
mode	ISP WDR 模式	输入

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_WDR_ENABLE_GET

【功能】

获取 ISP WDR 模式。

【语法】

int IMP_ISP_WDR_ENABLE_Get(IMPISPTuningOpsMode* mode);

【形参】

参数名称	描述	输入/输出
mode	ISP WDR 模式	输入

【返回值】

返回值	描述
0	成功
非 0	失败

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_EnableTuning

【功能】

使能 ISP 效果调试功能。

【语法】

Int IMP_ISP_EnableTuning(void);

【形参】

无。

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

无。

IMP_ISP_DisableTuning

【功能】

不使能 ISP 效果调试功能。

【语法】

int32_t IMP_ISP_DisableTuning(void);

【形参】

无。

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证在不使能 sensor 之前，先调用此函数。

【举例】

无。

IMP_ISP_SetSensorRegister

【功能】

设置 sensor 一个寄存器的值。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_ISP_SetSensorRegister(uint32_t reg, uint32_t value);

【形参】

参数名称	描述	输入/输出
reg	寄存器地址	输入
value	寄存器值	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证摄像头已经被使能。

【举例】

无。

IMP_ISP_GetSensorRegister

【功能】

获取 sensor 一个寄存器的值。

【语法】

int IMP_ISP_GetSensorRegister(uint32_t reg, uint32_t *value);

【形参】

参数名称	描述	输入/输出

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

reg	寄存器地址	输入
value	寄存器值	输出

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证摄像头已经被使能。

【举例】

无。

IMP_ISP_Tuning_GetSensorAttr

【功能】

获取填充参数。

【语法】

int IMP_ISP_Tuning_GetSensorAttr(IMPISPSENSORAttr *attr);

【形参】

参数名称	描述	输入/输出
attr	sensor 属性参数	输出

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_SetSensorFPS

【功能】

设置摄像头输出帧率。

【语法】

int IMP_ISP_Tuning_SetSensorFPS(uint32_t fps_num, uint32_t fps_den);

【形参】

参数名称	描述	输入/输出
fps_num	设定帧率的分子参数	输入
fps_den	设定帧率的分母参数	输入

【返回值】

返回值	描述
0	成功
非 0	失败

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 IMP_ISP_EnableSensor 和 IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_GetSensorFPS

【功能】

获取摄像头输出帧率。

【语法】

int IMP_ISP_Tuning_GetSensorAttr(IMPISPSENSORAttr *attr);

【形参】

参数名称	描述	输入/输出
fps_num	设定帧率的分子参数	输入
fps_den	设定帧率的分母参数	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 IMP_ISP_EnableSensor 和 IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_SetVideoDrop

【功能】

设置视频丢失功能。当出现 sensor 与主板的连接线路出现问题时，设置的回调函数会被执行。

【语法】

int IMP_ISP_Tuning_SetVideoDrop(void (*cb)(void));

【形参】

参数名称	描述	输入/输出
cb	回调函数	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_SetISPRunningMode

【功能】

设置 ISP 工作模式，正常模式或夜视模式或者定制；默认为正常模式。

【语法】

int IMP_ISP_Tuning_SetISPRunningMode(IMPISPRunningMode mode);

【形参】

参数名称	描述	输入/输出
mode	运行模式参数	输入

【返回值】

返回值	描述
0	成功
非 0	失败

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

示例代码：

```
IMPISPRunningMode mode;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
2.   if( it is during a night now){
3.       mode = IMPISP_RUNNING_MODE_NIGHT
4.   }else{
5.       mode = IMPISP_RUNNING_MODE_DAY;
6.   }
7.   ret = IMP_ISP_Tuning_SetISPRunningMode(IMPVI_MAIN, &mode);
8.   if(ret){
9.       IMP_LOG_ERR(TAG, "IMP_ISP_Tuning_SetISPRunningMode error !\n");
10.      return -1;
11.  }
```

IMP_ISP_Tuning_GetISPRunningMode

【功能】

获取 ISP 工作模式。

【语法】

int IMP_ISP_Tuning_GetISPRunningMode(IMPISPRunningMode *pmode);

【形参】

参数名称	描述	输入/输出
pmode	操作参数指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

无。

IMP_ISP_Tuning_SetAntiFlickerAttr

【功能】

设置 ISP 抗闪频属性。

【语法】

int IMP_ISP_Tuning_SetAntiFlickerAttr(IMPISPAntiflickerAttr attr);

【形参】

参数名称	描述	输入/输出
pattr	设置参数值	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。

【举例】

无。

IMP_ISP_Tuning_GetAntiFlickerAttr

【功能】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

获得 ISP 抗闪频属性。

【语法】

int IMP_ISP_Tuning_GetAntiFlickerAttr(IMPISPAntiflickerAttr *pattr);

【形参】

参数名称	描述	输入/输出
pattr	获取参数值指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。

【举例】

无。

IMP_ISP_Tuning_SetHVFLIP

【功能】

设置 HV Flip 的模式。

【语法】

int IMP_ISP_Tuning_SetHVFLIP(IMPISPHVFLIP hvflip);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
hvflip	HV Flip 模式	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_GetHVFLIP

【功能】

获取 HV Flip 的模式。

【语法】

int IMP_ISP_Tuning_GetHVFlip(IMPISPHVFLIP *hvflip);

【形参】

参数名称	描述	输入/输出
hvflip	HV Flip 模式	输出

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_SetBrightness

【功能】

设置 ISP 综合效果图片亮度。

【语法】

int IMP_ISP_Tuning_SetBrightness(unsigned char bright);

【形参】

参数名称	描述	输入/输出
bright	图片亮度参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。
- 默认值为 128，大于 128 增加亮度，小于 128 降低亮度。

【举例】

无。

IMP_ISP_Tuning_GetBrightness

【功能】

获取 ISP 综合效果图片亮度。

【语法】

int32_t IMP_ISP_Tuning_GetBrightness(IMPVI_NUM num, unsigned char *bright);

【形参】

参数名称	描述	输入/输出
bright	图片亮度参数指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。
- 默认值为 128，大于 128 增加亮度，小于 128 降低亮度。

【举例】

无。

IMP_ISP_Tuning_SetContrast

【功能】

设置 ISP 综合效果图片对比度。

【语法】

int IMP_ISP_Tuning_SetContrast(unsigned char contrast);

【形参】

参数名称	描述	输入/输出
contrast	图片对比度参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。
- 默认值为 128，大于 128 增加对比度，小于 128 降低对比度。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【举例】

无。

IMP_ISP_Tuning_GetContrast

【功能】

获取 ISP 综合效果图片对比度。

【语法】

int IMP_ISP_Tuning_GetContrast(unsigned char *pcontrast);

【形参】

参数名称	描述	输入/输出
contrast	图片对比度参数指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。
- 默认值为 128，大于 128 增加对比度，小于 128 降低对比度。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_ISP_Tuning_SetSharpness

【功能】

设置 ISP 综合效果图片锐度。

【语法】

int IMP_ISP_Tuning_SetSharpness(unsigned char sharpness);

【形参】

参数名称	描述	输入/输出
sharpness	图片锐度参数值	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。
- 默认值为 128，大于 128 增加锐度，小于 128 降低锐度。

【举例】

无。

IMP_ISP_Tuning_GetSharpness

【功能】

获取 ISP 综合效果图片锐度。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【语法】

int IMP_ISP_Tuning_GetSharpness(unsigned char *psharpness);

【形参】

参数名称	描述	输入/输出
sharpness	图片锐度参数值指针	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。
- 默认值为 128，大于 128 增加锐度，小于 128 降低锐度。

【举例】

无。

IMP_ISP_Tuning_SetSaturation

【功能】

设置 ISP 综合效果图片饱和度。

【语法】

int IMP_ISP_Tuning_SetSaturation(unsigned char sat);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
sat	图片饱和度参数值	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。
- 默认值为 128，大于 128 增加饱和度，小于 128 降低饱和度。

【举例】

无。

IMP_ISP_Tuning_GetSaturation

【功能】

获取 ISP 综合效果图片饱和度。

【语法】

int IMP_ISP_Tuning_GetSaturation(unsigned char *psat);

【形参】

参数名称	描述	输入/输出
sat	图片饱和度参数值指针	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 ISP 效果调试功能已使能。
- 默认值为 128，大于 128 增加饱和度，小于 128 降低饱和度。

【举例】

无。

IMP_ISP_Tuning_SetBcshHue

【功能】

设置图像的色调。

【语法】

int IMP_ISP_Tuning_SetBcshHue(unsigned char hue);

【形参】

参数名称	描述	输入/输出
hue	图像的色调参考值	输入

【返回值】

返回值	描述
0	成功。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。
- 默认值为 128，大于 128 正向调节色调，小于 128 反向调节色调，调节范围 0~255。

【举例】

无。

IMP_ISP_Tuning_GetBcshHue

【功能】

获取图像的色调值。

【语法】

int IMP_ISP_Tuning_GetBcshHue(unsigned char *hue);

【形参】

参数名称	描述	输入/输出
hue	图像的色调参考值指针	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。
- 默认值为 128，大于 128 正向调节色调，小于 128 反向调节色调，调节范围 0~255。

【举例】

无。

IMP_ISP_Tuning_SetModuleControl

【功能】

设置 ISP 各个模块 bypass 功能。

【语法】

int IMP_ISP_Tuning_SetModuleControl(IMPISPModuleCtl *ispmodule);

【形参】

参数名称	描述	输入/输出
ispmodule	ISP 各个模块 bypass 功能	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_GetModuleControl

【功能】

获取 ISP 各个模块 bypass 功能。

【语法】

int IMP_ISP_Tuning_GetModuleControl(IMPISPModuleCtl *ispmodule);

【形参】

参数名称	描述	输入/输出
ispmodule	ISP 各个模块 bypass 功能	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_ISP_Tuning_SetCCMAttr

【功能】

设置 CCM 属性。

【语法】

int IMP_ISP_Tuning_SetCCMAttr(IMPISPCCMAttr *ccm);

【形参】

参数名称	描述	输入/输出
csc	CSC 属性参数	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_GetCCMAttr

【功能】

获取 CCM 属性。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_ISP_Tuning_GetCCMAttr(IMPISPCCMAttr *ccm);

【形参】

参数名称	描述	输入/输出
csc	CSC 属性参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_SetAeWeight

【功能】

设置 AE 统计区域的权重。

【语法】

int IMP_ISP_Tuning_SetAeWeight(IMPISPWeight *ae_weight);

【形参】

参数名称	描述	输入/输出
ae_weight	权重信息	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_GetAeWeight

【功能】

获取 AE 统计区域的权重。

【语法】

int IMP_ISP_Tuning_GetAeWeight(IMPISPWeight *ae_weight);

【形参】

参数名称	描述	输入/输出
ae_weight	权重信息	输出

【返回值】

返回值	描述
0	成功。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_SetAwbWeight

【功能】

设置 AWB 统计区域的权重。

【语法】

int IMP_ISP_Tuning_SetAwbWeight(IMPISPWeight *awb_weight);

【形参】

参数名称	描述	输入/输出
awb_weight	各区域权重信息	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_GetAwbWeight

【功能】

获取 AWB 统计区域的权重。

【语法】

int IMP_ISP_Tuning_GetAwbWeight(IMPISPWeight *awb_weight);

【形参】

参数名称	描述	输入/输出
awb_weight	各区域权重信息	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

无。

IMP_ISP_Tuning_SetAfWeight

【功能】

设置 AF 统计区域的权重。

【语法】

int IMP_ISP_Tuning_SetAfWeight(IMPISPWeight *af_weigh);

【形参】

参数名称	描述	输入/输出
af_weight	各区域权重信息	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_GetAfWeight

【功能】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

获取 AF 统计区域的权重。

【语法】

int IMP_ISP_Tuning_GetAfWeight(IMPISPWeight *af_weight);

【形参】

参数名称	描述	输入/输出
af_weight	各区域权重信息	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_SetAutoZoom

【功能】

设置自动对焦功能的属性。

【语法】

int IMP_ISP_Tuning_SetAutoZoom(IMPISPAutoZoom *ispautozoom);

【形参】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

参数名称	描述	输入/输出
ispautozoom	自动对焦功能的属性	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，必须保证 IMP_ISP_EnableSensor 被执行且返回成功。

【举例】

无。

IMP_ISP_Tuning_SetMask

【功能】

设置填充参数。

【语法】

int IMP_ISP_Tuning_SetMask(IMPISPMASKAttr *mask);

【形参】

参数名称	描述	输入/输出
mask	填充参数	输入

【返回值】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

返回值	描述
0	成功。
非 0	失败。

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

IMP_ISP_Tuning_GetMask

【功能】

获取填充参数。

【语法】

int IMP_ISP_Tuning_GetMask(IMPISPMASKAttr *mask);

【形参】

参数名称	描述	输入/输出
mask	填充参数	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_isp.h

库文件：libimp.a / libimp.so

【注意】

- 在使用这个函数之前，IMP_ISP_EnableTuning 已被调用。

【举例】

无。

6.8 数据类型

IMP_ISP 相关结构类型如下：

表 6-2 T23 ISP 数据结构表

API 名称	功能
IMPSensorControlBusType	摄像头控制总线类型枚举
IMPI2Cinfo	摄像头 I2C 属性结构体
IMPSPIInfo	摄像头 SPI 属性结构体
IMPSensorInfo	摄像头注册信息结构体
IMPISPTuningOpsType	ISP 功能模式
IMPISPAntiflickerAttr	ISP 抗闪频属性参数结构体
IMPISPRunningMode	ISP 工作模式配置，正常模式或夜视模式
IMPISPAWBCluster	Cluster 模式白平衡参数
IMPISPTuningMode	模式选择选项
IMPISPAEMin	AE Min
IMPISPHVFLIP	HV Flip 模式
IMPISPMASKAttr	填充参数
IMPISPSENSORAttr	Sensor 属性参数
IMPISPCCMAttr	ISP 颜色矩阵属性
IMPISPAEAttr	ISP AE 手动模式属性
IMPISPAEState	AE 收敛相关参数
IMPISPScalerLv	缩放效果参数
IMPISPAeInitAttr	客户自定义自动曝光库的 AE 属性
IMPISPAwbAttr	客户自定义自动白平衡库的 AWB 属性
IMPISPBlcAtt	黑电平校正功能属性

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPISPWdrOutputMode	WDR 输出模式
IMPISPFrameDrop	丢帧参数

IMPSensorControlBusType

【说明】

摄像头控制总线类型枚举。

【定义】

```
typedef enum {
    TX_SENSOR_CONTROL_INTERFACE_I2C = 1,
    TX_SENSOR_CONTROL_INTERFACE_I2C,
} IMPSensorControlBusType;
```

【元素】

元素名称	描述
TX_SENSOR_CONTROL_INTERFACE_I2C	I2C 控制总线
TX_SENSOR_CONTROL_INTERFACE_I2C	SPI 控制总线

【注意】

无

IMPI2CInfo

【说明】

摄像头控制总线类型是 I2C 时，需要配置的参数结构体。

【定义】

```
typedef enum {
  IMPISP_TUNING_OPS_TYPE_AUTO,
  IMPISP_TUNING_OPS_TYPE_MANUAL,
  IMPISP_TUNING_OPS_TYPE_BUTT,
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
} IMPISPTuningOpsType;
typedef struct {
        char type[20];
        int addr;
        int i2c_adapter_id;
    } IMPI2CInfo;
```

【元素】

元素名称	描述
type[20]	I2C 设备名字，必须与摄像头驱动中 struct i2c_device_id 中 name 变量一致
addr	I2C 地址
i2c_adapter_id	I2C 控制器

【注意】

- I2C 设备名字，必须与摄像头驱动中 struct i2c_device_id 中 name 变量一致。

IMPSPIInfo

【说明】

摄像头控制总线类型是 SPI 时，需要配置的参数结构体。

【定义】

```
typedef struct {
    char modalias[32];
            int bus_num;
  } IMPSPIInfo;
```

【元素】

元素名称	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

modalias[32]	SPI 设备名字，必须与摄像头驱动中 struct spi_device_id 中 name 变量一致
bus_num	SPI 总线地址

【注意】

- SPI 设备名字，必须与摄像头驱动中 struct spi_device_id 中 name 变量一致 。

IMPSensorInfo

【说明】

摄像头注册信息结构体。

【定义】

```
typedef struct {
        char name[32];
        IMPSensorControlBusType cbus_type;
         union {
              IMPI2CInfo i2c;
              IMPSPIInfo spi;
        };
         unsigned short rst_gpio;
        unsigned short pwdn_gpio;
        unsigned short power_gpio;
   } IMPSensorInfo;
```

【元素】

元素名称	描述
name[32]	摄像头名字
cbus_type	摄像头控制总线类型
i2c	I2C 总线信息

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

spi	SPI 总线信息
rst_gpio	摄像头 reset 接口链接的 GPIO，注意：现在没有启用该参数
pwdn_gpio	摄像头 power down 接口链接的 GPIO，注意：现在没有启用该参数
power_gpio	摄像头 power 接口链接的 GPIO，注意：现在没有启用该参数
sensor_id	摄像头 ID 号
video_interface	摄像头接口类型
mclk	摄像头时钟源
default_boot	摄像头初始化配置选择

【注意】

- 在 sensor 驱动中会用过 default_boot 这个值选择使用那一个配置表。默认使用 0；如果 sensor 支持 dvp 接口又支持 mipi 接口，则默认 default_boot=0 是 dvp 接口，default_boot=1 是 mipi 接口。

IMPISPTuningOpsMode

【说明】

ISP 功能开关。

【定义】

```
typedef enum {
    IMPISP_TUNING_OPS_MODE_DISABLE,
    IMPISP_TUNING_OPS_MODE_ENABLE,
    IMPISP_TUNING_OPS_MODE_BUTT,
} IMPISPTuningOpsMode
```

【元素】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

元素名称	描述
IMPISP_TUNING_OPS_MODE_DISABLE	不使能该模块功能
IMPISP_TUNING_OPS_MODE_ENABLE	使能该模块功能
IMPISP_TUNING_OPS_MODE_BUTT	用于判断参数的有效性，参数大小必须小于这个值

【注意】

无。

IMPISPTuningOpsType

【说明】

ISP 功能选用开关。

【定义】

```
typedef enum {
    IMPISP_TUNING_OPS_TYPE_AUTO,
    IMPISP_TUNING_OPS_TYPE_MANUAL,
  IMPISP_TUNING_OPS_TYPE_BUTT,
  } IMPISPTuningOpsType;
```

【元素】

元素名称	描述
IMPISP_TUNING_OPS_MODE_DISABLE	该模块的操作为自动模式
IMPISP_TUNING_OPS_MODE_ENABLE	该模块的操作为手动模式
IMPISP_TUNING_OPS_MODE_	用于判断参数的有效性，参数大小必须小

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

BUTT	于这个值

【注意】

无。

IMPISPAntiflickerAttr

【说明】

ISP 抗闪频属性参数结构体。

【定义】

```
typedef enum {
    IMPISP_ANTIFLICKER_DISABLE,
    IMPISP_ANTIFLICKER_50HZ,
    IMPISP_ANTIFLICKER_60HZ,
    IMPISP_ANTIFLICKER_BUTT,
} IMPISPAntiflickerAttr;
```

【元素】

元素名称	描述
IMPISP_ANTIFLICKER_DISABLE	不使能 ISP 抗闪频功能
IMPISP_ANTIFLICKER_50HZ	使能 ISP 抗闪频功能，并设置频率为 50HZ
IMPISP_ANTIFLICKER_50HZ	使能 ISP 抗闪频功能，并设置频率为 60HZ
IMPISP_ANTIFLICKER_BUTT	用于判断参数的有效性，参数大小必须小于这个值

【注意】

无。

IMPISPRunningMode

【说明】

ISP 工作模式配置，正常模式或夜视模式。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【定义】

```
typedef enum {
    IMPISP_RUNNING_MODE_DAY = 0,
    IMPISP_RUNNING_MODE_NIGHT = 1,
    IMPISP_RUNNING_MODE_BUTT,
} IMPISPRunningMode;
```

【元素】

元素名称	描述
IMPISP_RUNNING_MODE_DAY	正常模式
IMPISP_RUNNING_MODE_NIGHT	夜视模式
IMPISP_RUNNING_MODE_BUTT	最大值

【注意】

无。

IMPISPAWBCluster

【说明】

Cluster 模式白平衡参数。

【定义】

```
typedef struct {
    IMPISPTuningOpsMode ClusterEn;
    IMPISPTuningOpsMode ToleranceEn;
    unsigned int tolerance_th;
    unsigned int awb_cluster[7];
}IMPISPAWBCluster;
```

【元素】

元素名称	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

ClusterEn	Cluster 模式白平衡使能
ToleranceEn	AWB 收敛容忍使能
tolerance_th	AWB 收敛容忍阈值，取值范围为 0~64
awb_cluster[7]	Cluster 模式白平衡参数

【注意】

无。

IMPISPTuningMode

【说明】

模式选择选项。

【定义】

```
typedef enum {
    IMPISP_TUNING_MODE_AUTO,
    IMPISP_TUNING_MODE_MANUAL,
    IMPISP_TUNING_MODE_RANGE,
    IMPISP_TUNING_MODE_BUTT,
} IMPISPTuningMode;
```

【元素】

元素名称	描述
IMPISP_TUNING_MODE_AUTO	该模块的操作为自动模式
IMPISP_TUNING_MODE_MANUAL	该模块的操作为手动模式
IMPISP_TUNING_MODE_RANGE	该模块的操作为设置范围模式

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPISP_TUNING_MODE_BUTT	用于判断参数的有效性，参数大小必须小于这个值

【注意】

无。

IMPISPAEMin

【说明】

AE Min。

【定义】

```
typedef struct {
    unsigned int min_it;
    unsigned int min_again;
    unsigned int min_it_short;
    unsigned int min_again_short;
} IMPISPAEMin;
```

【元素】

元素名称	描述
min_it	AE 最小曝光
min_again	AE 最小模拟增益
min_it_short	AE 短帧的最小曝光
min_again_short	AE 短帧的最小模拟增益

【注意】

无。

IMPISPHVFLIP

【说明】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

HV Flip 模式。

【定义】

```
typedef enum {
    IMPISP_FLIP_NORMAL_MODE = 0,
    IMPISP_FLIP_H_MODE = 1,
    IMPISP_FLIP_V_MODE = 2,
    IMPISP_FLIP_HV_MODE = 3,
    IMPISP_FLIP_MODE_BUTT,
} IMPISPHVFLIP;
```

【元素】

元素名称	描述
IMPISP_FLIP_NORMAL_MODE	正常模式
IMPISP_FLIP_H_MODE	镜像模式
IMPISP_FLIP_V_MODE	翻转模式
IMPISP_FLIP_HV_MODE	镜像并翻转模式

【注意】

无。

IMPISPMASKAttr

【说明】

填充参数。

【定义】

```
typedef struct {
    IMPISP_MASK_BLOCK_PAR chn0[4];
    IMPISP_MASK_BLOCK_PAR chn1[4];
    IMPISP_MASK_BLOCK_PAR chn2[4];
    IMPISP_MASK_TYPE mask_type;
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
} IMPISPMASKAttr;
```

【元素】

元素名称	描述
IMPISP_MASK_BLOCK_PAR chn0	通道 0 填充参数
IMPISP_MASK_BLOCK_PAR chn1	通道 1 填充参数
IMPISP_MASK_BLOCK_PAR chn2	通道 3 填充参数
IMPISP_MASK_TYPE mask_type	填充数据类型

【注意】

无。

IMPISPSENSORAttr

【说明】

Sensor 属性参数。

【定义】

```
typedef struct {
    unsigned int hts;
    unsigned int vts;
    unsigned int fps;
    unsigned int width;
    unsigned int height;
} IMPISPSENSORAttr;
```

【元素】

元素名称	描述
hts	sensor hts
vts	sensor vts

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

fps	sensor 帧率
width	sensor 输出宽度
height	sensor 输出的高度

【注意】

无。

IMPISPCCMAttr

【说明】

ISP 颜色矩阵属性。

【定义】

```
typedef struct {
    IMPISPTuningOpsMode ManualEn;
    IMPISPTuningOpsMode SatEn;
    float ColorMatrix[9];
} IMPISPCCMAttr;
```

【元素】

元素名称	描述
IMPISPTuningOpsMode ManualEn	手动 CCM 使能
IMPISPTuningOpsMode SatEn	手动模式下饱和度使能
ColorMatrix[9]	颜色矩阵

【注意】

无。

IMPISPAEAttr

【说明】

ISP AE 手动模式属性。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【定义】

```
typedef struct {
    /* 线性模式和 WDR 模式下长帧的 AE 手动模式属性*/
    IMPISPTuningOpsMode AeFreezenEn;
    IMPISPTuningOpsMode AeItManualEn;
    unsigned int AeIt;
    IMPISPTuningOpsMode AeAGainManualEn;
    unsigned int AeAGain;
    IMPISPTuningOpsMode AeDGainManualEn;
    unsigned int AeDGain;
    IMPISPTuningOpsMode AeIspDGainManualEn;
    unsigned int AeIspDGain;
    /* WDR 模式下短帧的 AE 手动模式属性*/
    IMPISPTuningOpsMode AeWdrShortFreezenEn;
    IMPISPTuningOpsMode AeWdrShortItManualEn;
    unsigned int AeWdrShortIt;
IMPISPTuningOpsMode AeWdrShortAGainManualEn;
    unsigned int AeWdrShortAGain;
    IMPISPTuningOpsMode AeWdrShortDGainManualEn;
    unsigned int AeWdrShortDGain;
    IMPISPTuningOpsMode AeWdrShortIspDGainManualEn;
    unsigned int AeWdrShortIspDGain;
} IMPISPAEAttr;
```

【元素】

元素名称	描述
AeFreezenEn	AE Freezen 使能

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

AeItManualEn	AE 曝光手动模式使能
AeIt	AE 手动模式下的曝光值，单位是曝光行
AeAGainManualEn	AE Sensor 模拟增益手动模式使能
AeAGain	AE Sensor 模拟增益值，单位是倍数 x 1024
AeDGainManualEn	AE Sensor 数字增益手动模式使能
AeDGain	AE Sensor 数字增益值，单位是倍数 x 1024
AeIspDGainManualEn	AE ISP 数字增益手动模式使能
AeIspDGain	AE ISP 数字增益值，单位倍数 x 1024
AeWdrShortFreezenEn	AE Freezen 使能
AeWdrShortItManualEn	AE 曝光手动模式使能
AeWdrShortIt	AE 手动模式下的曝光值，单位是曝光行
AeWdrShortAGainManualEn	AE Sensor 模拟增益手动模式使能
AeWdrShortAGain	AE Sensor 模拟增益值，单位是倍数 x 1024
AeWdrShortDGainManualEn	AESensor 数字增益手动模式使能
AeWdrShortDGain	AESensor 数字增益值，单位是倍数 x 1024
AeWdrShortIspDGainManual En	AE ISP 数字增益手动模式使能
AeWdrShortIspDGain	AE ISP 数字增益值，单位倍数 x 1024

【注意】

无。

IMPISPAEState

【说明】

AE 收敛相关参数。

【定义】

typedef struct {

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
    bool stable;
    unsigned int target;
    unsigned int ae_mean;
}IMPISPAEState;
```

【元素】

元素名称	描述
bool stable	AE 收敛状态，1:代表稳定 0:正在收敛
target	当前的目标亮度
ae_mean	叠加权重之后，AE 的当前的统计平均值

【注意】

无。

IMPISPScalerLv

【说明】

缩放效果参数。

【定义】

```
typedef struct {
    unsigned char channel;
    IMPISPScalerMethod method;
    unsigned char level;
} IMPISPScalerLv;
```

【元素】

元素名称	描述
channel	通道 0~2
method	缩放方法

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

level	缩放清晰度等级 范围 0~128

【注意】

无。

IMPISPAeInitAttr

【说明】

客户自定义自动曝光库的 AE 属性。

【定义】

```
typedef struct {
    uint32_t change;
    enum isp_core_expr_unit AeIntegrationTimeUnit;
    uint32_t AeIntegrationTime;
    uint32_t AeAGain;
    uint32_t    AeDGain;
    uint32_t    AeIspDGain;
    uint32_t    AeShortIntegrationTime;
    uint32_t    AeShortAGain;
    uint32_t    AeShortDGain;
    uint32_t    AeShortIspDGain;
    uint32_t    luma;
    uint32_t    luma_scence;
} IMPISPAeAttr;
```

【元素】

元素名称	描述
change	是否更新 AE 参数

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

AeIntegrationTimeUnit	AE 曝光时间单位
AeIntegrationTime	AE 的曝光值
AeAGain	AE Sensor 模拟增益值，单位是倍数 x 1024
AeDGain	AESensor 数字增益值，单位是倍数 x 1024
AeIspDGain	AE ISP 数字增益值，单位倍数 x 1024
AeShortIntegrationTime	AE 手动模式下的曝光值
AeShortAGain	AE Sensor 模拟增益值，单位是倍数 x 1024
AeShortDGain	AE Sensor 数字增益值，单位是倍数 x 1024
AeShortIspDGain	AE ISP 数字增益值，单位倍数 x 1024
luma	AE Luma 值
luma_scence	AE 场景 Luma 值

【注意】

无。

IMPISPAwbAttr

【说明】

客户自定义自动白平衡库的 AWB 属性。

【定义】

```
typedef struct {
    uint32_t change;
    uint32_t r_gain;
    uint32_t b_gain;
    uint32_t ct;
} IMPISPAwbAttr;
```

【元素】

元素名称	描述

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

change	是否更新 AWB 参数
r_gain	AWB 参数 r_gain
b_gain	AWB 参数 b_gain
ct	当前色温

【注意】

无。

IMPISPBlcAttr

【说明】

黑电平校正功能属性。

【定义】

```
typedef struct {
    unsigned int black_level_r;
    unsigned int black_level_gr;
    unsigned int black_level_gb;
    unsigned int black_level_b;
    unsigned int black_level_ir;
} IMPISPBlcAttr;
```

【元素】

元素名称	描述
black_level_r	R 通道
black_level_gr	GR 通道
black_level_gr	GB 通道
black_level_b	B 通道
black_level_ir	IR 通道

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPISPWdrOutputMode

【说明】

WDR 输出模式

【定义】

```
typedef enum {
        IMPISP_WDR_OUTPUT_MODE_FUS_FRAME,
        IMPISP_WDR_OUTPUT_MODE_LONG_FRAME,
        IMPISP_WDR_OUTPUT_MODE_SHORT_FRAME,
IMPISP_WDR_OUTPUT_MODE_BUTT,
} IMPISPWdrOutputMode;
```

【元素】

元素名称	描述
IMPISP_WDR_OUTPUT_MODE_FUS_FRAME	混合模式
IMPISP_WDR_OUTPUT_MODE_LONG_FRAME	长帧模式
IMPISP_WDR_OUTPUT_MODE_SHORT_FRAME	短帧模式
IMPISP_WDR_OUTPUT_MODE_BUTT	用于判断参数的有效性，参数大小必须小于这个值

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMPISPFrameDrop

【说明】

丢帧参数。

【定义】

```
typedef struct {
        IMPISPTuningOpsMode enable;
        uint8_t lsize;
        uint32_t fmark;
} IMPISPFrameDrop;
```

【元素】

元素名称	描述
IMPISPTuningOpsMode enable	使能标志
lsize	总数量(范围:0~31)
fmark	位标志(1 输出，0 丢失)

【注意】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

目录

7 IVS 智能分析 1
7.1 模块简介 1
7.2 使用方法 2
7.3 API 参考 3

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

7 IVS 智能分析

7.1 模块简介

IMP IVS 通过 IVS 通用接口 API 调用实例化的 IMPIVSInterface 以将智能分析算法嵌入到 SDK 中来分析 SDK 中的 frame 图像。

IMPIVSInterface 为通用算法接口，具体算法通过实现此接口并将其传给 IMP IVS 达到在 SDK 中运行具体算法的目的。

一个 channel 有且仅为单个算法实例的载体，必须将具体实现的通用算法接口传给具体的 channel 才能在 SDK 中运行算法。

IMPIVSInterface 成员 param 为成员函数 init 的参数。

IMP_IVS 会在传给成员函数 ProcessAsync 参数的 frame 时对其进行外部加锁，ProcessAsync 必须在使用完 frame 后调用 IMP_IVS_ReleaseData 释放 frame，以免死锁。

除上述通过绑定在数据流中的 IMP IVS 实现算法调用之外，还提供一种非绑定的方式实现算法调用，即获取 framesouce channel 的 frame 图像，直接调用 IMPIVSInterface 的成员函数实现算法处理。

7.2 使用方法

7.2.1 绑定模式的算法使用方法

以移动侦测算法为例，函数的具体实现见 sample-move_c.c 文件，步骤如下：

第一步：初始化系统，可以直接调用范例中的 sample_system_init()函数。整个

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

应用程序只能初始化系统一次，若之前已经初始化，这里不需要再初始化。

第二步：初始化 framesource 若算法所使用的 framesource 通道已创建，直接使用已经创建好的通道即可。若算法所使用的 framesource 通道未创建，可以调用范列中的 sample_framesource_init(IVS_FS_CHN, &fs_chn_attr)进行创建。

第三步：创建 ivs 具体算法通道组。多个算法可以共用一个通道组，也可以分别使用通道组 sample_ivs_move_init()。

第四步：绑定算法通道组和 framesource 通道组。

第五步：启动 framesource 和算法。建议算法通道号和算法编号一致，以便可以直接对应当前操作哪一个算法。

第六步：获取算法结果 Polling 结果、获取结果和释放结果必须严格对应，不能中间有中断；只有 Polling 结果正确返回，获取到的结果才会被更新，否则获取到的结果无法预知。

第七步：释放资源。

7.2.2 非绑定模式的算法使用方法

以移动侦测算法为例，函数的具体实现见 sample-Framesource-algo.c 文件，步骤如下：

第一步：初始化系统，可以直接调用范例中的 sample_system_init()函数。

第二步：初始化 framesource 若算法所使用的 framesource 通道已创建，直接使用已经创建好的通道即可。若算法所使用的 framesource 通道未创建，可以调用范列中的 sample_framesource_init(IVS_FS_CHN, &fs_chn_attr)进行创建。

第三步：初始化 interface 句柄，调用 interface->init()初始化算法，调用 IMP_FrameSource_SetFrameDepth()设置帧缓存。

第四步：循环调用 IMP_FrameSource_GetFrame 获取 frame，interface->preProcessSync 算法预处理，interface->processAsync 算法处理过程，interface->getResult 获取算法结果，interface->releaseResult 释放算法结果，IMP_FrameSource_ReleaseFrame 释放 frame。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

第五步：IMP_FrameSource_SetFrameDepth 缓存深度回 0，释放缓存，反初始化 interface 句柄，反初始化 framesouce 和系统。

7.3 API 参考

表 7-1 T23 IVS API 功能表

API 名称	功能
IMP_IVS_CreateGroup	创建通道组
IMP_IVS_DestroyGroup	销毁通道组
IMP_IVS_CreateChn	创建 IVS 功能对应的通道
IMP_IVS_DestroyChn	销毁 IVS 功能句柄对应的通道
IMP_IVS_RegisterChn	注册通道到通道组
IMP_IVS_UnRegisterChn	从通道组注消通道
IMP_IVS_StartRecvPic	通道开始接收图像
IMP_IVS_StopRecvPic	通道停止接收图像
IMP_IVS_PollingResult	阻塞判断是否可以获得 IVS 功能已计算出的智能分析结果
IMP_IVS_GetResult	获得 IVS 功能计算出的智能分析结果
IMP_IVS_ReleaseResult	释放 IVS 功能计算出的结果资源
IMP_IVS_ReleaseData	释放传给 Datacallback 的参数 frame
IMP_IVS_GetParam	获取通道算法参数
IMP_IVS_SetParam	设置通道算法参数

IMP_IVS_CreateGroup

【功能】

创建通道组。

【语法】

int IMP_IVS_CreateGroup(int GrpNum);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_IVS_DestroyGroup

【功能】

销毁通道组。

【语法】

int IMP_IVS_DestroyGroup(int GrpNum);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_IVS_CreateChn

【功能】

创建 IVS 功能对应的通道。

【语法】

int IMP_IVS_CreateChn(int ChnNum, IMPIVSInterface *handler);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入
handler	IVS 功能句柄	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

IMP_IVS_DestroyChn

【功能】

销毁 IVS 功能句柄对应的通道。

【语法】

int IMP_IVS_DestroyChn(int ChnNum);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

无。

IMP_IVS_RegisterChn

【功能】

注册通道到通道组。

【语法】

int IMP_IVS_RegisterChn(int GrpNum, int ChnNum);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入
ChnNum	IVS 功能对应的通道号	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

无。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

IMP_IVS_UnRegisterChn

【功能】

从通道组注消通道。

【语法】

int IMP_IVS_UnRegisterChn(int ChnNum);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

- 从号为 Grpnum 的通道组中注销号为 Chnnum 通道。

【举例】

无。

IMP_IVS_StartRecvPic

【功能】

通道开始接收图像。

【语法】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

int IMP_IVS_StartRecvPic(int ChnNum);

【形参】

参数名称	描述	输入/输出
GrpNum	通道组号	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

- 通道号为 Chnnum 的 IVS 功能通道开始接收图像做智能分析。

【举例】

无。

IMP_IVS_StopRecvPic

【功能】

通道停止接收图片。

【语法】

int IMP_IVS_StopRecvPic(int ChnNum);

【形参】

参数名称	描述	输入/输出
GrpNum	通道号	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

- 通道号为 Chnnum 的 IVS 功能通道停止接收图像，暂停智能分析。

【举例】

无。

IMP_IVS_PollingResult

【功能】

阻塞判断是否可以获得 IVS 功能已计算出的智能分析结果。

【语法】

int IMP_IVS_PollingResult(int ChnNum, int timeoutMs);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入
timeout	最大等待时间，单位 ms； IMP_IVS_DEFAULT_TIMEOUTMS: 库内部默认的等待时间：0，不等待； >0，用户设定的等待时间	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

- 只有该通道创建时参数 IMPIVSInterface 结构体中 ProcessAsync 函数成员返回 0 时，即实际检测正常返回时，此 Polling 函数才返回成功。

【举例】

无。

IMP_IVS_GetResult

【功能】

获得 IVS 功能计算出的智能分析结果。

【语法】

int IMP_IVS_GetResult(int ChnNum, void **result);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入
resultIVS	功能对应的通道号输出的结果，返回此通道对应的智能分析算法的结果指针，外部客户无需分配空间。	输入

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

- 根据不同 IVS 功能绑定的通道,输出其对应的结果。

【举例】

无。

IMP_IVS_ReleaseResult

【功能】

释放 IVS 功能计算出的结果资源。

【语法】

int IMP_IVS_ReleaseResult(int ChnNum, void *result);

【形参】

参数名称	描述	输入/输出
ChnNum	IVS 功能对应的通道号	输入

【返回值】

返回值	描述
0	成功。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

- 根据不同 IVS 功能绑定的通道,释放其输出的结果资源。

【举例】

无。

IMP_IVS_ReleaseData

【功能】

释放传给 Datacallback 的参数 frame。

【语法】

int IMP_IVS_ReleaseData(void *vaddr);

【形参】

参数名称	描述	输入/输出
vaddr	释放的空间虚拟地址	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

库文件：libimp.a / libimp.so

【注意】

- 必须使用此函数释放传给 Datacallback 的 frame 参数，否则肯定造成死锁。
- 此接口仅供算法提供商使用，算法使用客户无须关注。。

【举例】

无。

IMP_IVS_GetParam

【功能】

获取通道算法参数。

【语法】

int IMP_IVS_GetParam(int chnNum, void *param);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入
param	算法参数虚拟地址指针	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 根据不同 IVS 功能绑定的通道,释放其输出的结果资源。

【举例】

无。

IMP_IVS_SetParam

【功能】

设置通道算法参数。

【语法】

int IMP_IVS_SetParam(int chnNum, void *param);

【形参】

参数名称	描述	输入/输出
GrpNum	IVS 功能对应的通道组号	输入
param	算法参数虚拟地址指针	输入

【返回值】

返回值	描述
0	成功。
-1	失败。

【依赖】

头文件：imp_ivs.h

库文件：libimp.a / libimp.so

【注意】

- 根据不同 IVS 功能绑定的通道,释放其输出的结果资源。

【举例】

无。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.